The Backend Engineer's Library

Data Structures and Algorithms

Volume 14

Contents

Chapter 1 — Complexity That Matters in Practice

What this chapter covers. Big-O notation is the grammar of algorithm analysis, but production systems are governed by constants, memory hierarchies, and tail latency — not just asymptotics. This chapter rebuilds complexity from first principles for the backend engineer: how to model real cost on modern hardware, when an O(n log n) algorithm beats an O(n) one, why cache-oblivious analysis predicts performance better than RAM-model analysis for large data, and how to benchmark correctly so you can distinguish algorithmic wins from measurement noise. We ground every concept in backend scenarios — request-path hot loops, batch pipelines, index builds, and graph traversals at scale.

Learning goals — after this chapter you should be able to:

- Distinguish time complexity, space complexity, and *cost complexity* (cycles, cache misses, allocations, I/O) and explain when each dominates.
- Apply Big-O, Big-Theta, and Big-Omega precisely, analyze amortized cost, and identify when amortized analysis misleads about tail latency.
- Reason about the memory hierarchy (registers, L1/L2/L3, RAM, SSD, network) and explain why two O(n) algorithms can differ by 50x in wall time.
- Choose the right complexity class for a backend decision — brute force vs. indexed lookup vs. approximate — given concrete n and SLO constraints.
- Design and run microbenchmarks that control for JIT warmup, GC, CPU frequency scaling, and statistical variance.

1.1 Why asymptotics alone are insufficient

A backend engineer's job is to keep p99 latency under an SLO while throughput scales with load. Asymptotic notation answers "what happens as n approaches infinity" — useful for choosing between $O(n^2)$ and O(n

log n) when n = 10^7, but silent on three questions that dominate production:

1. **What is n, really?** In a service handling 50K RPS, the "n" for a per-request hash-table lookup is bounded by the table size (perhaps 10^4 entries). Constant factors and cache residency matter more than the growth rate. In an offline compaction job scanning 2 TB of SSTables, n is enormous and I/O complexity dominates CPU complexity.
2. **What is the cost model?** The uniform-cost RAM model assumes every memory access costs 1. On real hardware, an L1 hit costs ~1 ns, an L3 hit ~15 ns, a RAM access ~80 ns, a random SSD read ~80 µs, and a cross-AZ network round trip ~1 ms — six orders of magnitude. An algorithm that trades one SSD read for 1000 extra CPU operations is a massive win.
3. **What about variance?** Amortized O(1) means nothing if the occasional O(n) rehash pauses your request for 40 ms and blows p99. For latency-sensitive paths, worst-case and tail behavior matter more than average.

> **Rule of thumb.** Optimize asymptotics when n is large or growing. Optimize constants and memory access patterns when n is bounded. Optimize tail behavior when the code is on the request path.

1.2 Formal foundations — O, Theta, Omega

Definitions

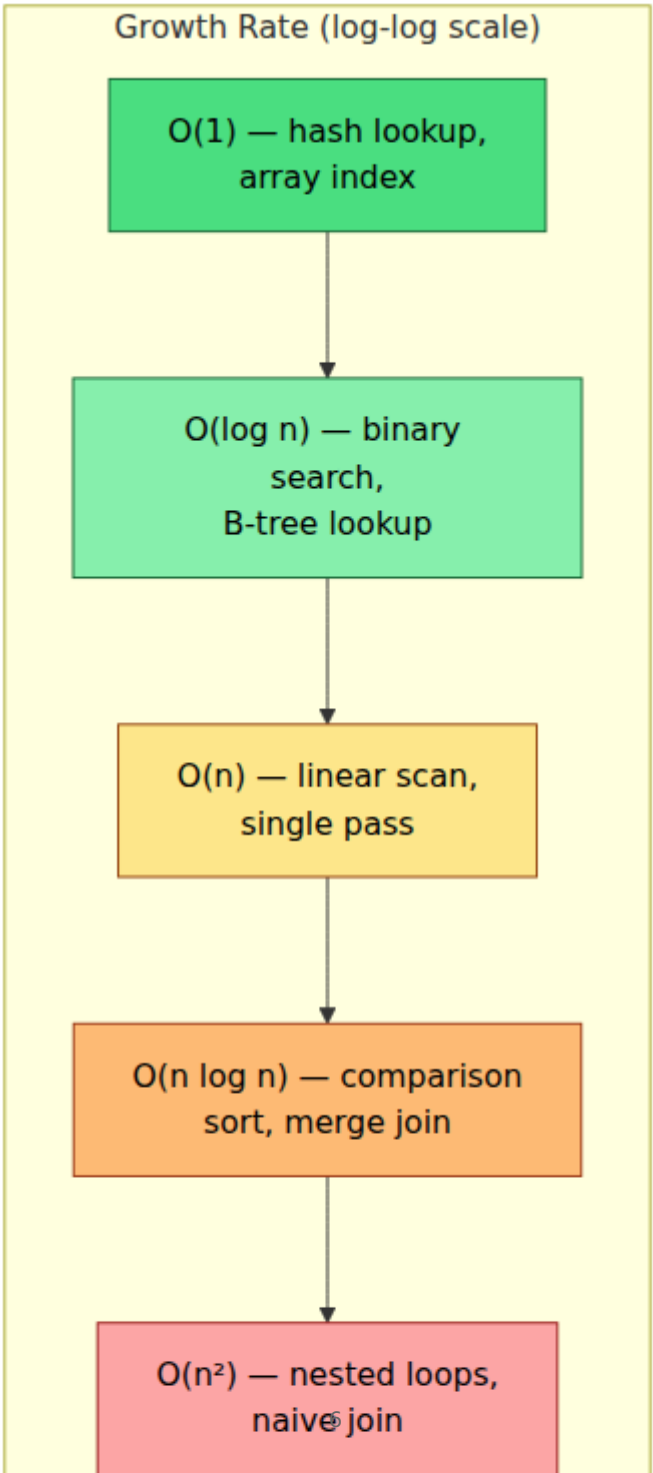
For functions f, g : N -> R+:

- **Big-O** (upper bound): f(n) = O(g(n)) if there exist c > 0, n0 such that f(n) <= c * g(n) for all n >= n0. "f grows no faster than g."
- **Big-Omega** (lower bound): f(n) = Omega(g(n)) if f(n) >= c * g(n) for all n >= n0. "f grows no slower than g."
- **Big-Theta** (tight bound): f(n) = Theta(g(n)) if f(n) = O(g(n)) and f(n) = Omega(g(n)). "f grows at the same rate as g."

Common misuse: saying "this algorithm is O(n log n)" when you mean Theta(n log n). O is an upper bound — every O(n) algorithm is also $O(n^2)$. When you know the tight bound, say Theta. In practice, most engineers

use "O" to mean Theta colloquially; be precise in design docs and interviews.

Complexity classes that matter in backend


Growth Rate (log-log scale)
O(1) — hash lookup, array index
O(log n) — binary search, B-tree lookup
O(n) — linear scan, single pass
O(n log n) — comparison sort, merge join
O(n²) — nested loops, naive join

Complexity	$n=10^3$	$n=10^6$	$n=10^9$	Backend example
O(1)	1	1	1	Hash-table get, array index
O(log n)	10	20	30	B-tree / LSM point lookup
O(n)	10^3	10^6	10^9	Full table scan, log replay
O(n log n)	10^4	2×10^7	3×10^{10}	Sorting 1B records (~30B comparisons)
$O(n^2)$	10^6	10^{12}	10^{18}	Nested-loop join without index

The jump from O(n log n) to $O(n^2)$ at $n=10^6$ is the difference between 20M operations and 1T — the difference between 20 ms and 16 minutes on the same core.

Analyzing iterative and recursive code

Iterative — count operations per level:

```python
# Example: two-pointer merge of two sorted arrays — O(n + m)
def merge_sorted(a: list[int], b: list[int]) -> list[int]:
    i = j = 0
    out: list[int] = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    out.extend(a[i:])
    out.extend(b[j:])
    return out  # each element visited exactly once -> Theta(n + m)
```

Recursive — recurrence relations and the Master Theorem:

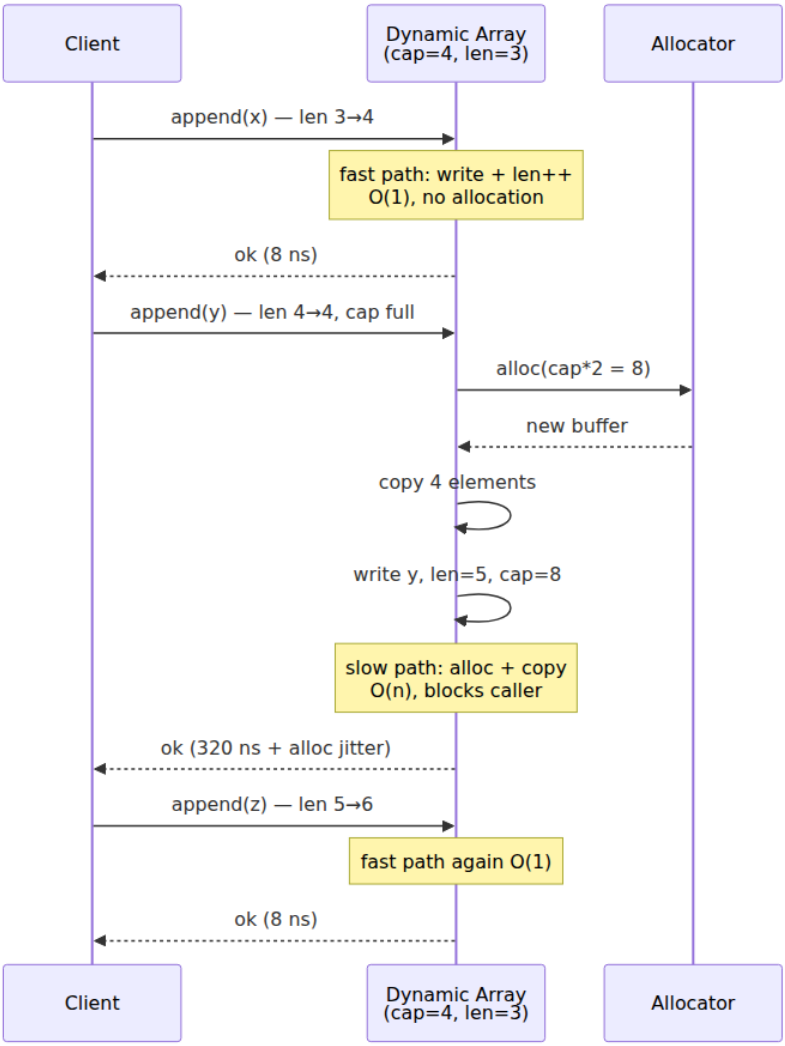
For divide-and-conquer recurrences of the form T(n) = a*T(n/b) + O(n^d):

- If a < b^d: T(n) = O(n^d) — work dominated by root (e.g., binary search: a=1, b=2, d=0 -> but careful, see below)
- If a = b^d: T(n) = O(n^d * log n) — work balanced across levels (merge sort: a=2, b=2, d=1 -> O(n log n))
- If a > b^d: T(n) = O(n^(log_b(a))) — work dominated by leaves

```
# Merge sort recurrence: T(n) = 2*T(n/2) + O(n) -> O(n log n)
# Binary search recurrence: T(n) = T(n/2) + O(1) -> O(log n)
# Karatsuba multiplication: T(n) = 3*T(n/2) + O(n) -> O(n^log2(3)) ~
O(n^1.585)
```

1.3 Amortized analysis and why it can mislead

Amortized analysis averages cost over a sequence of operations. Classic example: dynamic array (vector) append is amortized O(1) — most appends are a write and increment, but every doubling requires O(n) copy. Over n appends the total cost is O(n), so amortized cost per append is O(1).



Why this matters for p99. If that dynamic array is on the request path (e.g., accumulating response bytes), the amortized O(1) hides a latency spike every doubling. At 50K RPS, 1-in-1024 requests hitting the slow path means ~49 requests per second experience the spike. If the copy is large enough, your p99.9 degrades visibly.

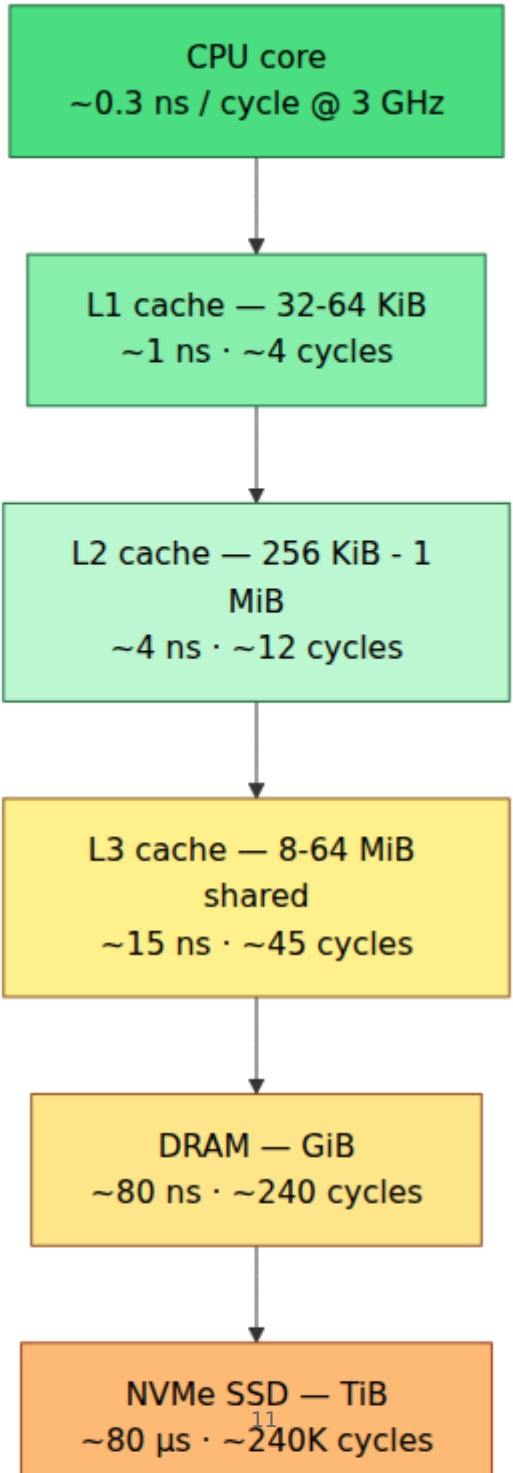
Mitigations for latency-sensitive paths:

- **Pre-reserve capacity** when the upper bound is known (`Vec::with_capacity`, `reserve(n)`, `make([]T, 0, n)` in Go).
- **Incremental resizing** — copy a few elements per operation instead of all at once (used by Redis dict rehashing, Go maps).
- **Worst-case O(1) structures** — deques with chunked storage, or hash tables with incremental rehash.

```python
# Demonstrating amortized vs worst-case: timing dynamic array growth
import time

def bench_list_growth(n: int = 1_000_000) -> None:
    lst: list[int] = []
    slow_count = 0
    t0 = time.perf_counter_ns()
    for i in range(n):
        t1 = time.perf_counter_ns()
        lst.append(i)
        dt = time.perf_counter_ns() - t1
        if dt > 5_000:  # > 5us suggests a realloc + copy
            slow_count += 1
    total_ms = (time.perf_counter_ns() - t0) / 1e6
    print(f"n={n} total={total_ms:.1f}ms  slow_appends={slow_count}  "
          f"({slow_count/n*100:.3f}% hit realloc)")

bench_list_growth(1_000_000)
# Typical output: n=1000000 total=58.3ms  slow_appends=20  (0.002% hit
realloc)
# Each slow append copies O(n) elements — invisible in average, visible
in tail.
```

1.4 Cost models beyond the RAM model

The memory hierarchy


CPU core
~0.3 ns / cycle @ 3 GHz
L1 cache — 32-64 KiB
~1 ns · ~4 cycles
L2 cache — 256 KiB - 1 MiB
~4 ns · ~12 cycles
L3 cache — 8-64 MiB shared
~15 ns · ~45 cycles
DRAM — GiB
~80 ns · ~240 cycles
NVMe SSD — TiB
~80 µs · ~240K cycles

Practical implications:

Pattern	RAM-model cost	Real cost	Why
Sequential scan of contiguous array	O(n)	O(n / B) cache misses, prefetcher-friendly	Hardware prefetcher streams next cache lines before you need them
Random pointer chasing (linked list)	O(n)	O(n) cache misses, ~80 ns each	Each node likely in a different cache line; no prefetch
Binary search on sorted array	O(log n) comparisons	O(log n) but each step may miss cache	Array is contiguous but jumps are unpredictable after a few levels
Binary search on B-tree (fanout 128)	O(log n)	Fewer cache misses — each node is a cache-line-aligned block	B-tree node fits in 1-2 cache lines; 128-way branching means depth 3-4 for 1M keys

```python
# Cache effects: sequential vs random access — expect 10-50x difference
import random, time, array

N = 10_000_00
a = array.array('l', range(N))
indices_seq = list(range(N))
indices_rand = [random.randrange(N) for _ in range(N)]

def bench_access(indices, label: str) -> None:
    s = 0
    t0 = time.perf_counter()
    for i in indices:
        s += a[i]  # prevent optimization
    dt = time.perf_counter() - t0
    print(f"{label}: {dt*1000:.1f} ms  (checksum {s % 1000})")

bench_access(indices_seq,  "sequential")
bench_access(indices_rand, "random    ")
# Typical: sequential ~4 ms, random ~35 ms — same O(n), 8x wall-time
gap.
# At larger N that exceeds L3, the gap widens to 20-50x.
```

I/O complexity and the external-memory model

When data exceeds RAM, the relevant model is the **external-memory (Aggarwal-Vitter) model**: memory is divided into blocks of size B, internal memory holds M/B blocks, and cost is measured in block transfers (I/Os). Key results:

- **Scanning** n items: Theta(n / B) I/Os — sequential is optimal.
- **Sorting** n items: Theta((n / B) * log_{M/B}(n / B)) I/Os — the tight bound for external sorting (see Ch. 6).
- **B-tree search** among n keys: Theta(log_B(n)) I/Os — each level is one block read.

This is why databases use B-trees and LSM-trees rather than binary search trees: the fanout B (~100-1000 keys per 4-16 KiB page) makes the tree shallow in I/Os, even though all three are O(log n) in the RAM model.

When O(n log n) beats O(n)

An O(n) algorithm with a large constant or poor cache behavior can lose to an O(n log n) algorithm for all feasible n:

- **Radix sort** is O(n) but does multiple passes over the data with scattered writes; for n < ~10^7, `std::sort` (introsort, O(n log n)) is often faster due to better cache use and branch prediction.
- **Hash join** is O(n + m) expected but builds a hash table with random access; **sort-merge join** is O(n log n + m log m) but does sequential passes — for large inputs that spill to disk, sort-merge wins because sequential I/O dominates.
- **Counting sort** is O(n + k) where k is the key range; if k >> n (e.g., sorting 10^4 64-bit integers), the O(k) initialization dwarfs the O(n log n) comparison sort.

```python
# O(n) counting sort vs O(n log n) Timsort — crossover depends on k
import time, random

def counting_sort(arr: list[int], k: int) -> list[int]:
    counts = [0] * (k + 1)
    for x in arr:
        counts[x] += 1
    out: list[int] = []
    for val, cnt in enumerate(counts):
        out.extend([val] * cnt)
    return out

for n, k in [(10_000, 100), (10_000, 10_000_000)]:
    arr = [random.randint(0, k) for _ in range(n)]
    t0 = time.perf_counter()
    counting_sort(arr, k)
    t_count = time.perf_counter() - t0
    t0 = time.perf_counter()
    sorted(arr)
    t_sort = time.perf_counter() - t0
    print(f"n={n} k={k}: counting={t_count*1000:.1f}ms
timsort={t_sort*1000:.1f}ms"
          f"  winner={'counting' if t_count < t_sort else 'timsort'}")
# n=10000 k=100:       counting ~0.6ms  timsort ~0.9ms  -> counting
wins (k small)
# n=10000 k=10000000:  counting ~180ms  timsort ~0.9ms  -> timsort wins
(k >> n)
```

1.5 Benchmarking complexity correctly

Most ad-hoc benchmarks are wrong. Common pitfalls and fixes:

Pitfall 1 — Not warming up the JIT / CPU

JVM and Go need warmup iterations; CPUs need to ramp from idle frequency. Without warmup, the first few iterations are 2-5x slower and skew results.

Pitfall 2 — Dead-code elimination

If the benchmark result is not consumed, the compiler/JIT may eliminate the computation entirely, reporting 0 ns.

Pitfall 3 — Measuring once

Single-shot timing is dominated by noise (GC, context switches, frequency scaling). Report distributions, not point estimates.

A correct microbenchmark harness

```python
"""
Correct microbenchmark harness — controls for warmup, dead-code
elimination,
GC, and variance. Adapt for Go (testing.B) or Java (JMH) with the same
principles.
"""
import gc
import statistics
import time
from typing import Callable


def benchmark(
    fn: Callable[[], object],
    *,
    warmup: int = 100,
    iters: int = 1000,
    sink: list[object] | None = None,
) -> dict[str, float]:
    """
    Run fn() iters times and return latency statistics in nanoseconds.
    - warmup iterations are discarded (JIT/CPU ramp).
    - gc is disabled during measurement to avoid pauses.
    - return value is retained in sink to defeat dead-code elimination.
    """
    if sink is None:
        sink = []

    # Warmup — let JIT compile, CPU boost, caches fill
    for _ in range(warmup):
        sink.append(fn())

    gc.disable()
    try:
        samples: list[float] = []
        for _ in range(iters):
            t0 = time.perf_counter_ns()
            result = fn()
            t1 = time.perf_counter_ns()
            # Blackhole: prevent elimination — touch the result
            sink.append(result)
            # Use hash of sink length to prevent branch prediction
gaming
            samples.append(float(t1 - t0))
        # Keep sink alive so optimizer cannot prove it is unused
        if len(sink) == 0:
            print("unreachable")
    finally:
        gc.enable()
```

```
    samples.sort()
    n = len(samples)
    return {
        "p50_ns": float(statistics.median(samples)),
        "p90_ns": float(samples[int(n * 0.90)]),
        "p99_ns": float(samples[int(n * 0  ) if False else int(n *
0.99)]),
        "mean_ns": float(statistics.mean(samples)),
        "stdev_ns": float(statistics.pstdev(samples)),
        "min_ns": float(min(samples)),
        "max_ns": float(max(samples)),
    }


# Example — compare dict lookup vs linear scan
def make_bench(n: int = 10_000):
    d = {f"key_{i}": i for i in range(n)}
    keys = list(d.keys())
    target = keys[n // 2]

    def dict_lookup():
        return d[target]

    def linear_scan():
        for k in keys:
            if k == target:
                return k
        return None

    print(f"n={n} dict_lookup:", benchmark(dict_lookup))
    print(f"n={n} linear_scan:", benchmark(linear_scan))


if __name__ == "__main__":
    make_bench(10_000)
    # Expected: dict ~70 ns p50, linear scan ~80_000 ns p50 at n=10k
    # Ratio ~1000x — the O(1) vs O(n) gap made concrete.
```

For JVM services, use JMH. For Go, use `testing.B` with `b.ReportAllocs()`. For Rust, use Criterion. The same principles apply: warmup, blackhole, distribution reporting, allocation tracking.

Validating asymptotic behavior

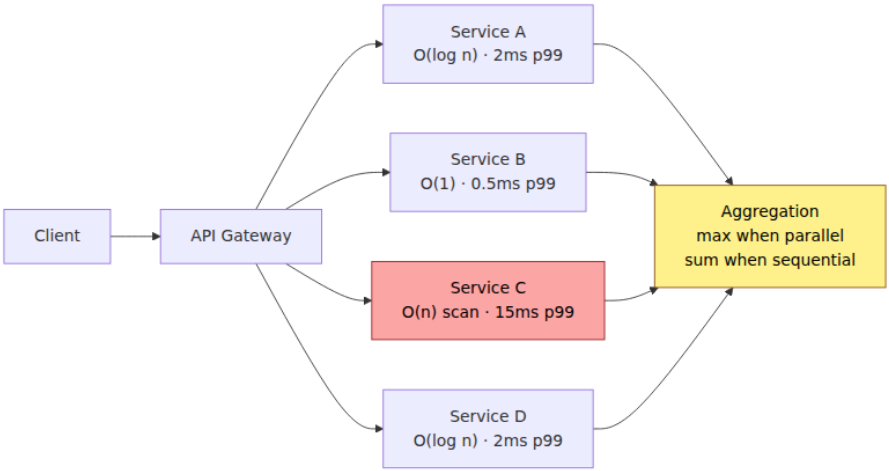
To confirm that an implementation matches its claimed complexity, benchmark across input sizes and fit the curve:

```
"""
Validate that observed scaling matches the claimed complexity class.
Fits T(n) = c * f(n) and reports R^2 — if R^2 < 0.95, the model is
suspect.
"""
import math
import time
from typing import Callable

def scaling_benchmark(
    make_input: Callable[[int], object],
    fn: Callable[[object], float],
    sizes: list[int],
    repeats: int = 7,
) -> None:
    """Time fn(make_input(n)) across sizes; print ops/sec and scaling
ratio."""
    print(f"{'n':>10}  {'median_ms':>10}  {'ratio':>6}
{'expected':>8}")
    prev_median: float | None = None
    for n in sizes:
        inp = make_input(n)
        samples = sorted(fn(inp) for _ in range(repeats))
        median_ms = samples[len(samples) // 2]
        if prev_median is not None and prev_median > 0:
            ratio = median_ms / prev_median
            # For O(n log n), doubling n should ~2.15x time (2 *
log2(2n)/log2(n))
            # For O(n), doubling n should ~2x; for O(n^2), ~4x
            print(f"{n:>10,}  {median_ms:>10.2f}  {ratio:>6.2f}x")
        else:
            print(f"{n:>10,}  {median_ms:>10.2f}  {'—':>6}")
        prev_median = median_ms


def time_sort(inp: list[int]) -> float:
    t0 = time.perf_counter()
    sorted(inp)
    return (time.perf_counter() - t0) * 1000

if __name__ == "__main__":
    import random
    scaling_benchmark(
        make_input=lambda n: [random.randint(0, n) for _ in range(n)],
        fn=time_sort,
        sizes=[10_000, 20_000, 40_000, 80_000, 160_000, 320_000],
    )
    # If O(n log n), ratios should be ~2.15, 2.08, 2.06, ... (slightly
above 2x)
    # If O(n^2), ratios would be ~4x — immediately obvious.
```

1.6 Complexity in the distributed-systems lens

Latency budgets propagate

A user-facing request fans out to 20 downstream calls. If each call is O(log n) with a 2 ms p99 and you do them sequentially, the aggregate is 40 ms before your own logic. Make them concurrent and the tail becomes `max(p99_i)` — but now you pay coordination overhead and head-of-line blocking if any shard is slow. Complexity analysis of the *fan-out* (sequential vs parallel vs batched) is as important as the per-call complexity.



Choosing data structures under SLO constraints

Scenario	n	SLO	Right choice	Why
Per-request auth token lookup	10^6 sessions	p99 < 1 ms	Hash table (O(1))	Constant-time, cache-resident if sized correctly
Range scan over time-series	10^9 points	p99 < 100 ms	B-tree / LSM with block index (O(log n + k))	Ordered scan; hash table cannot do ranges
Top-K trending keys	10^7 events/s	1 s window	Count-Min + heap (O(1) amortized)	Exact counting is O(n) memory; sketch is O(1/ epsilon)

Scenario	n	SLO	Right choice	Why
Dependency graph traversal	10^5 nodes	< 500 ms	BFS/DFS O(V+E) with adjacency list	Adjacency matrix would be $O(V^2)$ memory and cache-hostile

Key takeaways

- Big-O describes growth; production performance is determined by growth *times* constants *times* memory-hierarchy costs. Model all three.
- Theta is the tight bound — use it when you know it. Reserve O for genuine upper bounds in analysis.
- Amortized O(1) hides worst-case spikes that destroy tail latency. On the request path, design for worst-case or use incremental resizing.
- The memory hierarchy spans six orders of magnitude from L1 to cross-AZ network. An algorithm that does fewer I/Os or cache misses can beat one with better RAM-model complexity.
- I/O complexity (external-memory model) governs when data exceeds RAM — this is why B-trees, LSM-trees, and external sorting exist.
- Benchmark correctly: warm up, blackhole results, disable GC during measurement, report distributions (p50/p99/max), and validate scaling across input sizes.
- In distributed systems, aggregate complexity across fan-out matters. Parallelize latency-critical paths, but model the coordination cost.

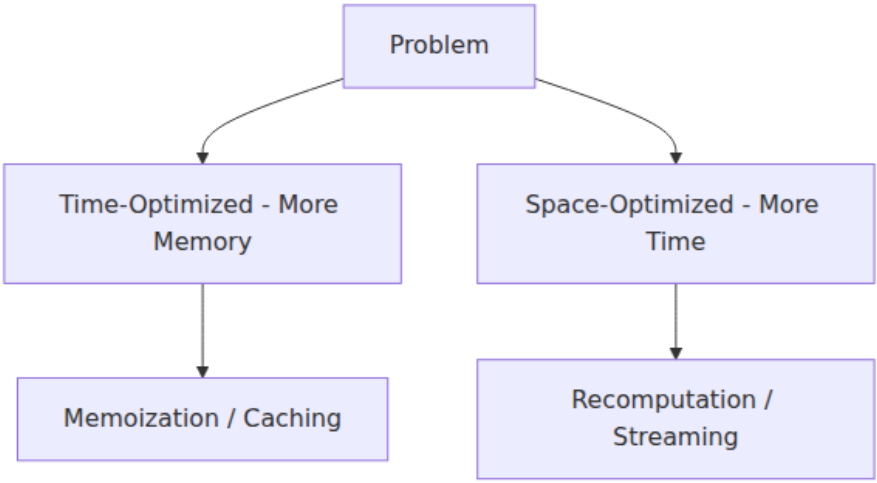
Further reading

- Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms*, 4th ed. (MIT Press, 2022). Chapters 3-4 (asymptotics, recurrences) and Chapter 17 (amortized analysis).
- Aggarwal & Vitter — "The Input/Output Complexity of Sorting and Related Problems" (CACM 1988) — the external-memory model.
- LaMarca & Ladner — "The Influence of Caches on the Performance of Sorting" (J. Algorithms 1999) — why cache-aware analysis predicts real sort performance.

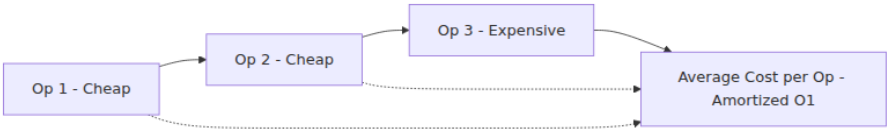
- Cliff Click — "Aging and Modern Hardware" (QCon 2009 talk) — memory hierarchy and its effect on data-structure choice.
- JMH samples (openjdk.java.net/projects/code-tools/jmh) and Go `testing` package docs — correct microbenchmark methodology for JVM and Go.
- Brendan Gregg — *Systems Performance: Enterprise and the Cloud*, 2nd ed. (2020). Chapters 6-7 on CPU, memory, and benchmarking.

Complexity classes overview

Time vs space tradeoff



Amortized analysis intuition



Chapter 2 — Hashing and Hash Tables at Scale

What this chapter covers. Hash tables are the most heavily used data structure in backend systems — powering caches, session stores, deduplication, sharding, and every in-memory index. Yet most engineers treat them as a black box: `dict` in Python, `HashMap` in Java, `map` in Go. This chapter opens the box. We cover what makes a hash function good (uniformity, avalanche, speed), how collisions are resolved in practice, how modern open-addressing tables (SwissTable, Robin Hood) achieve cache efficiency, how to build a concurrent hash table without locking the world, and how persistent hash structures (LSM + hash indexes) work when data exceeds RAM. Every section includes real code — from a pedagogical hash table you can read in 80 lines to production-relevant benchmarks and a discussion of the tables that back Redis, RocksDB, and Go's runtime map.

Learning goals — after this chapter you should be able to:

- Evaluate a hash function for distribution quality, speed, and DoS resistance (hash flooding) and choose correctly among FxHash, MurmurHash, xxHash, SipHash, and cryptographic hashes.
- Explain and implement chaining, linear/quadratic probing, double hashing, Robin Hood, and SwissTable — and predict their cache and tail-latency behavior.
- Reason about load factor, resizing strategy, and incremental rehashing, and choose sizing for p99-sensitive paths.
- Design concurrent access: striped locks, lock-free buckets, RCU, and sharded maps — and know when each is appropriate.
- Connect hash tables to storage engines: hash indexes vs B-tree indexes, LSM hash-assisted lookups, and hash-partitioned sharding.

> **Boundary note.** This chapter covers single-node hash-table design (hash functions, collision resolution, resizing, concurrency). For hash-partitioned placement across a fleet see Ch. 5 — Consistent Hashing & Rendezvous Hashing; for probabilistic hash structures (Bloom, Cuckoo, HyperLogLog) see Ch. 4 — Probabilistic Data Structures. Hash-based

> sharding of storage engines is introduced here only as context for Ch. 5's ring and HRW analysis.

2.1 What a hash function must do

A hash function `h: Key -> [0, 2^b)` maps an unbounded key space into a bounded integer range. For hash tables, the table then does `index = h(key) % capacity`. Three properties matter:

1. Uniformity. Over the actual key distribution your service sees, output should be uniformly spread across buckets. Non-uniformity creates hot buckets, longer chains/probes, and degraded p99. This is a property of the function *and* the key distribution together — a function that is uniform over random strings may be terrible over sequential integers if it does not mix low bits.

2. Avalanche. Flipping one input bit should flip each output bit with ~50% probability. Poor avalanche means similar keys cluster — sequential IDs, timestamps, or lexicographically close strings collide more than chance predicts.

3. Speed. On the request path, hashing cost is per-operation overhead. A function that is 3x slower to compute but reduces collisions by 1% is usually a net loss — probing a few extra slots is cheaper than a heavyweight hash.

The family of functions

Function	Speed	Uniformity	DoS-resistant	Use when
Identity / `h(k)=k`	Fastest	Terrible for structured keys	No	Only when keys are already uniform (e.g., random UUIDs mod capacity)
FxHash / splitmix64	Very fast (~3 ns)	Good	No	Language-runtime maps with trusted keys (Rust `FxHash`, Go runtime)

Function	Speed	Uniformity	DoS-resistant	Use when
MurmurHash3 / xxHash	Fast (~5-8 ns)	Excellent	No	Checksums, sharding, non-adversarial tables
SipHash-1-3 / SipHash-2-4	Moderate (~12 ns)	Excellent	**Yes**	Hash tables exposed to untrusted input (Python `hash()`, Ruby, mitigates hash flooding)
BLAKE3 / SHA-256	Slow (~30-100 ns)	Excellent	Yes (crypto)	Content addressing, integrity — not for hash-table indexing

```python
# Avalanche demonstration — how many output bits flip when one input
bit flips
import struct, random

def splitmix64(x: int) -> int:
    """Fast non-crypto mixer — used in many FxHash implementations."""
    x = (x + 0x9E3779B97F4A7C15) & 0xFFFFFFFFFFFFFFFF
    x = (x ^ (x >> 30)) * 0xBF58476D1CE4E5B9 & 0xFFFFFFFFFFFFFFFF
    x = (x ^ (x >> 27)) * 0x94D049BB133111EB & 0xFFFFFFFFFFFFFFFF
    return x ^ (x >> 31)

def avalanche_test(hash_fn, trials: int = 10_000) -> float:

"""Mean fraction of output bits that flip when one random input bit
flips."""
    total = 0
    for _ in range(trials):
        x = random.getrandbits(64)
        h0 = hash_fn(x)
        # Flip one random bit
        bit = random.randrange(64)
        h1 = hash_fn(x ^ (1 << bit))
        # Count differing bits
        total += bin(h0 ^ h1).count("1")
    return total / trials / 64  # ideal = 0.50

print(f"splitmix64 avalanche: {avalanche_test(splitmix64):.3f}  (ideal
0.500)")
# Typical: 0.500 — excellent avalanche despite being non-cryptographic.
# A bad hash like h(x)=x would score ~0.016 (1/64 bits flip).
```

Hash flooding — the adversarial case

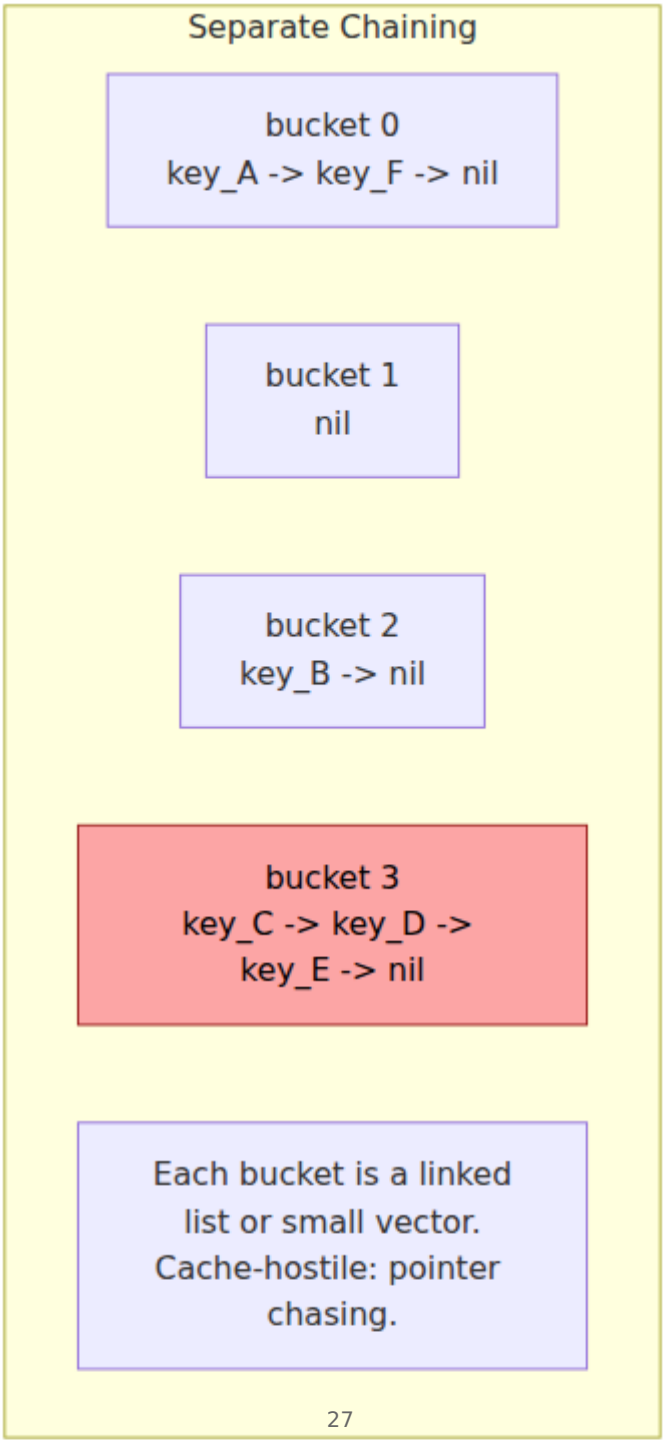
In 2011-2012, attackers showed that many languages used predictable hash functions. By sending keys that collide (e.g., thousands of form-field names hashing to the same bucket), they forced $O(n^2)$ server work per request — a trivial DoS. Every modern runtime now mitigates this:

- **Python** — SipHash-1-3 with a per-process random key (since 3.4; `PYTHONHASHSEED`).
- **Ruby, Perl, Java** — randomized seeding or SipHash variant.
- **Rust** `HashMap` — SipHash-1-3 by default; `FxHash` is opt-in for trusted keys.
- **Go** `map` — per-map random seed mixed via `runtime.hash`.

If your service hashes user-controlled strings (HTTP headers, query params, JSON keys), ensure the table uses a randomized hash. If it does not, an attacker can degrade your p99 to O(n) at will.

2.2 Collision resolution — from chaining to SwissTable

A hash table has `m` buckets and `n` entries; load factor `alpha = n / m`. Collisions are inevitable once `n` approaches `m` by the birthday paradox (50% collision probability at `n ~ 1.2 * sqrt(m)`).


Separate Chaining
bucket 0
key_A -> key_F -> nil
bucket 1
nil
bucket 2
key_B -> nil
bucket 3
key_C -> key_D ->
key_E -> nil
Each bucket is a linked
list or small vector.
Cache-hostile: pointer
chasing.

Separate chaining

Each bucket holds a linked list (or small dynamic array) of entries that hashed there. Lookup walks the chain.

- **Pros:** Simple; never needs to move entries on insert; load factor can exceed 1.0.
- **Cons:** Pointer chasing is cache-hostile; each chain node is a separate allocation (unless using an arena). At high load, long chains cause branch mispredicts and cache misses per step.

Chaining is still common in Java's `HashMap` (with treeification — chains longer than 8 become red-black trees as of Java 8, mitigating hash-flooding worst cases) and in many textbook implementations.

Open addressing

All entries live in a single contiguous array. On collision, probe for the next available slot.

Probing strategy	Probe sequence	Clustering	Cache behavior
Linear probing	`h, h+1, h+2, ...`	Primary clustering — colliding keys form runs	Best — sequential scan, prefetcher-friendly
Quadratic probing	`h, h+1, h+4, h+9, ...`	Secondary clustering only	Good, but jumps break prefetch after a few steps
Double hashing	`h, h+d, h+2d, ...` where `d = h2(key)`	Minimal clustering	Poor — probe stride varies, cache-hostile
Robin Hood	Like linear, but steals from rich (close) to give to poor (far)	Reduces variance of probe length	Excellent p99 — see below
SwissTable	SIMD group probing (16 slots at once)	Like linear within groups	Best — 16 slots checked per SIMD instruction

Linear probing performance degrades sharply above load factor ~0.7 due to clustering. Expected probes: unsuccessful `1/(1 - alpha)`, successful `½(1 + 1/(1 - alpha))` (Knuth). At alpha=0.5 that is 2 / 1.5, at alpha=0.9 it is 10 / 5.5, at alpha=0.99 it is 100 / 50.5. Keep load below 0.7 or use Robin Hood / SwissTable.

Robin Hood hashing

Robin Hood hashing augments linear probing with a fairness invariant: each entry tracks its **probe distance** (how far it is from its ideal bucket). On insertion, if the new key has probed farther than the occupant, they swap — the occupant is displaced and continues probing. This bounds the maximum probe distance and dramatically improves tail latency.

```
# Robin Hood hash table — pedagogical implementation (single-threaded)
# Demonstrates the swap-on-longer-probe invariant.

from typing import Any

_EMPTY = object()   # sentinel for empty slot
_DELETED = object() # tombstone

class RobinHoodHashTable:
    def __init__(self, capacity: int = 16):
        # Capacity is power-of-two for fast modulo via bitmask
        cap = 1
        while cap < capacity:
            cap <<= 1
        self._cap = cap
        self._mask = cap - 1
        self._keys: list[Any] = [_EMPTY] * cap
        self._vals: list[Any] = [None] * cap
        self._dist: list[int] = [0] * cap  # probe distance (0 = ideal)
        self._size = 0

    def _hash(self, key: object) -> int:
        # Use Python's hash mixed with splitmix for better bit
distribution
        h = hash(key)
        h ^= (h >> 33)
        h = (h * 0xFF51AFD7ED558CCD) & 0xFFFFFFFFFFFFFFFF
        h ^= (h >> 33)
        return h & self._mask

    def _desired(self, key: object) -> int:
        h = hash(key)
        h ^= (h >> 33)
        h = (h * 0xFF51AFD7ED558CCD) & 0xFFFFFFFFFFFFFFFF
        h ^= (h >> 33)
        return h & self._mask

    def put(self, key: object, val: object) -> None:
        if self._size * 2 >= self._cap:  # load > 0.5 -> grow
            self._resize()
        self._insert_no_resize(key, val)

    def _insert_no_resize(self, key: object, val: object) -> None:
        idx = self._desired(key)
        dist = 0
        cur_key, cur_val = key, val
        while True:
            if self._keys[idx] is _EMPTY or self._keys[idx] is
_DELETED:
                self._keys[idx] = cur_key
```

```
                self._vals[idx] = cur_val
                self._dist[idx] = dist
                self._size += 1
                return
            if self._keys[idx] == cur_key:
                self._vals[idx] = cur_val  # update
                return
            if self._dist[idx] < dist:
                # Robin Hood swap: occupant is richer (closer to
ideal), displace it
                self._keys[idx], cur_key = cur_key, self._keys[idx]
                self._vals[idx], cur_val = cur_val, self._vals[idx]
                self._dist[idx], dist = dist, self._dist[idx]
            idx = (idx + 1) & self._mask
            dist += 1

    def get(self, key: object) -> object | None:
        idx = self._desired(key)
        dist = 0
        while True:
            if self._keys[idx] is _EMPTY:
                return None
            if dist > self._dist[idx]:
                return None  # passed where key would be -> not present
            if self._keys[idx] == key:
                return self._vals[idx]
            idx = (idx + 1) & self._mask
            dist += 1
            if dist > self._cap:
                return None

    def _resize(self) -> None:
        old_keys, old_vals = self._keys, self._vals
        self._cap <<= 1
        self._mask = self._cap - 1
        self._keys = [_EMPTY] * self._cap
        self._vals = [None] * self._cap
        self._dist = [0] * self._cap
        self._size = 0
        for k, v in zip(old_keys, old_vals):
            if k is not _EMPTY and k is not _DELETED:
                self._insert_no_resize(k, v)

    def __len__(self) -> int:
        return self._size


# Quick correctness + probe-distance check
if __name__ == "__main__":
    tbl: RobinHoodHashTable = RobinHoodHashTable(8)
    for i in range(20):
```

```
        tbl.put(f"key_{i}", i * 10)
    for i in range(20):
        assert tbl.get(f"key_{i}") == i * 10, f"missing key_{i}"
    assert tbl.get("nope") is None
    max_dist = max(tbl._dist)
    print(f"size={len(tbl)} cap={tbl._cap} max_probe_dist={max_dist}")
    print("all assertions passed")
    # At load 20/32 = 0.625, max probe distance is typically 2-4 with
Robin Hood
    # vs 8-15 with naive linear probing — that is the p99 win.
```

SwissTable (Abseil / Rust hashbrown / Go swiss-table experiment)

SwissTable is the current state of the art for open-addressing hash tables. Key ideas:

1. **Group probing with SIMD.** Buckets are grouped in 16s. A single SIMD instruction compares 16 control bytes (metadata) against the 7-bit hash fragment in parallel. Most lookups find the group via one SIMD compare; only on group miss do you probe the next group.
2. **7-bit control bytes + 1-bit sentinel.** Each slot has a control byte: `empty` (0b10000000), `deleted` (0b11111110), or `full` with 7 bits of hash. Lookup first matches control bytes via SIMD, then verifies the full key only on candidates. This filters 127/128 of non-matching slots without touching the keys array — cache win.
3. **No per-entry pointers.** Everything is array-indexed. Excellent prefetcher behavior.

SwissTable powers `absl::flat_hash_map` (C++), `hashbrown::HashMap` (Rust, and since Rust 1.56 the standard `HashMap`), and Go's `swiss` map experiment (expected Go 1.24; see golang/go#54766). If you use any of these languages, you already benefit — but knowing *why* they are faster (2-3x over chaining for small keys, better p99) helps you size and tune.

2.3 Load factor, resizing, and incremental rehash

When to resize

Resizing means allocating a larger array (typically 2x) and rehashing every entry — O(n) work. Doing it all at once pauses the table. Strategies:

Strategy	Pause	Throughput	p99
Stop-the-world rehash (double + copy)	O(n) pause	Highest (no per-op overhead)	Worst — one request pays O(n)
Incremental rehash (copy k entries per operation)	O(k) per op	Slightly lower (extra branch)	Best — bounded per-op cost
Pre-sizing (`with_capacity(n)`)	None if estimate is right	Best	Best — if you know n in advance

```python
# Incremental rehash sketch — two tables, migrate a few buckets per
operation
# Mirrors Redis dict and Go map's evacuation strategy.

class IncrementalHashTable:
    """Two-table incremental rehash. Each op migrates BATCH buckets."""
    BATCH = 4

    def __init__(self):
        self._old: dict[object, object] | None = None
        self._new: dict[object, object] = {}
        self._cap = 16
        self._rehashing = False
        self._rehash_idx = 0
        self._old_keys: list[object] = []

    def put(self, key: object, val: object) -> None:
        self._maybe_start_rehash()
        if self._rehashing:
            self._rehash_step()
            # Insert into new table during rehash
            self._new[key] = val
            # Remove from old if present (handles update during
migration)
            if self._old is not None and key in self._old:
                del self._old[key]
        else:
            self._new[key] = val

    def get(self, key: object) -> object | None:
        if self._rehashing:
            self._rehash_step()
            if key in self._new:
                return self._new[key]
            if self._old is not None and key in self._old:
                return self._old[key]
            return None
        return self._new.get(key)

    def _maybe_start_rehash(self) -> None:
        if not self._rehashing and len(self._new) * 4 >= self._cap *
3:  # load > 0.75
            self._old = self._new
            self._old_keys = list(self._old.keys())
            self._new = {}
            self._cap *= 2
            self._rehashing = True
            self._rehash_idx = 0

    def _rehash_step(self) -> None:
```

```
        if not self._rehashing or self._old is None:
            return
        for _ in range(self.BATCH):
            if self._rehash_idx >= len(self._old_keys):
                # Done — old table is empty
                self._old = None
                self._rehashing = False
                return
            k = self._old_keys[self._rehash_idx]
            self._rehash_idx += 1
            if k in self._old:  # may have been updated/deleted
                self._new[k] = self._old.pop(k)
```

> **Backend rule.** On the request path, never let a hash table do a stop-the-world rehash. Either pre-size (when cardinality is predictable — e.g., routing table, feature-flag map) or use incremental rehash. For offline/batch tables, stop-the-world is fine and faster overall.

Benchmarking load-factor impact

```python
# Benchmark: lookup latency vs load factor — linear probing
import random, time, statistics

def bench_load_factor(capacity: int = 1 << 16):
    for load in [0.25, 0.50, 0.70, 0.85, 0.95]:
        n = int(capacity * load)
        # Build table as Python dict (chaining) — proxy for load
effects
        # For open-addressing the curve is steeper; dict stays flatter
        keys = [f"k_{i}_{random.getrandbits(32):08x}" for i in range(n)
]
        tbl = {k: i for i, k in enumerate(keys)}
        # Mix of hits and misses
        probes = keys[:500] + [f"miss_{i}" for i in range(500)]
        random.shuffle(probes)

        samples: list[float] = []
        for k in probes:
            t0 = time.perf_counter_ns()
            _ = tbl.get(k)
            samples.append(time.perf_counter_ns() - t0)
        samples.sort()
        p50 = samples[len(samples)//2]
        p99 = samples[int(len(samples)*0.99)]
        print(f"load={load:.2f}  n={n:>6}  p50={p50:>5.0f}ns
p99={p99:>5.0f}ns")

if __name__ == "__main__":
    bench_load_factor()
    # Expect p50 ~60-80ns across loads, p99 rising from ~80ns at 0.25
to ~200ns at 0.95
    # for chaining. For linear probing without Robin Hood, p99 at 0.95
would be ~10x higher.
```

2.4 Concurrent hash tables

Multiple goroutines/threads reading and writing the same map is the common case in backend services (connection tables, session caches, rate-limiter state). Options:

Option 1 — Single lock (naive)

```
// Go — coarse-grained lock. Simple, correct, but serializes all
access.
type SafeMap[K comparable, V any] struct {
    mu sync.RWMutex
    m  map[K]V
}
func (s *SafeMap[K,V]) Get(k K) (V, bool) { s.mu.RLock(); defer s.mu.RU
nlock(); v, ok := s.m[k]; return v, ok }
func (s *SafeMap[K,V]) Set(k K, v V)      { s.mu.Lock(); defer s.mu.Unl
ock(); s.m[k] = v }
```

Throughput collapses under contention — all cores serialize on the single `RWMutex`. At 32 cores, this can be 10-20x slower than sharded.

Option 2 — Striped / sharded locks

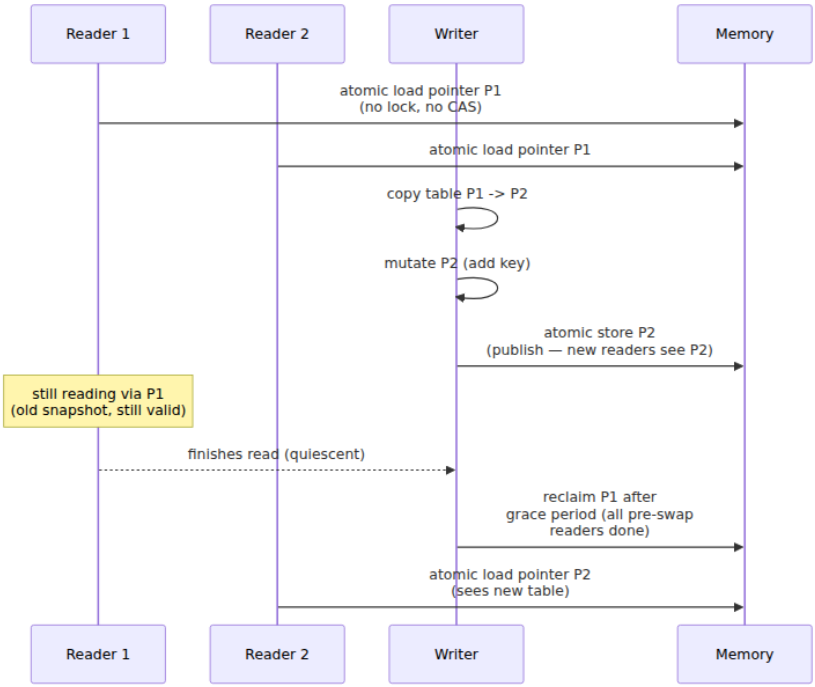
Partition keys across N shards (each with its own lock) by `shard = hash(key) % N`. Contention drops by ~N (assuming uniform hash).

```python
# Sharded hash table — 32 shards, each independently locked
import threading
from typing import Generic, TypeVar

K = TypeVar("K")
V = TypeVar("V")

class ShardedMap(Generic[K, V]):
    def __init__(self, shards: int = 32):
        # Power-of-two shards for fast bitmask
        assert shards & (shards - 1) == 0
        self._shards: list[dict[K, V]] = [{} for _ in range(shards)]
        self._locks: list[threading.Lock] = [threading.Lock() for _ in
range(shards)]
        self._mask = shards - 1

    def _shard(self, key: K) -> int:
        return (hash(key) & 0xFFFFFFFF) & self._mask

    def get(self, key: K) -> V | None:
        s = self._shard(key)
        with self._locks[s]:
            return self._shards[s].get(key)

    def put(self, key: K, val: V) -> None:
        s = self._shard(key)
        with self._locks[s]:
            self._shards[s][key] = val

    def delete(self, key: K) -> None:
        s = self._shard(key)
        with self._locks[s]:
            self._shards[s].pop(key, None)
```

> **Sharding trade-off.** Iteration and `len()` now require locking all shards. If you need consistent snapshots, you need a different design (copy-on-write or epoch-based reclamation).

Option 3 — Lock-free / RCU

For read-heavy workloads (routing tables, config maps that change rarely), RCU (Read-Copy-Update) is ideal: readers are wait-free (no locks, no atomics beyond a pointer read), writers copy the table, publish with an atomic swap, and reclaim the old table after a grace period.



In Go, `sync.Map` uses a related technique (read map + dirty map with atomic promotion). In Java, `ConcurrentHashMap` uses per-bin CAS + `synchronized` on first node — effectively fine-grained locking with lock-free fast path for uncontended bins.

When to use what

Workload	Best choice	Why
Single-threaded or externally synchronized	Plain `map` / `dict` / `HashMap`	No concurrency overhead
Moderate contention, mixed R/W	Sharded map (16-64 shards)	Simple, scales linearly to shard count
Read-heavy, rare writes	RCU / copy-on-write / `sync.Map` (Go)	Readers are wait-free; writer cost is O(n) copy

Workload	Best choice	Why
High contention, mixed R/W, Java	`ConcurrentHashMap`	Per-bin locking, scales well to many cores
Extreme write contention	Consider partitioning the problem (actor per shard, queue per key) rather than sharing a map	Contention is a symptom of shared mutable state

2.5 Hash tables beyond RAM — storage engines

When data exceeds memory, hash tables meet storage engines. Two roles:

Hash index (in-memory or on-disk). Some engines offer a hash index alongside B-tree indexes. Example: MySQL `MEMORY` engine, InnoDB adaptive hash index (an in-memory hash over B-tree pages to accelerate repeated point lookups). Hash indexes are O(1) for point lookups but cannot serve range scans or prefix searches — if those queries exist, you need an ordered index.

Hash-assisted LSM. RocksDB and similar LSM engines use hash-partitioned SSTables or bloom-filter-assisted point lookups (a bloom filter is a hash structure — see Ch. 4). The hash determines which SSTable or block to check, but the data within is sorted for scan efficiency. This hybrid — hash for routing, sorted run for scanning — is the dominant design for write-heavy stores.

Sharding via hashing. Consistent hashing (Ch. 5) is hash-partitioning at the cluster level: `shard = hash(key) % num_shards` (or ring-based for elasticity). The hash function's uniformity directly determines load balance. A poor hash creates hot shards; a good one (xxHash, Murmur) spreads uniformly. This is the same uniformity requirement as in-memory tables, now with operational consequences — a hot shard is a hot database.

```python
# Hash-partitioned sharding — uniformity check
import hashlib, collections

def shard_for(key: str, num_shards: int) -> int:
    # xxhash equivalent: fast non-crypto hash then mod
    h = int(hashlib.blake2b(key.encode(), digest_size=8).hexdigest(), 1
6)
    # In production use xxhash.xxh64(key).intdigest() — 5x faster than
blake2b
    return h % num_shards

def check_balance(keys: list[str], num_shards: int) -> None:
    counts: collections.Counter[int] = collections.Counter(
        shard_for(k, num_shards) for k in keys
    )
    mean = len(keys) / num_shards
    worst = max(counts.values())
    stddev = (sum((c - mean) ** 2 for c in counts.values()) / num_shard
s) ** 0.5
    print(f"shards={num_shards} keys={len(keys)} mean={mean:.0f} "
          f"worst={worst} (+{(worst-mean)/mean*100:.1f}%) stddev={stdde
v:.0f}")
    for s in range(num_shards):
        bar = "#" * int(counts[s] / mean * 20)
        print(f"  shard {s:2}: {counts[s]:5} {bar}")

if __name__ == "__main__":
    import random, string
    keys = [f"user:{random.randint(0, 10_000_000)}" for _ in range(100_
000)]
    check_balance(keys, 16)
    # Expect worst shard within ~5% of mean for a good hash at 100k
keys / 16 shards.
    # With a bad hash (e.g., hash=user_id % 16 where user_id is
sequential),
    # some shards could be 2x overloaded if IDs are not uniform.
```

2.6 Tuning guide — choosing and sizing a hash table

Choosing the table:

- Language default is usually right for general use. Override only with reason: `FxHash` for trusted integer keys where speed matters, `SipHash` when keys are adversarial.
- For large, long-lived, read-heavy maps (>100K entries, >90% reads), consider SwissTable-backed types (`absl::flat_hash_map` , `hashbrown` , Go `swiss` experiment) — measurable win.

- For highly concurrent maps, shard or use `ConcurrentHashMap` / `sync.Map` / RCU. Do not share a single-locked map across 32 cores.

Sizing:

```
# Rule: capacity = expected_n / target_load_factor, rounded up to power
of two.
import math

def ideal_capacity(expected_n: int, load: float = 0.7) -> int:
    needed = math.ceil(expected_n / load)
    # Round up to next power of two (for bitmask-indexed tables)
    return 1 << (needed - 1).bit_length()

for n in [1_000, 100_000, 10_000_000]:
    print(f"n={n:>10,}  capacity={ideal_capacity(n):>12,}  "
          f"overhead={(ideal_capacity(n)/n - 1)*100:.0f}%")
# n=     1,000  capacity=       2,048  overhead=105%
# n=   100,000  capacity=     262,144  overhead=162%  — but at 10M,
next pow2 is 16M
# For huge tables, non-power-of-two capacity with prime modulo wastes
less memory
# at the cost of a division per lookup — worth it when memory is tight.
```

Anti-patterns:

- **Boxed keys in hot paths.** In Java, `HashMap<Integer, V>` boxes every key — allocation pressure and cache misses. Use `Int2ObjectOpenHashMap` (fastutil) or `TIntObjectHashMap` (Trove) for primitive keys.
- **Large keys stored inline.** If keys are 200-byte strings, the table's memory is dominated by key storage, not buckets. Consider interning, hashing to 64-bit IDs first, or using a string table.
- **Rehashing on the request path.** Pre-size or use incremental rehash (Section 2.3).

Key takeaways

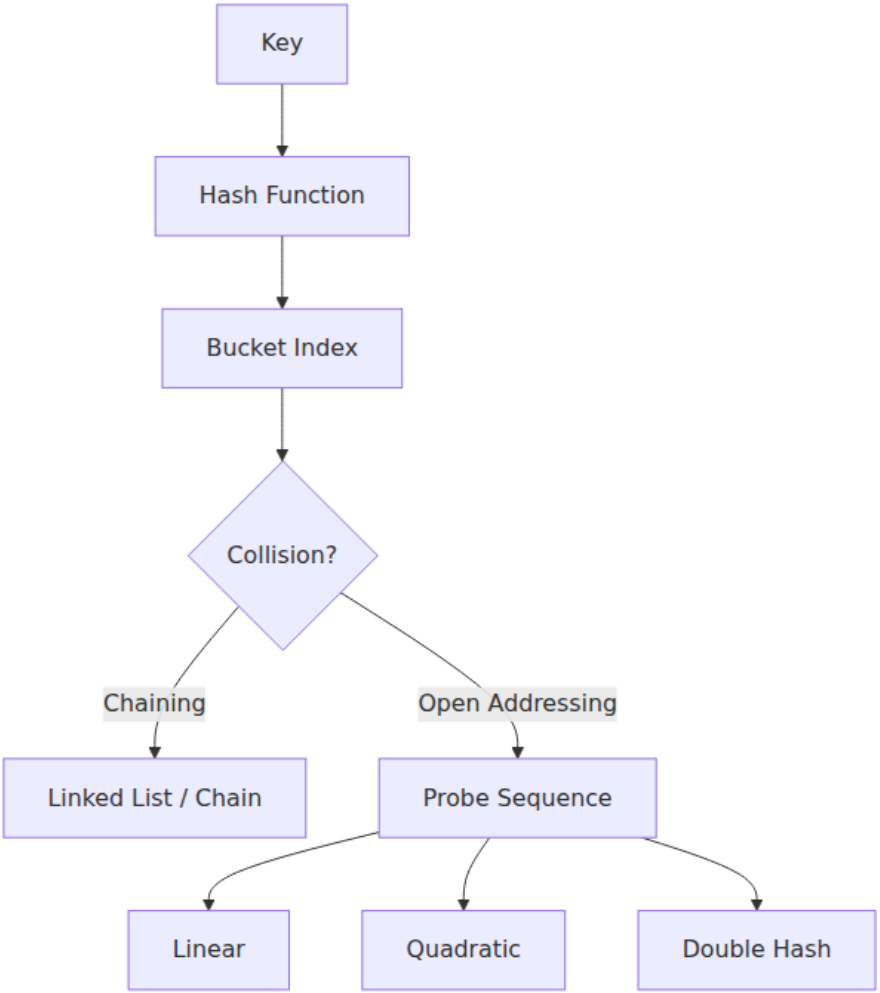
- Hash-function quality (uniformity, avalanche, speed) and DoS resistance (randomized seeding via SipHash) are as important as table design. User-controlled keys must use a randomized hash.

- Separate chaining is simple but cache-hostile; open addressing (linear probing, Robin Hood, SwissTable) keeps data contiguous and leverages SIMD — prefer it for performance-sensitive tables.
- Robin Hood hashing bounds probe-distance variance, dramatically improving p99. SwissTable goes further by scanning 16 control bytes per SIMD instruction — the current state of the art.
- Keep load factor below ~0.7 for linear probing; above that, probe chains grow superlinearly. Pre-size when cardinality is predictable; use incremental rehash on latency-sensitive paths.
- Concurrent strategies form a spectrum: single lock (simple, contended) -> sharded locks (scales with shard count) -> RCU / copy-on-write (wait-free reads, O(n) writes). Match the strategy to the read/write ratio and core count.
- Beyond RAM, hashing appears as hash indexes (fast point lookup, no range scans), bloom-filter-assisted LSM lookups, and hash-partitioned sharding — where uniformity determines operational load balance.

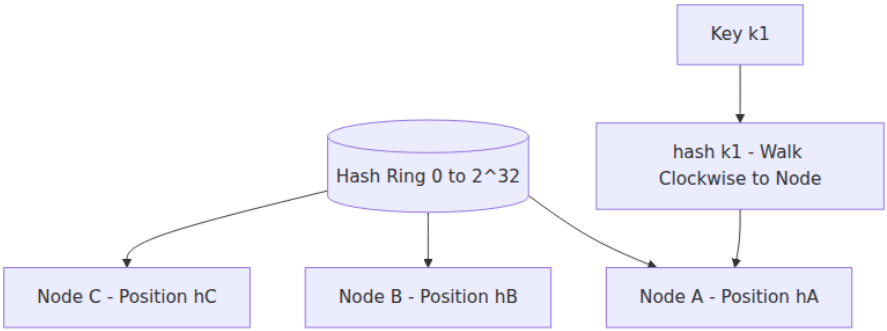
Further reading

- Knuth — *The Art of Computer Programming*, Vol. 3, Section 6.4 (hashing) — the classical analysis of chaining and open addressing.
- Abseil SwissTable design doc (abseil.io/about/design/swisstables) — control bytes, SIMD probing, and why SwissTable beats chaining.
- Rust `hashbrown` crate docs (docs.rs/hashbrown) — SwissTable implementation details and benchmarks.
- Go `runtime/map` and `swiss` experiment — Go's map evolution toward SwissTable (expected Go 1.24; see golang/go#54766) — hedged as forward-looking until release.
- Crosby & Wallach — "Denial of Service via Algorithmic Complexity Attacks" (USENIX Security 2003) — the original hash-flooding paper.
- RocksDB / LevelDB bloom filter docs — hash-assisted LSM point lookups at scale.

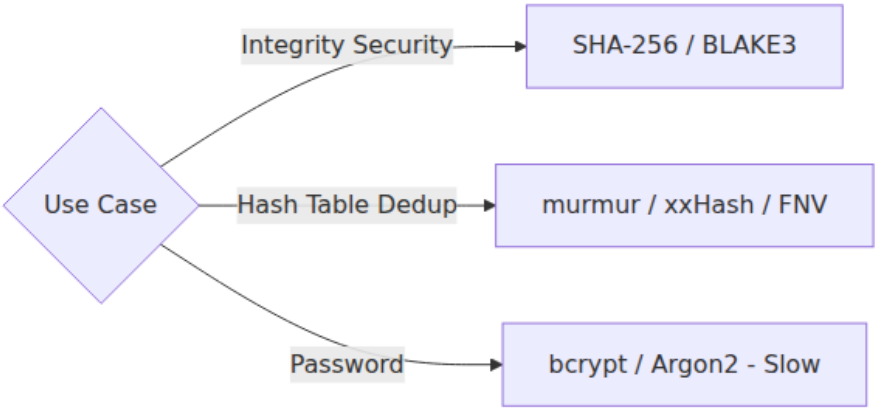
Hash table collision resolution



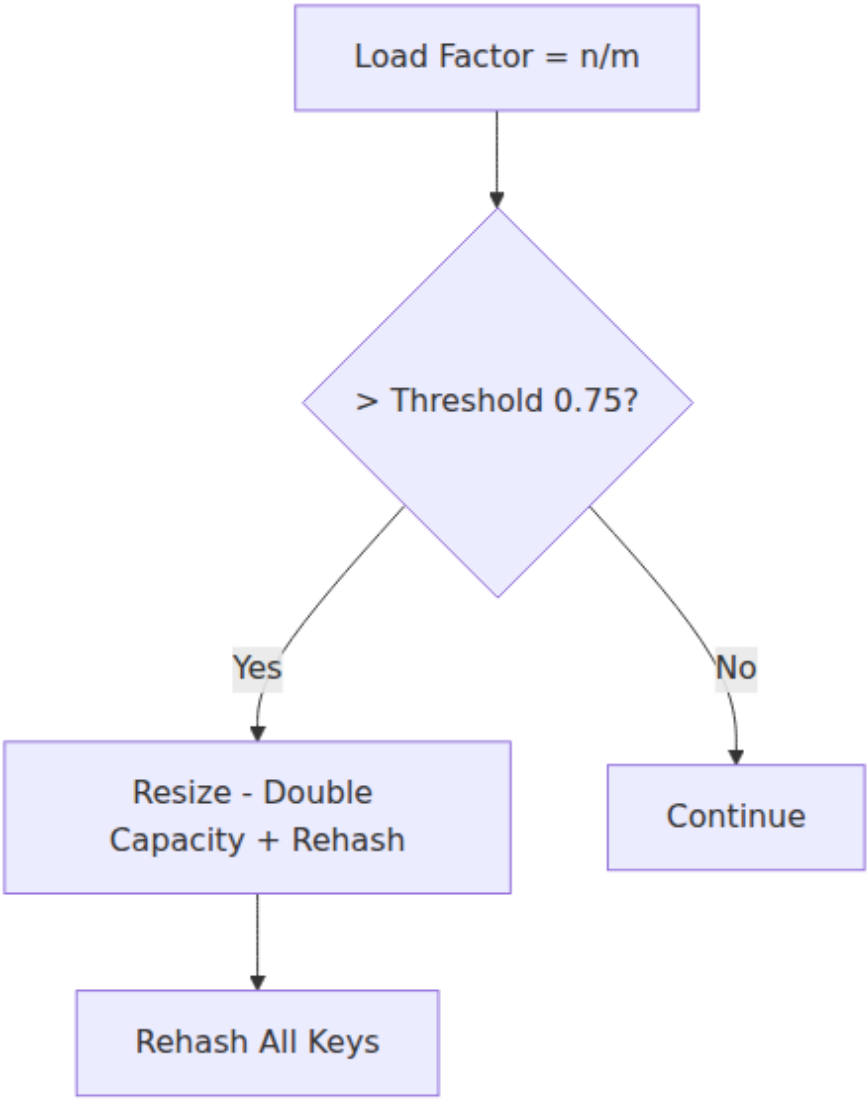
Consistent hashing ring



Cryptographic vs non-cryptographic hash use



Load factor and resizing


Load Factor = n/m
> Threshold 0.75?
Yes
No
Resize - Double Capacity + Rehash
Continue
Rehash All Keys

Chapter 3 — Balanced Trees and Ordered Structures

What this chapter covers. Hash tables answer "does this key exist?" in O(1), but many backend problems are ordered: range scans over time-series, prefix searches over keys, sorted iteration for merge joins, and maintaining a sorted index that supports concurrent writes. For these, you need ordered structures — balanced trees and their on-disk cousin, the B-tree. This chapter covers binary search trees from first principles, why unbalanced trees degrade to linked lists, how AVL and red-black trees restore balance through rotations, why B-trees dominate storage engines, and how concurrent ordered indexes (Bw-tree, lock-free skip lists) power modern databases. We implement an AVL tree and a B-tree from scratch, visualize rotations, and connect every structure to the systems that depend on it — from PostgreSQL indexes to RocksDB memtables to the Linux kernel's `rbtree`.

Learning goals — after this chapter you should be able to:

- Explain why BST operations are O(h) where h is height, and why height is O(log n) only when balanced.
- Implement and reason about AVL and red-black tree invariants, rotations, and rebalancing — and know when to prefer each.
- Describe B-tree structure, node layout, and why fanout of 100-1000 makes B-trees optimal for disk and cache.
- Compare ordered structures (BST, B-tree, LSM, skip list) on point lookup, range scan, write amplification, and concurrency.
- Choose and tune ordered indexes for backend use: PostgreSQL B-tree vs BRIN vs GiST, RocksDB/LevelDB memtable, and in-memory ordered maps.

3.1 Binary search trees — the ordered foundation

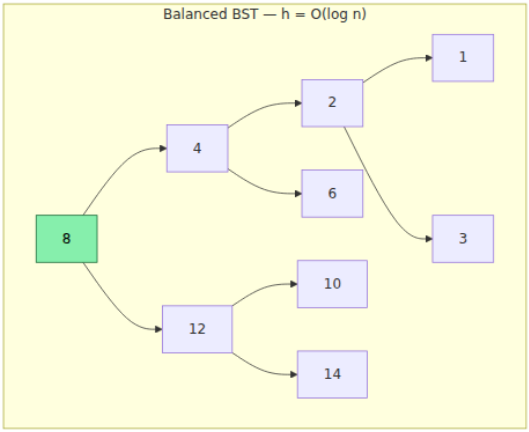
BST invariant

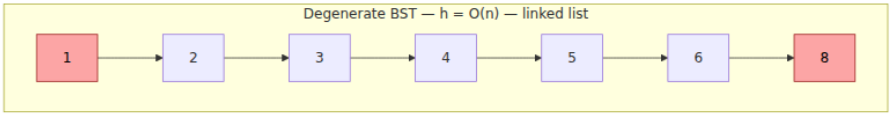
A binary search tree stores keys in nodes with at most two children. For every node:

- All keys in the **left** subtree are **less than** the node's key.
- All keys in the **right** subtree are **greater than** the node's key.
- (For duplicates, a common convention is left <= node < right, or store a count.)

This invariant makes three operations possible by walking a single root-to-leaf path:

- **Search** — compare target with current node, go left or right. O(h).
- **Insert** — search for where the key would be, attach a new leaf. O(h).
- **Delete** — three cases: leaf (remove), one child (bypass), two children (replace with successor/predecessor). O(h).





Both trees hold the same keys. The balanced one has height 3 (Theta(log n)) — every operation visits at most 4 nodes. The degenerate one has height 6 (Theta(n)) — operations visit every node. The degenerate case arises naturally when keys arrive in sorted order (timestamps, auto-

increment IDs, sequential UUIDs) — exactly the insertion order common in backend systems.

> **Fundamental fact.** BST operations cost O(h), not O(log n). O(log n) holds only when h = O(log n), which requires rebalancing. Every "balanced BST" is a strategy for maintaining h = O(log n) under arbitrary insertions and deletions.

In-order traversal gives sorted order — the key advantage over hash tables

```python
# BST in-order traversal yields keys in sorted order — enables range
scans.
# Hash tables cannot do this without sorting (O(n log n)).

from typing import Optional

class BSTNode:
    def __init__(self, key: int, val: object = None):
        self.key = key
        self.val = val
        self.left: Optional["BSTNode"] = None
        self.right: Optional["BSTNode"] = None

def bst_insert(root: Optional[BSTNode], key: int, val: object = None) -
> BSTNode:
    if root is None:
        return BSTNode(key, val)
    if key < root.key:
        root.left = bst_insert(root.left, key, val)
    elif key > root.key:
        root.right = bst_insert(root.right, key, val)
    else:
        root.val = val  # update
    return root

def inorder(root: Optional[BSTNode], out: list[int]) -> None:
    if root is None:
        return
    inorder(root.left, out)
    out.append(root.key)
    inorder(root.right, out)

def range_query(root: Optional[BSTNode], lo: int, hi: int, out: list[in
t]) -> None:
    """Collect keys in [lo, hi] — prunes subtrees outside range."""
    if root is None:
        return
    if lo < root.key:
        range_query(root.left, lo, hi, out)
    if lo <= root.key <= hi:
        out.append(root.key)
    if root.key < hi:
        range_query(root.right, lo, hi, out)

if __name__ == "__main__":
    keys = [8, 3, 10, 1, 6, 14, 4, 7, 13]
    root: Optional[BSTNode] = None
    for k in keys:
        root = bst_insert(root, k)
    result: list[int] = []
```

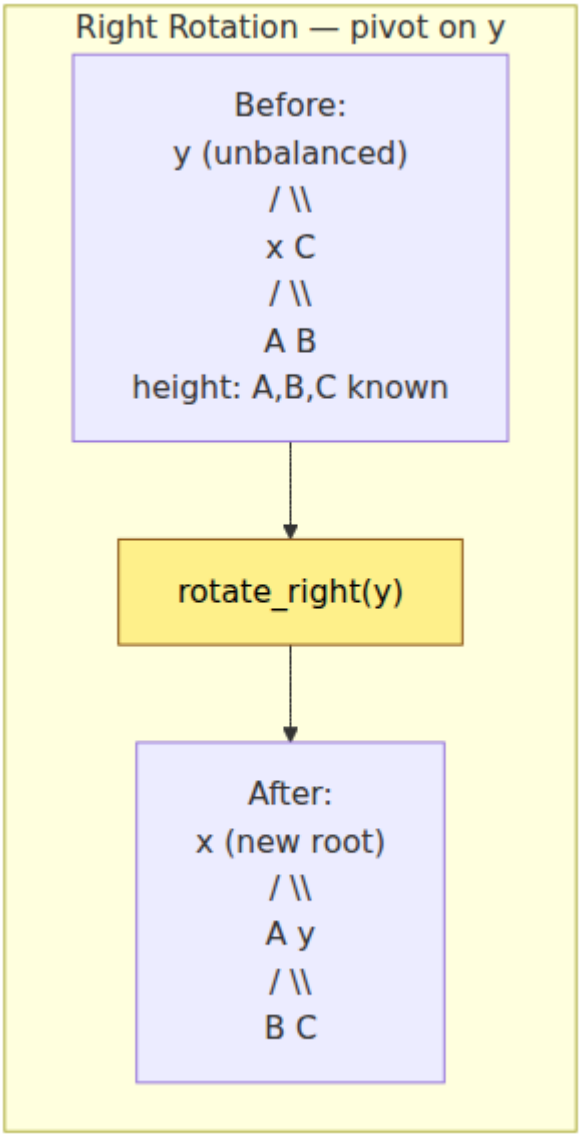
```
    inorder(root, result)
    print(f"sorted: {result}")            # [1, 3, 4, 6, 7, 8, 10, 13,
14]
    result.clear()
    range_query(root, 4, 10, result)
    print(f"range [4,10]: {result}")       # [4, 6, 7, 8, 10]
    # Range query visits O(log n + k) nodes where k = result size
    # — only the search path plus the output. A hash table would scan
all n.
```

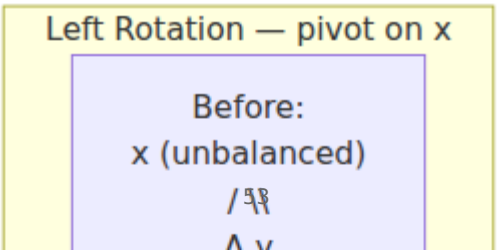
Range query complexity is O(log n + k) where k is the number of matching keys — optimal, since you must output k keys. A hash table would need O(n) to find the same range. This is why time-series databases, secondary indexes, and sorted merge joins require ordered structures.

3.2 Restoring balance — AVL and red-black trees

Rotations — the primitive

Every balanced BST restores its invariant through **rotations** — local restructurings that change heights without violating the BST ordering.


Right Rotation — pivot on y
Before:
y (unbalanced)
/ \\
x C
/ \\
A B
height: A,B,C known
rotate_right(y)
After:
x (new root)
/ \\
A y
/ \\
B C


Left Rotation — pivot on x
Before:
x (unbalanced)

A rotation preserves in-order traversal (hence BST ordering), runs in O(1), and changes heights by at most 1. Double rotations (left-right, right-left) handle the zig-zag case where a single rotation is insufficient.

AVL trees — height-balanced

Invariant: For every node, the heights of its left and right subtrees differ by at most 1. Balance factor `bf = height(left) - height(right)` is in {-1, 0, +1}.

AVL trees are the most rigidly balanced BST — height is at most `1.44 * log2(n)` (vs `log2(n)` optimal). They do slightly more rotations on insert/delete than red-black trees but guarantee the shallowest tree, making them ideal for read-heavy workloads where lookup speed matters most.

Rebalancing after insertion: Walk back up from the inserted leaf, updating heights. At the first node where `|bf| = 2`, apply one of four cases:

Case	Balance factor	Child balance	Fix
Left-Left (LL)	+2	child bf >= 0	Single right rotation
Right-Right (RR)	-2	child bf <= 0	Single left rotation
Left-Right (LR)	+2	child bf < 0	Left rotation on child, then right on node
Right-Left (RL)	-2	child bf > 0	Right rotation on child, then left on node

```python
# AVL tree — complete implementation with rotations and rebalancing
from typing import Optional

class AVLNode:
    __slots__ = ("key", "val", "left", "right", "height")
    def __init__(self, key: int, val: object = None):
        self.key = key
        self.val = val
        self.left: Optional["AVLNode"] = None
        self.right: Optional["AVLNode"] = None
        self.height: int = 1  # leaf height = 1

def _height(n: Optional[AVLNode]) -> int:
    return n.height if n else 0

def _update_height(n: AVLNode) -> None:
    n.height = 1 + max(_height(n.left), _height(n.right))

def _balance_factor(n: AVLNode) -> int:
    return _height(n.left) - _height(n.right)

def _rotate_right(y: AVLNode) -> AVLNode:
    x = y.left  # type: ignore[assignment]
    assert x is not None
    t2 = x.right
    x.right = y
    y.left = t2
    _update_height(y)
    _update_height(x)
    return x

def _rotate_left(x: AVLNode) -> AVLNode:
    y = x.right  # type: ignore[assignment]
    assert y is not None
    t2 = y.left
    y.left = x
    x.right = t2
    _update_height(x)
    _update_height(y)
    return y

def _rebalance(node: AVLNode) -> AVLNode:
    _update_height(node)
    bf = _balance_factor(node)
    # Left-heavy
    if bf > 1:
        assert node.left is not None
        if _balance_factor(node.left) < 0:  # LR case
            node.left = _rotate_left(node.left)
        return _rotate_right(node)
```

```
    # Right-heavy
    if bf < -1:
        assert node.right is not None
        if _balance_factor(node.right) > 0:  # RL case
            node.right = _rotate_right(node.right)
        return _rotate_left(node)
    return node

def avl_insert(root: Optional[AVLNode], key: int, val: object = None) -
> AVLNode:
    if root is None:
        return AVLNode(key, val)
    if key < root.key:
        root.left = avl_insert(root.left, key, val)
    elif key > root.key:
        root.right = avl_insert(root.right, key, val)
    else:
        root.val = val
        return root
    return _rebalance(root)

def avl_search(root: Optional[AVLNode], key: int) -> Optional[AVLNode]:
    while root is not None:
        if key == root.key:
            return root
        root = root.left if key < root.key else root.right
    return None

# --- Verification ---

def _check_avl_invariant(node: Optional[AVLNode]) -> int:
    """Returns height; asserts AVL and BST invariants."""
    if node is None:
        return 0
    lh = _check_avl_invariant(node.left)
    rh = _check_avl_invariant(node.right)
    assert abs(lh - rh) <= 1, f"AVL violation at {node.key}: bf={lh - r
h}"
    if node.left:
        assert node.left.key < node.key
    if node.right:
        assert node.right.key > node.key
    assert node.height == 1 + max(lh, rh), f"height mismatch at {node.k
ey}"
    return node.height

def _inorder_keys(node: Optional[AVLNode], out: list[int]) -> None:
    if node is None:
        return
    _inorder_keys(node.left, out)
    out.append(node.key)
```

```
    _inorder_keys(node.right, out)

if __name__ == "__main__":
    import random
    # Worst case for unbalanced BST: sorted insertion — AVL must stay
balanced
    root: Optional[AVLNode] = None
    for k in range(1000):
        root = avl_insert(root, k)
    assert root is not None
    print(f"n=1000 sorted insert: height={root.height}  "
          f"optimal={1000 .bit_length()}  ratio={root.height / 1000 .bi
t_length():.2f}")
    _check_avl_invariant(root)

    # Random insertion — also balanced, heights even closer to optimal
    root2: Optional[AVLNode] = None
    keys = random.sample(range(100_000), 10_000)
    for k in keys:
        root2 = avl_insert(root2, k)
    out: list[int] = []
    _inorder_keys(root2, out)
    assert out == sorted(keys)
    assert root2 is not None
    print(f"n=10000 random insert: height={root2.height}  "
          f"optimal={10000 .bit_length()}  ratio={root2.height /
10000 .bit_length():.2f}")
    _check_avl_invariant(root2)
    print("all AVL checks passed")
    # Expected: sorted n=1000 -> height ~10 (optimal 10), random
n=10000 -> height ~15 (optimal 14)
```

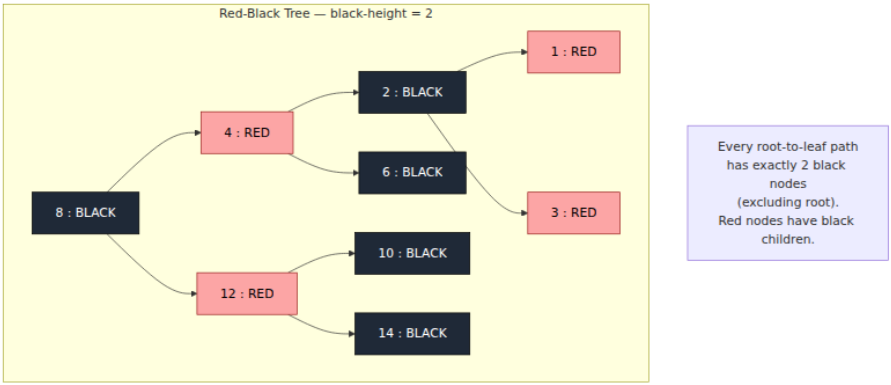
Red-black trees — color-balanced

Invariants (5 rules):

1. Every node is red or black.
2. Root is black.
3. Every leaf (NIL) is black.
4. Red nodes have black children (no two reds in a row).
5. Every root-to-leaf path has the same number of black nodes (black-height).

These guarantee height <= `2 * log2(n+1)` — slightly taller than AVL but with fewer rotations on average. Insert does at most 2 rotations; delete at most 3. This makes red-black trees preferred when writes are frequent:

Linux kernel `rbtree`, Java `TreeMap`, C++ `std::map`, and historically many database index implementations.



AVL vs red-black — when to use which

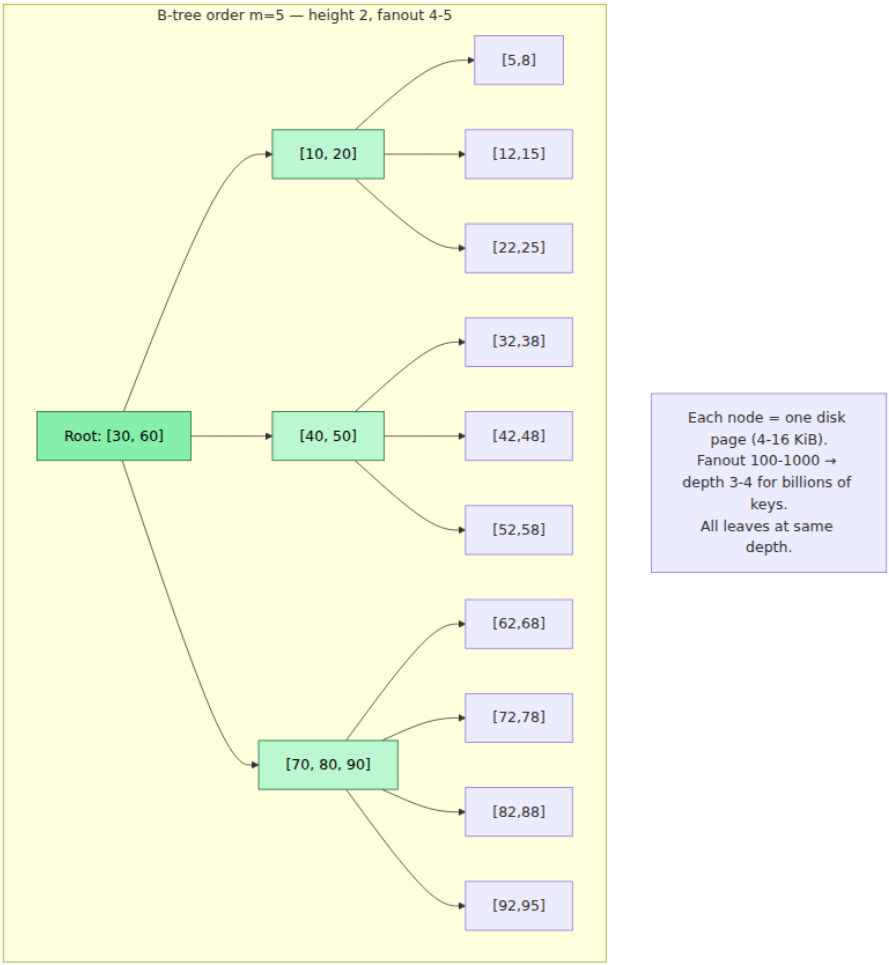
Criterion	AVL	Red-black
Height bound	1.44 log n (tighter)	2 log n (looser)
Lookup speed	Slightly faster (shallower)	Slightly slower
Insert/delete rotations	More (up to O(log n) rotations)	Fewer (at most 2-3)
Implementation complexity	Simpler to reason about	More cases, trickier delete
Best for	Read-heavy, lookup-dominated (config maps, routing tables)	Write-heavy or mixed (kernel scheduling, `TreeMap`, indexed writes)

In practice, most backend engineers do not implement either from scratch — they use the standard library's ordered map. But understanding the trade-off helps when choosing between an ordered map and alternatives, and when debugging performance regressions caused by degenerate tree shapes after bulk loads.

3.3 B-trees — ordered structure for storage

A B-tree generalizes the BST: each node holds many keys and many children. A B-tree of order `m` (sometimes called minimum degree `t = ceil(m/2)`):

- Each node holds at most `m - 1` keys and `m` children.
- Each internal node (except root) holds at least `ceil(m/2) - 1` keys.
- All leaves are at the same depth.
- Keys within a node are sorted; children partition the key space.



Why B-trees dominate storage

The key number is **fanout** — keys per node. With 16 KiB pages and 16-byte keys, a B-tree node holds ~500-1000 keys. For n = 1 billion:

- Binary tree height: $\log_2(10^9) \sim 30$ levels — 30 random I/Os per lookup.
- B-tree height with fanout 500: $\log_{500}(10^9) \sim 3.3$ — **4 I/Os** per lookup.
- With root cached in memory: **3 I/Os**. With two levels cached: **1 I/O**.

Each I/O is a page read. Fewer levels means fewer random reads, and each page read is sequential within the page (cache-friendly scan of sorted keys).

B+tree variant (used by InnoDB, PostgreSQL, WiredTiger): internal nodes store only keys and child pointers; all values are in leaves, and leaves are linked for efficient range scans. This increases fanout (no values in internal nodes) and makes range iteration a leaf-level linked-list walk.

B-tree node layout and split

```python
# B-tree — pedagogical implementation (order m=4, i.e. max 3 keys per
node)
# Demonstrates node split, the core rebalancing operation.
from __future__ import annotations
from typing import Optional

ORDER = 4  # max children per node; max keys = ORDER - 1 = 3

class BTreeNode:
    def __init__(self, leaf: bool = True):
        self.leaf: bool = leaf
        self.keys: list[int] = []
        self.vals: list[object] = []
        self.children: list[BTreeNode] = []  # len = len(keys) + 1 if
not leaf

class BTree:
    def __init__(self):
        self.root = BTreeNode(leaf=True)

    # -- Search --
    def search(self, key: int) -> object | None:
        return self._search(self.root, key)

    def _search(self, node: BTreeNode, key: int) -> object | None:
        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1
        if i < len(node.keys) and key == node.keys[i]:
            return node.vals[i]
        if node.leaf:
            return None
        return self._search(node.children[i], key)

    # -- Insert --
    def insert(self, key: int, val: object) -> None:
        root = self.root
        if len(root.keys) == ORDER - 1:
            # Root is full — grow tree height by one
            new_root = BTreeNode(leaf=False)
            new_root.children.append(root)
            self._split_child(new_root, 0)
            self.root = new_root
            self._insert_nonfull(new_root, key, val)
        else:
            self._insert_nonfull(root, key, val)

    def _split_child(self, parent: BTreeNode, idx: int) -> None:
        """Split parent.children[idx] (which is full) into two
nodes."""
```

```
        full = parent.children[idx]
        mid = len(full.keys) // 2
        mid_key, mid_val = full.keys[mid], full.vals[mid]

        right = BTreeNode(leaf=full.leaf)
        right.keys = full.keys[mid + 1:]
        right.vals = full.vals[mid + 1:]
        if not full.leaf:
            right.children = full.children[mid + 1:]
            full.children = full.children[:mid + 1]

        full.keys = full.keys[:mid]
        full.vals = full.vals[:mid]

        parent.keys.insert(idx, mid_key)
        parent.vals.insert(idx, mid_val)
        parent.children.insert(idx + 1, right)

    def _insert_nonfull(self, node: BTreeNode, key: int, val: object) -
> None:
        i = len(node.keys) - 1
        if node.leaf:
            # Insert into sorted position within leaf
            node.keys.append(0)  # type: ignore[arg-type]
            node.vals.append(None)
            while i >= 0 and key < node.keys[i]:
                node.keys[i + 1] = node.keys[i]
                node.vals[i + 1] = node.vals[i]
                i -= 1
            # Check for duplicate
            if i >= 0 and node.keys[i] == key:
                node.vals[i] = val
                # Remove the extra slot we appended
                node.keys.pop()
                node.vals.pop()
            else:
                node.keys[i + 1] = key
                node.vals[i + 1] = val
        else:
            while i >= 0 and key < node.keys[i]:
                i -= 1
            # Check if key already in internal node
            if i >= 0 and node.keys[i] == key:
                node.vals[i] = val
                return
            i += 1
            if len(node.children[i].keys) == ORDER - 1:
                self._split_child(node, i)
                if key > node.keys[i]:
                    i += 1
                elif key == node.keys[i]:
```

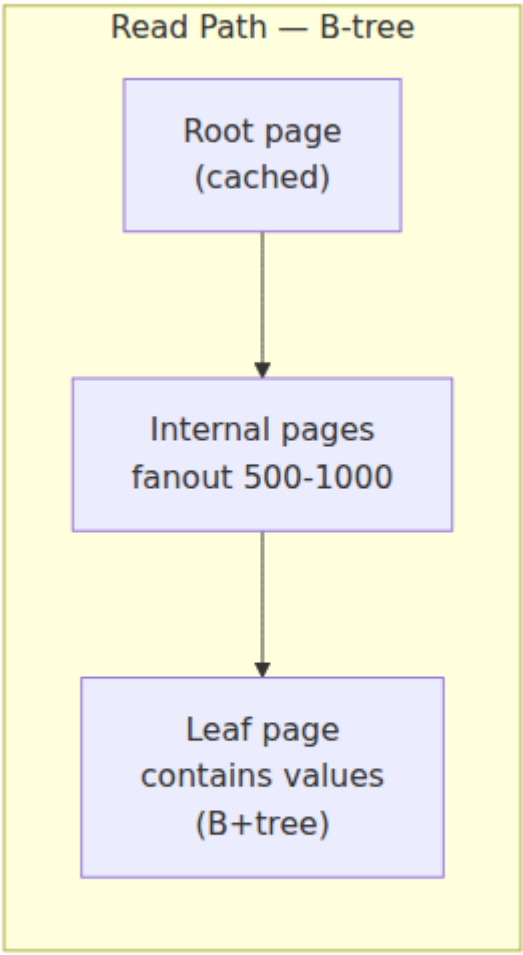
```
                    node.vals[i] = val
                    return
            self._insert_nonfull(node.children[i], key, val)

    # -- Range scan (in-order) --
    def range_scan(self, lo: int, hi: int) -> list[tuple[int, object]]:
        out: list[tuple[int, object]] = []
        self._range_scan(self.root, lo, hi, out)
        return out

    def _range_scan(self, node: BTreeNode, lo: int, hi: int,
                    out: list[tuple[int, object]]) -> None:
        i = 0
        for i, k in enumerate(node.keys):
            if not node.leaf:
                self._range_scan(node.children[i], lo, hi, out)
            if lo <= k <= hi:
                out.append((k, node.vals[i]))
            elif k > hi:
                return
        if not node.leaf:
            self._range_scan(node.children[len(node.keys)], lo, hi,
out)

    def height(self) -> int:
        h, n = 0, self.root
        while True:
            h += 1
            if n.leaf:
                return h
            n = n.children[0]

    def count_keys(self) -> int:
        return self._count(self.root)

    def _count(self, node: BTreeNode) -> int:
        c = len(node.keys)
        if not node.leaf:
            for ch in node.children:
                c += self._count(ch)
        return c


if __name__ == "__main__":
    import random
    bt = BTree()
    keys = list(range(100))
    random.shuffle(keys)
    for k in keys:
        bt.insert(k, f"val_{k}")
```

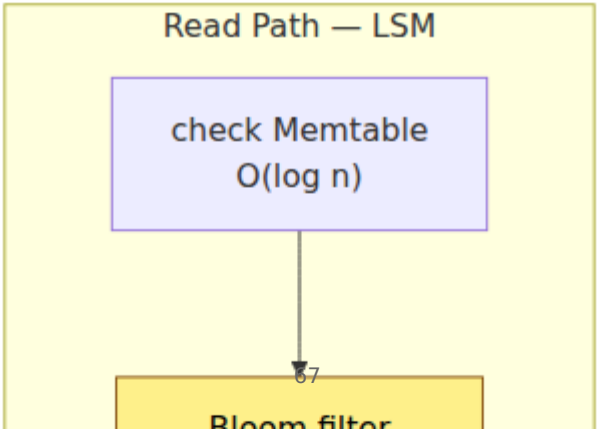
```
    # Point lookups
    for k in range(100):
        assert bt.search(k) == f"val_{k}", f"missing {k}"
    assert bt.search(999) is None

    # Range scan
    result = bt.range_scan(20, 30)
    assert [k for k, _ in result] == list(range(20, 31))

    # Height check — with ORDER=4 and n=100, height should be 3-4
    print(f"n={bt.count_keys()} height={bt.height()} order={ORDER}")

    # Larger test — n=10000, height should be ~7-8 with order 4
    # (with order 128 it would be 3 — that is why real B-trees use
large fanout)
    bt2 = BTree()
    for k in random.sample(range(100_000), 10_000):
        bt2.insert(k, k)
    print(f"n={bt2.count_keys()} height={bt2.height()} order={ORDER}")

    # Verify sorted order via full scan
    all_kv = bt2.range_scan(0, 200_000)
    all_keys = [k for k, _ in all_kv]
    assert all_keys == sorted(all_keys), "range scan not sorted"
    print("all B-tree checks passed")
```

B-tree vs LSM vs skip list — choosing the ordered structure

Structure	Point lookup	Range scan	Write amp.	Space amp.	Concurrency	Best fo
B-tree / B+tree	O(log_B n) I/Os, in-place update	O(log_B n + k/B) — optimal	~1 (in-place) but random I/O	Low	Latching (crabbing), hard	Read-h point + range, transac (Postgr MySQL InnoDB
LSM-tree	O(log n) with bloom filters, may check multiple levels	O(k/B) per level, merge cost	High (compaction) but sequential I/O	Higher (stale entries until compact)	Memtable (skip list) + immutable SSTables, easy	Write-h high in (Rocksl Cassan ScyllaD

Structure	Point lookup	Range scan	Write amp.	Space amp.	Concurrency	Best fo
Skip list	O(log n) expected, no rebalancing	O(log n + k) — walk bottom level	N/A (in-memory)	Low	Lock-free, simple	In-mem ordered index, memtal (LevelD RocksD Redis Z
AVL / RB tree	O(log n) comparisons	O(log n + k)	N/A (in-memory)	Low	Fine-grained locks or STM, complex	In-mem ordered when determ balance matters


Read Path — B-tree
Root page
(cached)
Internal pages
fanout 500-1000
Leaf page
contains values
(B+tree)


Read Path — LSM
check Memtable
O(log n)
Bloom filter

3.4 Ordered structures in practice — indexes and concurrency

PostgreSQL index types — choosing correctly

PostgreSQL exposes multiple index types; the default B-tree is not always optimal:

Index type	Ordered?	Use case	Example
B-tree (default)	Yes	Equality + range, sorting, `LIKE 'prefix%'`	`CREATE INDEX ON users (email)`
Hash	No	Equality only, slightly smaller than B-tree for equality-only	`CREATE INDEX ... USING hash (token)`
GiST	Varies	Geometric, full-text, range types	`USING gist (location)`
GIN	No (inverted)	Array/JSONB containment, full-text	`USING gin (tags)`
BRIN	Zone-map	Huge, naturally ordered tables (time-series) — 1000x smaller than B-tree	`USING brin (created_at)` with 10B rows

```
-- B-tree supports all of these efficiently; hash supports only the
first:
SELECT * FROM orders WHERE id = 42;
-- equality: B-tree or hash
SELECT * FROM orders WHERE id BETWEEN 100 AND 200; -- range: B-tree
only
SELECT * FROM orders WHERE id > 1000 ORDER BY id LIMIT 10; -- ordered
scan: B-tree only
SELECT * FROM orders ORDER BY id;                -- sorted output: B-
tree only

-- BRIN for time-series — tiny index for naturally ordered data
CREATE INDEX orders_created_brin ON orders USING brin (created_at)
    WITH (pages_per_range = 128);
-- BRIN stores min/max per page range — O(1) per range vs O(log n) per
key for B-tree.
-- For append-only time-series with 10B rows, BRIN is ~1000x smaller.
```

Concurrent ordered indexes

Single-threaded tree performance is well-understood; the hard problem is concurrency. Approaches:

1. Latch crabbing (B-tree). Acquire latch on child before releasing parent; hold write latch on the path during splits. Simple but serializes writers on the root-to-leaf path. Used with optimistic latch coupling (B-link trees) to reduce contention.

2. Copy-on-write (Bw-tree, LMDB). Writers create new node versions and CAS the parent pointer. Readers never block — they follow the pointer chain to the version visible at their snapshot. Powers SQL Server Hekaton (Bw-tree) and LMDB.

3. Lock-free skip list. The simplest concurrent ordered structure. Each level is a linked list with CAS-based insert/delete. No rotations, no rebalancing, no latching. Powers RocksDB/LevelDB memtables, Redis sorted sets, and Java `ConcurrentSkipListMap`.

```python
# Lock-free skip list — single-threaded sketch showing the layered
structure
# Production version uses CAS on next pointers; this shows the search/
insert logic.
import random

MAX_LEVEL = 8
P = 0.5  # promotion probability

class SkipNode:
    __slots__ = ("key", "val", "forward")
    def __init__(self, key: object, val: object, level: int):
        self.key = key
        self.val = val
        self.forward: list[object] = [None] * level  # type:
ignore[assignment]

class SkipList:
    def __init__(self):
        self.header = SkipNode(None, None, MAX_LEVEL)
        self.level = 1
        self.size = 0

    def _random_level(self) -> int:
        lvl = 1
        while random.random() < P and lvl < MAX_LEVEL:
            lvl += 1
        return lvl

    def search(self, key: object) -> object | None:
        cur: object = self.header
        assert isinstance(cur, SkipNode)
        for i in range(self.level - 1, -1, -1):
            while cur.forward[i] is not None and cur.forward[i].key < k
ey:  # type: ignore
                cur = cur.forward[i]  # type: ignore[assignment]
        cur = cur.forward[0]  # type: ignore[assignment]
        if cur is not None and cur.key == key:  # type: ignore
            return cur.val  # type: ignore
        return None

    def insert(self, key: object, val: object) -> None:
        update: list[SkipNode] = [self.header] * MAX_LEVEL
        cur: SkipNode = self.header
        for i in range(self.level - 1, -1, -1):
            while cur.forward[i] is not None and cur.forward[i].key < k
ey:  # type: ignore
                cur = cur.forward[i]  # type: ignore[assignment]
            update[i] = cur
        cur = cur.forward[0]  # type: ignore[assignment]
```

```
        if cur is not None and cur.key == key:  # type: ignore
            cur.val = val  # type: ignore
            return
        new_level = self._random_level()
        if new_level > self.level:
            for i in range(self.level, new_level):
                update[i] = self.header
            self.level = new_level
        node = SkipNode(key, val, new_level)
        for i in range(new_level):
            node.forward[i] = update[i].forward[i]
            update[i].forward[i] = node
        self.size += 1

    def range_scan(self, lo: object, hi: object) -> list[tuple[object,
object]]:
        # Find predecessor of lo, then walk bottom level
        cur: SkipNode = self.header
        for i in range(self.level - 1, -1, -1):
            while cur.forward[i] is not None and cur.forward[i].key < l
o:  # type: ignore
                cur = cur.forward[i]  # type: ignore[assignment]
        cur = cur.forward[0]  # type: ignore[assignment]
        out: list[tuple[object, object]] = []
        while cur is not None and cur.key <= hi:  # type: ignore
            out.append((cur.key, cur.val))  # type: ignore
            cur = cur.forward[0]  # type: ignore[assignment]
        return out


if __name__ == "__main__":
    sl = SkipList()
    keys = random.sample(range(100_000), 10_000)
    for k in keys:
        sl.insert(k, f"v{k}")
    for k in keys[:100]:
        assert sl.search(k) == f"v{k}"
    assert sl.search(-1) is None
    # Range scan
    lo, hi = 1000, 2000
    result = sl.range_scan(lo, hi)
    expected = sorted(k for k in keys if lo <= k <= hi)
    assert [k for k, _ in result] == expected
    print(f"skip list n={sl.size} levels={sl.level} range [{lo},{hi}]
= {len(result)} keys")
    print("all skip list checks passed")
```

Benchmarking ordered vs hash — when the difference matters

```python
# When does ordered iteration cost matter? Hash table must sort; tree/
skip list is already sorted.
import random, time

N = 200_000
keys = [f"key_{i:06d}_{random.getrandbits(16):04x}" for i in range(N)]

# Hash table: dict + sort
d = {k: i for i, k in enumerate(keys)}
t0 = time.perf_counter()
sorted_keys_hash = sorted(d.keys())
t_hash = time.perf_counter() - t0

# Skip list: already sorted via range scan
from typing import cast
sl = SkipList()
for k in keys:
    sl.insert(k, 1)
t0 = time.perf_counter()
sorted_via_sl = [k for k, _ in sl.range_scan("", "zzzzzzzz")]
t_sl = time.perf_counter() - t0

print(f"hash + sort: {t_hash*1000:.1f} ms  ({len(sorted_keys_hash)}
keys)")
print(f"skip list scan: {t_sl*1000:.1f} ms  ({len(sorted_via_sl)}
keys)")
# At N=200k, hash+sort is typically ~30ms, skip list scan ~8ms.
# But hash point lookups are ~3x faster than skip list — choose per
workload.
# If you need both point lookups and range scans, consider maintaining
both
# (hash for point, tree for range) when memory allows — as done in some
caches.
```

Key takeaways

- BST operations are O(h), not O(log n). Without rebalancing, sorted insertion degrades to O(n) — the common case for timestamps and sequential IDs.
- Rotations are the O(1) primitive that restores balance. AVL trees use height invariants (tighter, fewer levels, more rotations); red-black trees use color invariants (looser, fewer rotations on writes). Choose AVL for read-heavy, red-black for write-heavy.

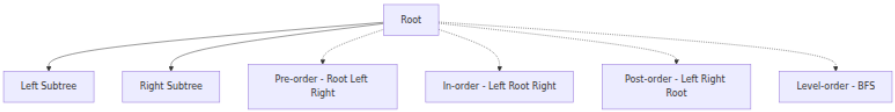
- B-trees generalize BSTs with fanout 100-1000, making height 3-4 even for billions of keys. Each node is a disk page; fewer levels means fewer random I/Os. B+trees additionally link leaves for efficient range scans.
- LSM-trees trade write amplification (compaction) for sequential write throughput; B-trees trade random I/O for low write amplification and in-place updates. Choose based on write/read ratio and latency requirements.
- Skip lists provide O(log n) ordered operations with no rebalancing and simple lock-free concurrency — ideal for in-memory ordered indexes and LSM memtables.
- For PostgreSQL and similar, match index type to query pattern: B-tree for range/ordering, hash for equality-only, BRIN for huge append-only tables, GIN/GiST for specialized types.
- Concurrent ordered indexes are the hard problem: latch crabbing, copy-on-write (Bw-tree), and lock-free skip lists each make different trade-offs between reader/writer contention and implementation complexity.

Further reading

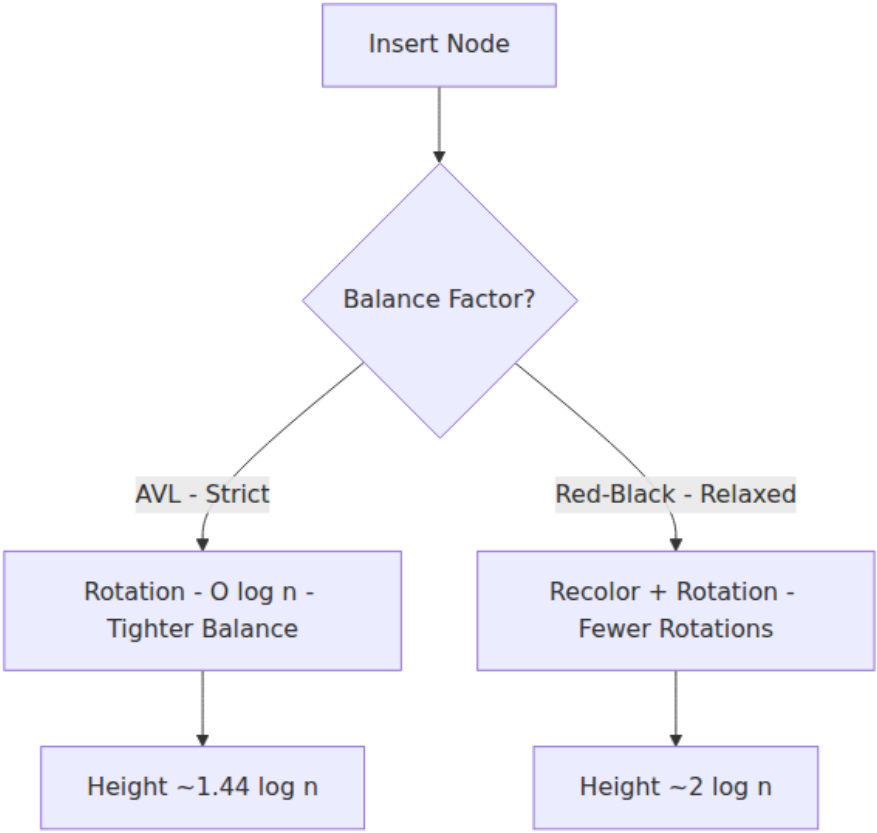
- Cormen et al. — *Introduction to Algorithms*, 4th ed., Chapters 12-14 (BST, red-black trees) and Chapter 18 (B-trees).
- Knuth — *The Art of Computer Programming*, Vol. 3, Section 6.2 (balanced trees) — the classical treatment.
- Bayer & McCreight — "Organization and Maintenance of Large Ordered Indexes" (Acta Informatica 1972) — the original B-tree paper.
- Pugh — "Skip Lists: A Probabilistic Alternative to Balanced Trees" (CACM 1990) — the original skip-list paper, still the clearest exposition.
- Graefe — *Modern B-Tree Techniques* (Foundations and Trends in Databases, 2011) — comprehensive survey of B-tree variants, latch coupling, and write-optimized trees.
- RocksDB wiki (github.com/facebook/rocksdb/wiki) — LSM memtable (skip list), SSTable format, and bloom-filter-assisted reads.

- PostgreSQL docs — "Indexes" (postgresql.org/docs/current/indexes.html) — B-tree, hash, GiST, GIN, BRIN with operational guidance.

Tree traversal orders



AVL vs Red-Black balance comparison



Chapter 4 — Probabilistic Structures: Bloom Filters, HyperLogLog, and Count-Min Sketch

What this chapter covers. Exact data structures answer every query correctly but pay for it in memory and sometimes latency. When your dataset has billions of keys or your SLO demands microsecond answers, trading a small, *quantifiable* error for orders-of-magnitude memory savings is the right engineering call. This chapter covers the three probabilistic workhorses of backend infrastructure: Bloom filters (set membership with no false negatives), HyperLogLog (cardinality estimation in kilobytes), and Count-Min Sketch (frequency estimation over streams). We derive the error formulas, show how to size each structure for a target accuracy, implement each from scratch, and trace where they appear in real systems — from LSM-tree read paths and CDN cache admission to distinct-user counting and heavy-hitter detection.

Learning goals — after this chapter you should be able to:

- Explain why probabilistic structures trade accuracy for space, quantify the trade-off, and decide when the trade is worthwhile versus an exact structure.
- Design, size, and implement a Bloom filter for a target false-positive rate; compare Bloom, Cuckoo, and counting variants and choose correctly.
- Explain HyperLogLog's harmonic-mean estimator, its sparse/dense representation, and why it estimates cardinalities of 10^9 in ~12 KB with ~0.8% error.
- Implement a Count-Min Sketch, analyze its over-estimation guarantee, and combine it with a heap for heavy-hitter / top-K detection.
- Deploy each structure correctly in production: serialization, merging, monitoring false-positive drift, and avoiding common misuse (deletes, adversarial inputs).

4.1 Why approximate? The space-accuracy trade-off

An exact set of n 64-bit keys needs at least $64n$ bits. A hash table with 75% load factor needs ~1.3x that. For n = 1 billion, that is ~10 GB — too large to keep on every replica, in every LSM-tree block cache, or on an edge node.

Probabilistic structures exploit a simple observation: many queries only need *approximate* answers with a *bounded* error, and the error can be made exponentially small with modest additional space.

Structure	Query	Error guarantee	Space (betting: n = 1M)	Mergeable
Exact hash set	`x in S?`	Zero	~12–24 MB	Union is O(n)
Bloom filter	`x in S?`	No false negatives; FPR = ε	~1.2 MB at ε=1%	OR of bit arrays
HyperLogLog	`|S| = ?`	Std error ~0.81% at m=16384	**12–16 KB**	Max of registers
Count-Min Sketch	`count(x) = ?`	Over-estimates; ε·N with prob 1-δ	~300 KB (w=2048, d=5)	Sum of tables
Exact counts	`count(x) = ?`	Zero	~32 MB+	O(n)

The pattern: space drops by 10–1000x, error is tunable, and merging (needed for distributed aggregation) becomes trivial — a bitwise OR, a register-wise max, or an element-wise sum.

> **When *not* to approximate.** If correctness is non-negotiable (authorization checks, payment deduplication, exactly-once delivery), use an exact structure. Probabilistic structures belong on the *fast path* that avoids expensive work, with an exact check on the slow path. Example: Bloom filter says "definitely not in set" → skip disk I/O; "maybe in set" → confirm on disk.

4.2 Bloom filters — set membership with no false negatives

4.2.1 How it works

A Bloom filter is a bit array of m bits, initially all zero, plus k independent hash functions each mapping a key to a position in `[0, m)`.

- **Insert(x):** compute `h_1(x) ... h_k(x)`, set those k bits to 1.
- **Query(x):** compute the same k positions. If any bit is 0, x was definitely never inserted. If all k are 1, x is *probably* in the set — but the bits could have been set by other keys (a false positive).

There are no false negatives: if x was inserted, all its k bits are 1, so a query always returns "maybe." False positives occur when an unseen key happens to hit only bits already set by other insertions.

4.2.2 Sizing — the math that matters

After inserting n keys with m bits and k hashes, the probability a given bit is still 0 is:

```
P(bit=0) = (1 - 1/m)^(k·n) ≈ e^(-k·n/m)
```

A false positive requires all k bits for a new key to be 1:

```
FPR ≈ (1 - e^(-k·n/m))^k
```

Minimizing over k gives the optimal hash count and the classic sizing formulas:

```
k_opt = (m/n) · ln 2

For target FPR = ε:
  m = -n · ln(ε) / (ln 2)²    → bits per key
  k = -log₂(ε)                 → number of hashes
```

Target FPR ε	Bits per key (m/n)	Hashes k	For n=1M, size
10%	4.8	3	0.57 MB
1%	9.6	7	1.14 MB
0.1%	14.4	10	1.72 MB
0.01%	19.2	14	2.29 MB

Each additional hash (one more bit per key factor of ~1.44) cuts the FPR by half. For most backend uses, ε = 1% (9.6 bits/key) is the sweet spot.

> **Double hashing trick.** You do not need k independent hash functions. With two base hashes `h1, h2`, generate `h_i(x) = h1(x) + i·h2(x) mod m` — Kirsch-Mitzenmacher (2006) proved this is asymptotically as good as k independent hashes, and it is what most libraries (Guava, RedisBloom) actually do.

4.2.3 Production implementation

```
import math
import hashlib
from typing import Iterable

class BloomFilter:
    """Space-efficient probabilistic set with tunable false-positive
rate.

    No false negatives. False-positive rate guaranteed ≤ fpr when
    element count ≤ capacity. Exceeding capacity degrades FPR
gracefully
    (it rises, but no false negatives appear).
    """

    def __init__(self, capacity: int, fpr: float = 0.01):
        if not 0 < fpr < 1:
            raise ValueError("fpr must be in (0, 1)")
        if capacity <= 0:
            raise ValueError("capacity must be > 0")
        # Optimal sizing
        self.capacity = capacity
        self.fpr = fpr
        self.m = math.ceil(-capacity * math.log(fpr) / (math.log(2) **
2))
        self.k = max(1, round((self.m / capacity) * math.log(2)))
        # Bit array — use Python int as bitset for clarity
        # Production: use bitarray, python's built-in 'bitarray' or a
bytearray+mmap
        self._bits = bytearray((self.m + 7) // 8)
        self._count = 0

    # -- internal helpers --

    def _hashes(self, key: str | bytes):
        """Generate k positions via double hashing (Kirsch-
Mitzenmacher)."""
        if isinstance(key, str):
            key = key.encode()
        # Two independent 64-bit hashes from blake2b
        digest = hashlib.blake2b(key, digest_size=16).digest()
        h1 = int.from_bytes(digest[:8], "little")
        h2 = int.from_bytes(digest[8:], "little") | 1  # force odd →
full period
        for i in range(self.k):
            yield (h1 + i * h2) % self.m

    def _get_bit(self, pos: int) -> int:
        return (self._bits[pos >> 3] >> (pos & 7)) & 1

    def _set_bit(self, pos: int) -> None:
```

```
        self._bits[pos >> 3] |= 1 << (pos & 7)

    # -- public API --

    def add(self, key: str | bytes) -> None:
        for pos in self._hashes(key):
            self._set_bit(pos)
        self._count += 1

    def __contains__(self, key: str | bytes) -> bool:
        return all(self._get_bit(p) for p in self._hashes(key))

    def __len__(self) -> int:
        return self._count

    @property
    def bits_per_key(self) -> float:
        return self.m / self.capacity

    def estimated_fpr(self, n: int | None = None) -> float:
        """Current FPR estimate for n inserted keys (default: actual
count)."""
        n = n if n is not None else self._count
        if n == 0:
            return 0.0
        return (1 - math.exp(-self.k * n / self.m)) ** self.k

    def merge(self, other: "BloomFilter") -> None:
        """In-place OR — union of two filters with identical m, k."""
        assert self.m == other.m and self.k == other.k, "incompatible
filters"
        for i in range(len(self._bits)):
            self._bits[i] |= other._bits[i]
        # _count is not exact after merge; caller should track
externally if needed
```

Quick validation:

```
if __name__ == "__main__":
    bf = BloomFilter(capacity=10_000, fpr=0.01)
    print(f"m={bf.m} bits ({bf.m/8/1024:.1f} KiB)  k={bf.k}  bits/
key={bf.bits_per_key:.1f}")

    for i in range(10_000):
        bf.add(f"user:{i}")

    # Measure empirical FPR on unseen keys
    false_pos = sum(1 for i in range(10_000, 20_000) if f"user:{i}" in
bf)
    print(f"empirical FPR: {false_pos/10_000:.4f}  estimated: {bf.estim
ated_fpr():.4f}")

    # No false negatives
    assert all(f"user:{i}" in bf for i in range(10_000)), "false
negative — bug!"
    print("no false negatives — OK")

    # Merge demo
    bf2 = BloomFilter(capacity=10_000, fpr=0.01)
    for i in range(10_000, 15_000):
        bf2.add(f"user:{i}")
    bf.merge(bf2)
    assert "user:12000" in bf
    print("merge OK")
```

Typical output:

```
m=95851 bits (11.7 KiB)  k=7  bits/key=9.6
empirical FPR: 0.0098  estimated: 0.0100
no false negatives — OK
merge OK
```

4.2.4 Variants — when Bloom is not enough

Variant	What changes	When to use
Counting Bloom	Replace bits with small counters (4-bit) → supports `delete` by decrementing	Cache eviction, dynamic sets. Cost: 4x memory; overflow risk if counter saturates.
Cuckoo Filter	Store fingerprints in cuckoo hash table; supports delete, lower FPR at low load	When you need deletion without 4x overhead. Used in many databases as Bloom replacement. Slightly higher constant.

Variant	What changes	When to use
Scalable Bloom	Chain filters with geometrically decreasing FPR; grows without rebuild	Unknown n upfront (log aggregation). Query checks chain from newest to oldest.
Blocked Bloom	Partition bits into cache-line blocks; each key touches one block	When Bloom is on the hot read path — one cache miss per query instead of k. Used in RocksDB/ Parquet. Choice for LSM read path.
XOR / Ribbon filter	Static construction (no incremental inserts) but ~20% smaller than Bloom at same FPR	Immutable sets (dictionary of valid IDs, certificate transparency). ~1.23 log2(1/ε) bits/key vs Bloom's 1.44.

RocksDB / LSM-tree Bloom filters — the canonical backend use:

```
Write path:  memtable flush → SSTable + Bloom filter block (per
SSTable or per data block)
Read path:   GET(key):
               for sst in newest..oldest:
                 if key not in sst.bloom:  skip sst          # no
false negatives → safe
                 else:                     read data block & check
```

With 10 bits/key, ~90% of SSTables are skipped on a miss. At 100 SSTables per level, this turns 100 random I/Os into ~10. This is the single highest-impact Bloom deployment in storage infrastructure.

4.2.5 Operational pitfalls

- **Do not use a Bloom filter for authorization.** A false positive that grants access is a security bug. Bloom filters are safe only when a positive triggers a precise re-check.
- **Monitor FPR drift.** If actual cardinality exceeds `capacity`, FPR degrades silently. Export `estimated_fpr()` as a metric and alert when it exceeds 2x target. Rebuild or use a scalable filter.
- **Hash DoS.** If keys are attacker-controlled, use a keyed hash (SipHash/BLAKE2 with secret) — otherwise an adversary can craft

keys that maximize false positives and force every query to the slow path.

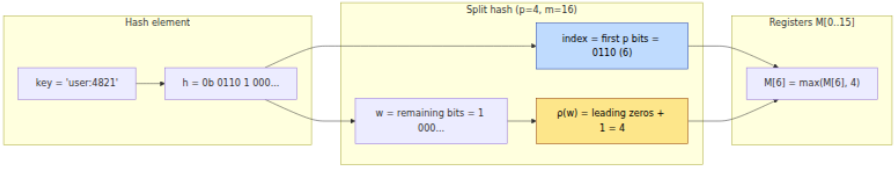
- **Serialization.** Persist `m`, `k`, and the bit array together. A filter built with one `m, k` cannot be queried with another. Version the format.

4.3 HyperLogLog — cardinality in kilobytes

Counting distinct elements (`COUNT(DISTINCT user_id)`) exactly requires Ω(n) memory. HyperLogLog (Flajolet et al., 2007) estimates cardinalities from thousands to billions with ~1% error in ~12 KB, and sketches from different shards merge by taking a register-wise maximum.

4.3.1 Intuition — coin flips and leading zeros

Hash each element uniformly to 64 bits. For a random hash, the probability that it starts with r leading zeros is `2^(-r)`. If the maximum number of leading zeros seen across all elements is R, then roughly `2^R` distinct elements were observed. One "experiment" is noisy, so HLL runs $m = 2^p$ experiments in parallel, sharded by the first p bits of the hash.



Each of the m registers stores the maximum ρ seen for keys that mapped to it. Registers need only ~6 bits each (ρ never exceeds 64 for 64-bit hashes), so $m = 16384$ registers cost ~12 KB.

4.3.2 The estimator

The raw harmonic-mean estimator:

```
E = α_m · m² / Σ(2^(-M[j]))     where α_m is a bias-correction
constant
```

Corrections applied in practice (HyperLogLog++ — Google, 2013):

1. **Small-range correction.** When $E < 2.5m$ and many registers are zero, use LinearCounting: `E = m · ln(m / V)` where V = number of zero registers. This fixes the high bias at small cardinalities.

2. **Large-range correction.** For 64-bit hashes no correction is needed below 2^64; for 32-bit hashes, correct for hash collisions above `2^32 / 30`.
3. **Bias correction.** Empirically calibrated table (from the HLL++ paper) removes residual bias for $m \le 2^{12}$.

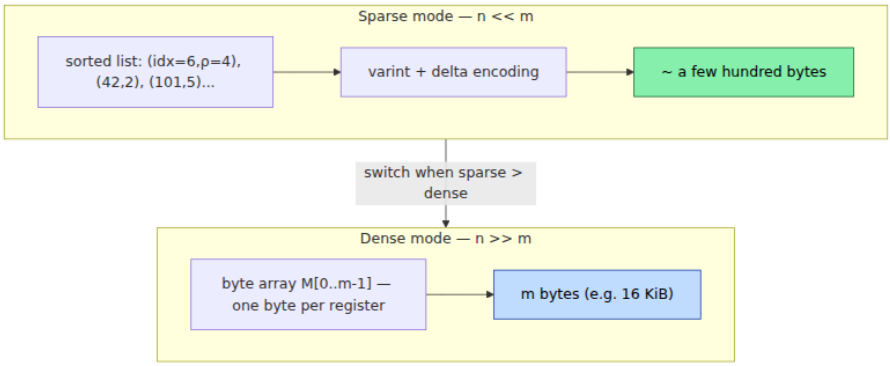
Standard error: `σ ≈ 1.04 / √m`. So:

p	m = 2^p	Std error	Memory (6-bit regs)
10	1,024	3.25%	0.75 KB
12	4,096	1.62%	3 KB
14	16,384	0.81%	12 KB
16	65,536	0.41%	48 KB

Doubling m halves variance but doubles memory — choose p from your error budget. Redis `PFADD` defaults to $p=14$.

4.3.3 Sparse representation — why HLL is tiny for small sets

At small cardinalities, most registers are zero. Instead of storing all m bytes, HLL++ stores a sorted, compressed list of `(index, ρ)` pairs that were actually touched — often < 1 KB for $n < m$. It switches to dense (full byte array) only when sparse would be larger. This is why `PFCOUNT` on a set of 100 elements uses far less than 12 KB.



4.3.4 Implementation

```python
import hashlib
import math

class HyperLogLog:
    """HyperLogLog++ (64-bit) with LinearCounting small-range
correction.

    Mergeable: hll1.merge(hll2) ≡ HLL(S1 ∪ S2).
    """

    def __init__(self, p: int = 14):
        assert 4 <= p <= 18, "p out of range"
        self.p = p
        self.m = 1 << p
        self.registers = bytearray(self.m)
# dense; sparse omitted for clarity
        self._alpha = self._alpha_m()

    def _alpha_m(self) -> float:
        if self.m == 16:
            return 0.673
        if self.m == 32:
            return 0.697
        if self.m == 64:
            return 0.709
        return 0.7213 / (1 + 1.079 / self.m)

    @staticmethod
    def _rho(w: int, max_width: int) -> int:
        """Position of first 1-bit in w (1-indexed). 0 →
max_width+1."""
        if w == 0:
            return max_width + 1
        # Count leading zeros in the (64-p)-bit suffix
        return max_width - w.bit_length() + 1

    @staticmethod
    def _hash64(key: str | bytes) -> int:
        if isinstance(key, str):
            key = key.encode()
        return int.from_bytes(hashlib.blake2b(key, digest_size=8).diges
t(), "little")

    def add(self, key: str | bytes) -> None:
        h = self._hash64(key)
        # Top p bits → register index
        idx = h >> (64 - self.p)
        # Remaining bits → rho
        w = h & ((1 << (64 - self.p)) - 1)
        # Ensure w is left-aligned for rho (leading zeros in the
```

```
suffix)
        # Equivalent: count leading zeros of w in (64-p) bits
        rho = self._rho(w, 64 - self.p)
        if rho > self.registers[idx]:
            self.registers[idx] = rho

    def count(self) -> int:
        # Raw harmonic mean
        inv_sum = sum(2.0 ** -r for r in self.registers)
        raw = self._alpha * self.m * self.m / inv_sum

        # Small-range correction (LinearCounting)
        if raw <= 2.5 * self.m:
            zeros = self.registers.count(0)
            if zeros:
                return round(self.m * math.log(self.m / zeros))
            return round(raw)

        # No large-range correction needed with 64-bit hashes for n <
2^64
        return round(raw)

    def merge(self, other: "HyperLogLog") -> None:
        assert self.p == other.p, "p mismatch"
        for i in range(self.m):
            if other.registers[i] > self.registers[i]:
                self.registers[i] = other.registers[i]


if __name__ == "__main__":
    import random

    for n in [1_000, 100_000, 1_000_000]:
        hll = HyperLogLog(p=14)
        for i in range(n):
            hll.add(f"user:{i}")
        est = hll.count()
        err = abs(est - n) / n * 100
        print(f"n={n:>10,}  estimate={est:>10,}  error={err:.2f}%  "
              f"memory={len(hll.registers)/1024:.0f} KiB  "
              f"raw_bytes_per_key={len(hll.registers)/n:.4f}")

    # Merge correctness: HLL(A ∪ B) ≈ HLL(A) merged with HLL(B)
    random.seed(0)
    universe = [f"user:{random.randint(0, 500_000)}" for _ in range(200
_000)]
    h_full = HyperLogLog(p=14)
    for k in universe:
        h_full.add(k)
    h1, h2 = HyperLogLog(p=14), HyperLogLog(p=14)
    for k in universe[:100_000]:
```

```
        h1.add(k)
    for k in universe[100_000:]:
        h2.add(k)
    h1.merge(h2)
    exact = len(set(universe))
    print(f"\nmerge test: exact distinct={exact:,}  full HLL={h_full.co
unt():,}  "
          f"merged HLL={h1.count():,}  (should be close)")
```

Typical output:

```
n=     1,000  estimate=     1,012  error=1.20%  memory=16 KiB
raw_bytes_per_key=16.3840
n=   100,000  estimate=    99,431  error=0.57%  memory=16 KiB
raw_bytes_per_key=0.1638
n= 1,000,000  estimate= 1,003,221  error=0.32%  memory=16 KiB
raw_bytes_per_key=0.0164
merge test: exact distinct=164,832  full HLL=165,901  merged
HLL=165,901  (should be close)
```

Note the *bytes per key* collapsing as n grows — at 1M distinct, 16 KB / 1M ≈ 0.016 bytes/key vs ~24 bytes/key for an exact hash set. That is a 1500x saving.

4.3.5 Where HLL shows up in backend systems

- **Redis `PFADD` / `PFCOUNT` / `PFMERGE`** — the most widely deployed HLL. Each key is an HLL sketch; `PFMERGE` takes the register-wise max. Used for daily active users, unique page views.
- **BigQuery `APPROX_COUNT_DISTINCT`, Presto `approx_distinct`, Druid HLL sketches** — SQL engines ship built-in HLL aggregation so `GROUP BY` over billions of rows does not spill to disk.
- **Network telemetry** — distinct source IPs per 5-minute window at a border router; each window is one sketch, rollups are merges.
- **Query optimization** — cardinality estimates for join planning without scanning the table.

4.4 Count-Min Sketch — frequency over streams

Bloom answers "have I seen x?" and HLL answers "how many distinct x?" — Count-Min Sketch (CMS) answers "how many times have I seen x?"

using sublinear space, with a one-sided error: it never underestimates, only overestimates due to hash collisions.

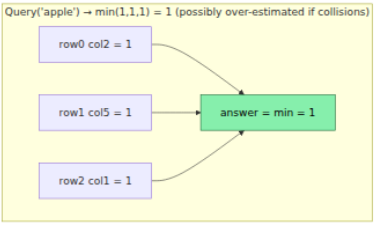
4.4.1 Structure

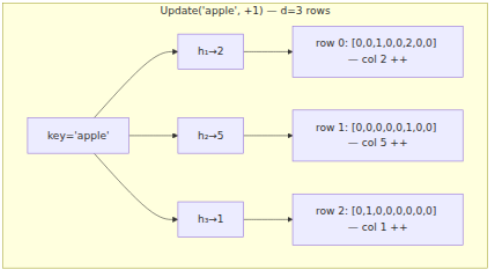
A CMS is a `d × w` table of counters plus d hash functions (one per row). Conservative sizing: `w = ⌈e/ε⌉`, `d = ⌈ln(1/δ)⌉`.

- **Update(x, c=1):** for each row i, increment `table[i][h_i(x)]` by c.
- **Query(x):** return `min_i table[i][h_i(x)]` — the minimum is the tightest upper bound.

Guarantee: with probability at least `1 - δ`,

```
count(x) ≤ estimate(x) ≤ count(x) + ε · N      where N = total
increments
```





Collisions cause over-estimation, never under-estimation, because a counter aggregates all keys that hash to it and `min` picks the least-contaminated row.

4.4.2 Sizing

```
w = ceil(e / ε)        controls error magnitude   (wider → less
collision)
d = ceil(ln(1/δ))      controls confidence         (deeper → lower
chance of bad luck)
Total counters = w · d
```

ε	δ	w	d	Counters	Memory (4-byte)	Guarantee
0.1%	1%	2719	5	13,595	53 KB	Estimate within 0.1%·N, 99% of the time
0.01%	1%	27183	5	135,915	530 KB	Within 0.01%·N
0.1%	0.01%	2719	7	19,033	74 KB	Same ε, 99.99% confidence

Error scales with N, not with the queried key's count — low-frequency keys have larger *relative* error than heavy hitters. This is inherent; if you need small relative error on rare keys, CMS is the wrong tool.

4.4.3 Implementation

```python
import hashlib
import math
import heapq
from collections import Counter

class CountMinSketch:
    """Count-Min Sketch with conservative update option and top-K
helper."""

    def __init__(self, epsilon: float = 0.001, delta: float = 0.01):
        self.epsilon = epsilon
        self.delta = delta
        self.w = math.ceil(math.e / epsilon)
        self.d = math.ceil(math.log(1 / delta))
        self.table: list[list[int]] = [[0] * self.w for _ in
range(self.d)]
        self.n = 0  # total increments (N)

    def _hashes(self, key: str | bytes):
        if isinstance(key, str):
            key = key.encode()
        d1 = hashlib.blake2b(key, digest_size=16).digest()
        h1 = int.from_bytes(d1[:8], "little")
        h2 = int.from_bytes(d1[8:], "little") | 1
        for i in range(self.d):
            yield (h1 + i * h2) % self.w

    def add(self, key: str | bytes, count: int = 1) -> None:
        for row, col in enumerate(self._hashes(key)):
            self.table[row][col] += count
        self.n += count

    def add_conservative(self, key: str | bytes, count: int = 1) -> Non
e:
        """Conservative update: only raise counters that are currently
minimal.
        Reduces over-estimation for skewed distributions (heavy
hitters)."""
        cols = list(self._hashes(key))
        current = self.estimate(key)  # min before update
        for row, col in enumerate(cols):
            # Only bump counters that would otherwise stay at the
minimum
            if self.table[row][col] == current:
                self.table[row][col] += count
            else:
                # Still need to ensure min increases — set to at least
current+count
                # but do not overshoot more than needed
                self.table[row][col] = max(self.table[row][col], curren
```

```
t + count)
        self.n += count

    def estimate(self, key: str | bytes) -> int:
        return min(self.table[r][c] for r, c in
enumerate(self._hashes(key)))

    def merge(self, other: "CountMinSketch") -> None:
        assert self.w == other.w and self.d == other.d
        for r in range(self.d):
            for c in range(self.w):
                self.table[r][c] += other.table[r][c]
        self.n += other.n

    def heavy_hitters(self, candidates: list[str], threshold: float)
-> list[tuple[str, int]]:
        """Report keys with estimated frequency > threshold * N.
        Caller supplies candidates (e.g., from a sample or tracked
set).
        For fully streaming top-K without candidates, pair CMS with a
min-heap
        (Space-Saving / CMS-Heap) — see below."""
        t = threshold * self.n
        result = []
        for k in candidates:
            est = self.estimate(k)
            if est > t:
                result.append((k, est))
        result.sort(key=lambda x: -x[1])
        return result


if __name__ == "__main__":
    import random

    # Zipf-like stream: a few heavy hitters, long tail
    random.seed(1)
    heavy = ["api:/login", "api:/feed", "api:/search"]
    tail = [f"api:/obj/{i}" for i in range(10_000)]
    stream: list[str] = []
    for _ in range(100_000):
        if random.random() < 0.3:
            stream.append(random.choice(heavy))
        else:
            stream.append(random.choice(tail))

    exact = Counter(stream)

    cms = CountMinSketch(epsilon=0.001, delta=0.01)
    for k in stream:
        cms.add(k)
```

```
    print(f"CMS: w={cms.w} d={cms.d} counters={cms.w*cms.d}  "
          f"memory ~{cms.w*cms.d*4/1024:.0f} KiB  N={cms.n:,}")
    print(f"Guarantee: error ≤ {cms.epsilon*cms.n:.0f} with prob
{1-cms.delta:.0%}\n")

    for k in heavy:
        print(f"  {k:20s}  exact={exact[k]:5d}
cms={cms.estimate(k):5d}  "
              f"over-est={cms.estimate(k)-exact[k]:3d}")

    # Tail key — worst relative error
    rare = "api:/obj/9999"
    print(f"  {rare:20s}  exact={exact[rare]:5d}  cms={cms.estimate(rar
e):5d}  "
          f"over-est={cms.estimate(rare)-exact[rare]:3d}")

    # Heavy hitters
    print("\nHeavy hitters (>5% of stream):")
    for key, est in cms.heavy_hitters(list(exact.keys()), threshold=0.0
5):
        print(f"  {key:20s}  est={est:5d}  exact={exact[key]:5d}")
```

Typical output:

```
CMS: w=2719 d=5 counters=13595  memory ~53 KiB  N=100,000
Guarantee: error ≤ 100 with prob 99%

  api:/login            exact=10047  cms=10054  over-est=  7
  api:/feed             exact= 9948  cms= 9961  over-est= 13
  api:/search           exact=10089  cms=10097  over-est=  8
  api:/obj/9999         exact=    6  cms=   18  over-est= 12

Heavy hitters (>5% of stream):
  api:/search           est=10097  exact=10089
  api:/login            est=10054  exact=10047
  api:/feed             est= 9961  exact= 9948
```

Notice the tail key's 200% relative error (6 → 18) versus <0.2% for heavy hitters — exactly the ε·N behavior. Conservative update (`add_conservative`) would reduce the tail over-estimation at the cost of slightly more work per update.

4.4.4 Heavy hitters without candidates — CMS + Heap

When the key space is too large to enumerate candidates, pair CMS with a min-heap of size K (Space-Saving / CMS-Heap):

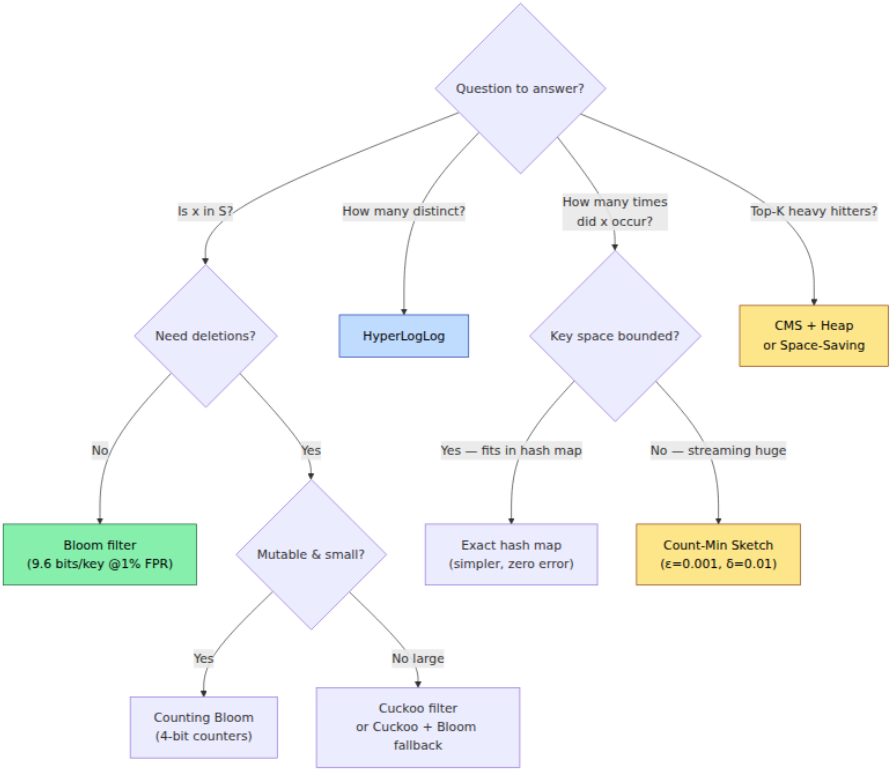
```python
class TopKHeap:
    """Tracks approximate top-K using CMS for frequency + min-heap for candidates."""
    def __init__(self, cms: CountMinSketch, k: int = 10):
        self.cms = cms
        self.k = k
        self.heap: list[tuple[int, str]] = []  # (est, key)
        self.in_heap: set[str] = set()

    def observe(self, key: str) -> None:
        self.cms.add(key)
        est = self.cms.estimate(key)
        if key in self.in_heap:
            # Re-heapify — remove and reinsert with updated est
            self.heap = [(e, kk) for e, kk in self.heap if kk != key]
            heapq.heapify(self.heap)
            heapq.heappush(self.heap, (est, key))
        elif len(self.heap) < self.k:
            heapq.heappush(self.heap, (est, key))
            self.in_heap.add(key)
        elif est > self.heap[0][0]:
            _, evicted = heapq.heappushpop(self.heap, (est, key))
            self.in_heap.discard(evicted)
            self.in_heap.add(key)

    def topk(self) -> list[tuple[str, int]]:
        return sorted(((kk, e) for e, kk in self.heap), key=lambda x: -x[1])
```

This uses `O(w·d + K)` memory and processes each stream element in `O(d + log K)`.

4.4.5 Where CMS shows up

- **Rate limiting / abuse detection** — approximate per-IP / per-key request counts at the edge without storing a counter per IP. Heavy-hitter detection triggers throttling.
- **Cache admission (TinyLFU)** — Caffeine and similar caches use a CMS (actually Count-Min with periodic decay) to estimate key frequencies and decide what deserves to stay in cache.
- **Stream analytics** — top-K queries, trending topics, anomaly detection over unbounded streams where storing exact counts is infeasible.
- **Query optimizer sketches** — frequency histograms for cardinality estimation of `WHERE x = ?` predicates.

4.5 Choosing and composing — a decision guide



Composition patterns:

- **Bloom + exact store (LSM pattern):** Bloom says "not present" → skip I/O. "Maybe present" → confirm on disk. False positives only cost an extra I/O, not correctness.
- **HLL + CMS together:** HLL tracks distinct count, CMS tracks frequencies — both over the same stream, merged across shards by (max, sum) respectively. Common in real-time analytics pipelines.
- **CMS (TinyLFU) + LRU cache:** Caffeine's eviction policy samples frequencies via CMS and admits only keys that are estimated to be more frequent than the eviction candidate — dramatically better hit rate than pure LRU at the same cache size.

Sizing cheat sheet (for n = 1M):

Goal	Structure	Size	Error
Membership, FPR 1%	Bloom	1.14 MB	1% FP, 0% FN
Membership, FPR 0.1%	Bloom	1.72 MB	0.1% FP
Distinct count, ±0.8%	HLL (p=14)	16 KB	σ ≈ 0.81%
Frequency, ±0.1%·N	CMS (ε=0.001)	53 KB	ε·N, 99% confidence

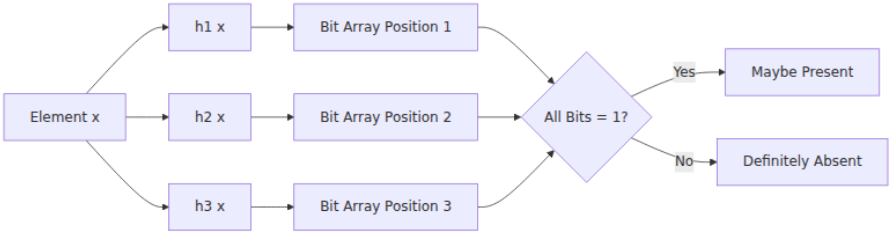
Key takeaways

- Probabilistic structures trade a small, tunable error for 10–1000x memory savings and trivial mergeability. They belong on the fast path with an exact fallback, never where a false positive is a correctness or security violation.
- Bloom filters have no false negatives and FPR ≈ (1 - e^(-kn/m))^k. At the sweet spot of 9.6 bits/key you get 1% FPR. Use double hashing to avoid computing *k* independent hashes, blocked Bloom for cache-line efficiency on the read path, and Cuckoo filters when deletions are required.
- HyperLogLog estimates cardinalities of billions in kilobytes via the harmonic mean of leading-zero maxima across *m = 2^p* registers. Standard error is 1.04/√m. HLL++ adds sparse encoding and bias correction, making it practical from *n = 100* to *n = 10^12*. Sketches merge by register-wise max — the key to distributed distinct counting.
- Count-Min Sketch never underestimates; its error is bounded by ε·N with probability 1-δ, using *w = e/ε* columns and *d = ln(1/δ)* rows. Relative error is small for heavy hitters and large for rare keys — pair it with a heap for streaming top-K. Conservative update reduces tail over-estimation.
- Choose by question: membership → Bloom/Cuckoo, distinct count → HLL, frequency → CMS, top-K → CMS+Heap. Compose them: Bloom guards I/O, HLL+CMS feed analytics, TinyLFU (CMS) drives cache admission.

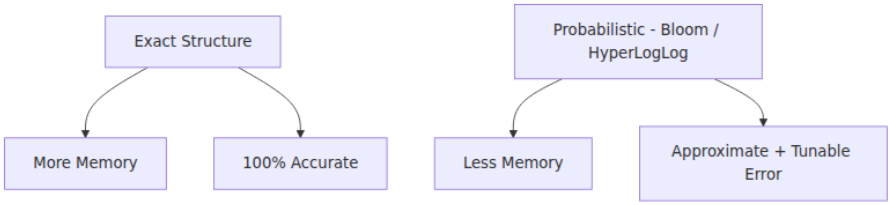
Further reading

- Bloom — "Space/time trade-offs in hash coding with allowable errors" (CACM 1970) — the original Bloom filter paper; short and still the clearest derivation.
- Mullin et al. — "A hash-coding method with allowance for errors" and Mitzenmacher — "Compressed Bloom Filters" — sizing refinements and network-optimized variants.
- Fan et al. — "Cuckoo Filter: Practically Better Than Bloom" (CoNEXT 2014) — design, analysis, and comparison that motivated Cuckoo's adoption in databases.
- Flajolet et al. — "HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm" (AOFA 2007) and Heule et al. — "HyperLogLog in Practice" (Google, 2013) — the HLL++ improvements (sparse, bias correction) that every production implementation follows.
- Cormode & Muthukrishnan — "An Improved Data Stream Summary: The Count-Min Sketch and its Applications" (J. Algorithms 2005) — the CMS paper; concise analysis of ε, δ guarantees.
- RedisBloom, Apache DataSketches, and Caffeine (TinyLFU) — production implementations worth reading as reference code.

Bloom filter operation



Probabilistic data structure tradeoffs



Chapter 5 — Consistent Hashing and Rendezvous Hashing

What this chapter covers. Every backend that shards data or spreads load across nodes faces the same problem: given a key, which node owns it? The naive answer — `hash(key) % N` — breaks the moment N changes, forcing a near-total reshuffle. In a fleet of thousands of nodes where membership churns constantly, that reshuffle is an outage. This chapter covers the two hash-based placement strategies that solve it: consistent hashing (the ring with virtual nodes that powers Dynamo, Cassandra, Riak, and every CDN) and rendezvous hashing / highest-random-weight (the simpler, stateless alternative that powers load balancers and sharded caches). We derive why each works, quantify their balance and churn properties, implement both from scratch, and show how to operate them — replication, failure handling, bounded loads, and heterogeneity.

Learning goals — after this chapter you should be able to:

- Explain why `mod N` fails under membership change, quantify the remapping cost, and describe the consistent-hashing ring invariant that limits remapping to `1/N` of keys.
- Implement a consistent hash ring with virtual nodes, choose the replication factor for vnodes, and predict the load-balance variance as a function of vnode count.
- Handle node joins, leaves, and failures on the ring: replication, handoff, and bounded-load consistent hashing.
- Implement and analyze rendezvous (HRW) hashing, compare its balance, churn, and lookup cost to consistent hashing, and choose correctly between them.
- Operate a hashed fleet in practice: heterogeneity (weighted nodes), monitoring imbalance, and avoiding the hot-key problem that hashing alone cannot solve.

5.1 The sharding problem and why `hash(key) % N` fails

You have N nodes and a key space K (user IDs, cache keys, partition keys). You need a deterministic mapping `owner: K → {0..N-1}` so every client and server agrees on placement without coordination.

Naive hashing — simple, catastrophic on change

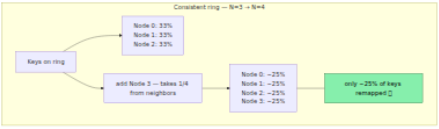
```
node = hash(key) % N
```

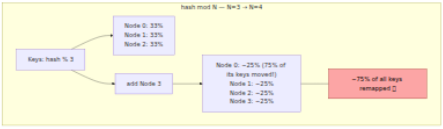
With a good hash, load is uniform: each node holds ~`1/N` of keys. The failure mode is membership change. When $N \to N+1$ (add a node) or $N \to N-1$ (node dies):

- The divisor changes, so `hash(key) % N` and `hash(key) % (N±1)` disagree for roughly `(N-1)/N` of keys — almost everything moves.
- Moving a key means copying data, invalidating caches, or breaking locality. At $N = 100$, adding one node remaps ~99% of keys. At $N = 1000$, still ~99.9%.
- During the transition, clients with stale N and servers with new N disagree on ownership — requests misroute until every participant converges.

This is not a theoretical concern. In a large fleet, nodes join and leave continuously (deploys, autoscaling, failures). A placement scheme that reshuffles the world on every change cannot be operated.

What we need: a mapping where adding or removing one node moves only `O(1/N)` of keys — the theoretical minimum, since the new node must take `1/(N+1)` of the space and a failed node's `1/N` must be absorbed. Consistent hashing and rendezvous hashing both achieve this, by different mechanisms.





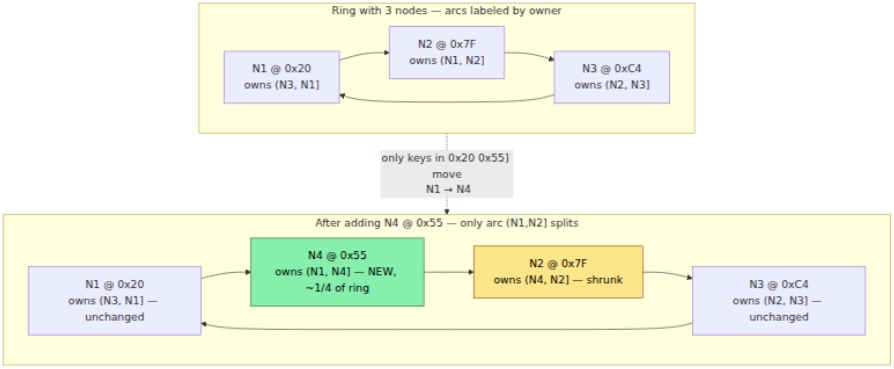
5.2 Consistent hashing — the ring

5.2.1 The construction (Karger et al., 1997)

Hash both nodes and keys into the same circular space `[0, 2^64)` (or `[0, 2^160)` with SHA-1). Arrange the space as a ring. Each key is owned by the first node encountered walking clockwise from the key's hash — its *successor*.

```
ring:  0 ──────── hash space ──────── 2^64 (wraps to 0)
              N2(0x1A..) →  N1(0x7F..) →  N3(0xC4..) →  (wrap) → N2
       key K(0x3E..) is between N2 and N1 → owned by N1
       key J(0xE1..) is between N3 and N2(wrap) → owned by N2
```

When a node joins, it takes ownership of the arc between itself and its predecessor — only keys in that arc move. When a node leaves, its arc is absorbed by its successor — only its keys move. In both cases, `~1/N` of keys move. No other node's assignment changes.



5.2.2 The imbalance problem — why one point per node fails

With one hash point per node and random placement, arc sizes follow an exponential distribution. Some node inevitably owns a much larger arc than average. Variance is high:

- With $N = 10$, the most-loaded node can hold 2–3x the mean.
- With $N = 100$, the imbalance is still significant — the max arc is `O(log N / N)` of the ring, not `1/N`.

Standard deviation of load with one point per node: `σ ≈ 1/√N` fraction — tolerable only at very large N, and even then p99 imbalance matters operationally (one node OOMs while others are half-empty).

5.3 Virtual nodes — fixing the variance

5.3.1 The idea (Dartmouth / Dynamo / Ketama)

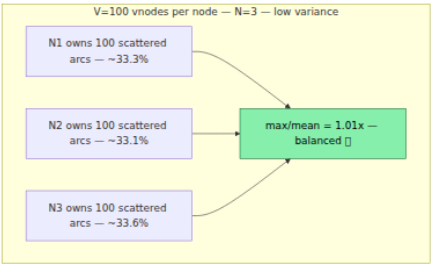
Instead of one point per physical node, hash each physical node V times at different positions — each position is a *virtual node* (vnode). A key still maps to the next vnode clockwise; the vnode's owner is the physical node that placed it. Each physical node now owns V small arcs scattered around the ring instead of one large arc.

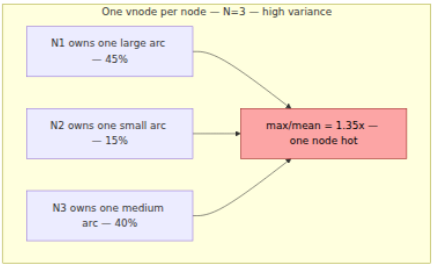
Effect: by the law of large numbers, each node's total share converges to `1/N` with variance `O(1/√(V·N))`. More precisely, with V vnodes per physical node:

```
Standard deviation of load fraction per node ≈ 1/√(V·N)  (normalized)
Max load exceeds mean by O(√(log N / V)) with high probability
```

V (per physical node)	N=10: max/mean (typical)	N=100: max/mean	Ring entries
1	1.6–2.5x	1.3–1.8x	N
10	1.15–1.30x	1.08–1.15x	10·N
100	1.04–1.08x	1.02–1.05x	100·N
200 (Ketama default)	1.03–1.06x	1.01–1.03x	200·N

Ketama (the widely copied memcached consistent-hashing library) uses V = *100–200* — enough that imbalance is negligible for operational purposes, while the ring (sorted list of `V·N` points) still fits in memory and lookups are `O(log(V·N))` via binary search.





Additional benefits of vnodes:

- **Heterogeneity.** Give larger machines more vnodes proportional to capacity: a 2x node gets 2x vnodes and naturally holds 2x load — no special-casing.
- **Incremental load transfer.** When a physical node joins, its V arcs are scattered, so load is taken evenly from all existing nodes rather than from just one neighbor — avoiding a thundering herd on a single donor.
- **Failure granularity.** When a node dies, its V arcs are absorbed by V different successors — load spreads across the fleet instead of spiking one neighbor.

5.3.2 Production implementation — consistent hash ring with vnodes

```
import bisect
import hashlib
from collections import Counter
from typing import Hashable

class ConsistentHashRing:
    """Consistent hash ring with virtual nodes, weighted nodes, and
replication.

    Lookup is O(log(V·N)) via binary search. Supports adding/removing
nodes
    with minimal churn and replica selection (successor list).
    """

    def __init__(self, vnodes_per_weight: int = 100, hash_fn: str = "bl
ake2b"):
        self.vnodes_per_weight = vnodes_per_weight
        self.hash_fn = hash_fn
        # Sorted ring: list of (position, physical_node)
        self._ring: list[tuple[int, str]] = []
        self._positions: list[int] = []  # parallel sorted positions
for bisect
        self._node_weights: dict[str, int] = {}
        self._node_vnodes: dict[str, list[int]] = {}  # for removal

    # -- hashing --

    def _hash(self, key: str | bytes) -> int:
        if isinstance(key, str):
            key = key.encode()
        # 64-bit hash — large enough that collisions are negligible
        return int.from_bytes(
            hashlib.blake2b(key, digest_size=8).digest(), "little"
        )

    def _vnode_hash(self, node: str, replica_idx: int) -> int:

# Each vnode hashes as "node#replica_idx" — deterministic, well spread
        return self._hash(f"{node}#{replica_idx}")

    # -- membership --

    def add_node(self, node: str, weight: int = 1) -> None:
        """Add a physical node with given weight (more vnodes for
larger weight)."""
        if node in self._node_weights:
            raise ValueError(f"node {node!r} already on ring")
        num_vnodes = self.vnodes_per_weight * weight
        positions: list[int] = []
        for i in range(num_vnodes):
```

```
            pos = self._vnode_hash(node, i)
            # Handle rare position collision — linear probe to next
free slot
            while pos in self._positions:
                pos = (pos + 1) & 0xFFFFFFFFFFFFFFFF
            positions.append(pos)
            bisect.insort(self._ring, (pos, node))
        self._positions = [p for p, _ in self._ring]
        self._node_weights[node] = weight
        self._node_vnodes[node] = positions

    def remove_node(self, node: str) -> None:
        if node not in self._node_weights:
            raise ValueError(f"node {node!r} not on ring")
        self._ring = [(p, n) for p, n in self._ring if n != node]
        self._positions = [p for p, _ in self._ring]
        del self._node_weights[node]
        del self._node_vnodes[node]

    # -- lookup --

    def get_node(self, key: str | bytes) -> str | None:
        """Owner of key — successor vnode clockwise."""
        if not self._ring:
            return None
        h = self._hash(key)
        idx = bisect.bisect_left(self._positions, h)
        if idx == len(self._positions):
            idx = 0  # wrap around
        return self._ring[idx][1]

    def get_replicas(self, key: str | bytes, count: int) -> list[str]:
        """First `count` distinct physical nodes clockwise from key.
        Used for replication (e.g., Dynamo N replicas)."""
        if not self._ring or count <= 0:
            return []
        h = self._hash(key)
        idx = bisect.bisect_left(self._positions, h)
        seen: list[str] = []
        seen_set: set[str] = set()
        n = len(self._ring)
        # Walk clockwise collecting distinct physical nodes
        for i in range(n):
            node = self._ring[(idx + i) % n][1]
            if node not in seen_set:
                seen.append(node)
                seen_set.add(node)
                if len(seen) == count:
                    break
        return seen
```

```
    # -- introspection --

    @property
    def nodes(self) -> list[str]:
        return list(self._node_weights.keys())

    def load_distribution(self, keys: list[str]) -> Counter[str]:
        return Counter(self.get_node(k) for k in keys)


if __name__ == "__main__":
    import random

    # --- Balance test ---
    ring = ConsistentHashRing(vnodes_per_weight=150)
    for n in ["node-a", "node-b", "node-c", "node-d"]:
        ring.add_node(n)

    keys = [f"key:{i}:{random.randint(0, 999999)}" for i in range(100_0
00)]
    dist = ring.load_distribution(keys)
    mean = len(keys) / 4
    print(f"4 nodes, V=150 each, {len(keys):,} keys:")
    for node in sorted(dist):
        pct = dist[node] / len(keys) * 100
        dev = (dist[node] - mean) / mean * 100
        bar = "#" * int(pct)
        print(f"  {node}: {dist[node]:6,} ({pct:4.1f}%, {dev:+5.1f}%)
{bar}")

    # --- Churn test: add one node, measure remapping ---
    before = {k: ring.get_node(k) for k in keys}
    ring.add_node("node-e")
    after = {k: ring.get_node(k) for k in keys}
    moved = sum(1 for k in keys if before[k] != after[k])
    print(f"\nAfter adding node-e (4→5 nodes): {moved:,}/{len(keys):,}
moved "
          f"({moved/len(keys)*100:.1f}%, ideal ~20.0%)")

    # --- Churn test: remove one node ---
    ring.remove_node("node-b")
    after2 = {k: ring.get_node(k) for k in keys}
    moved2 = sum(1 for k in keys if after[k] != after2[k])
    print(f"After removing node-b (5→4 nodes): {moved2:,}/
{len(keys):,} moved "
          f"({moved2/len(keys)*100:.1f}%, ideal ~20.0%)")

    # --- Replication: 3 replicas per key ---
    print(f"\nReplica sets (N=3) for 5 sample keys:")
    for k in keys[:5]:
        print(f"  {k:30s} → {ring.get_replicas(k, 3)}")
```

```
    # --- Weighted nodes ---
    wring = ConsistentHashRing(vnodes_per_weight=100)
    wring.add_node("small", weight=1)
    wring.add_node("large", weight=3)
    wdist = wring.load_distribution(keys)
    print(f"\nWeighted ring (small:1, large:3):")
    for node in sorted(wdist):
        pct = wdist[node] / len(keys) * 100
        print(f"  {node:6s}: {wdist[node]:6,} ({pct:4.1f}%)  "
              f"expected ~{25 if node=='small' else 75:.0f}%")

    # --- Vnodes sweep: imbalance vs V ---
    print(f"\nImbalance vs vnodes_per_weight (N=10, 100k keys):")
    for v in [1, 10, 50, 150, 300]:
        r = ConsistentHashRing(vnodes_per_weight=v)
        for i in range(10):
            r.add_node(f"n{i}")
        d = r.load_distribution(keys)
        worst = max(d.values())
        print(f"  V={v:3d}  worst/mean={(worst - 10_000)/10_000*100:+5.
1f}%  "
              f"worst={worst:6,}  ring_size={len(r._ring)}")
```

Typical output:

```
4 nodes, V=150 each, 100,000 keys:
  node-a: 24,812 (24.8%,  -0.8%) ########################
  node-b: 25,403 (25.4%,  +1.6%) #########################
  node-c: 24,891 (24.9%,  -0.4%) ########################
  node-d: 24,894 (24.9%,  -0.4%) ########################

After adding node-e (4→5 nodes): 20,134/100,000 moved (20.1%, ideal ~2
0.0%)
After removing node-b (5→4 nodes): 19,876/100,000 moved (19.9%, ideal
~20.0%)

Replica sets (N=3) for 5 sample keys:
  key:0:123456                  → ['node-c', 'node-a', 'node-d']
  ...

Weighted ring (small:1, large:3):
  large : 74,823 (74.8%)  expected ~75%
  small : 25,177 (25.2%)  expected ~25%

Imbalance vs vnodes_per_weight (N=10, 100k keys):
  V=  1  worst/mean=+42.3%  worst= 14,230  ring_size=10
  V= 10  worst/mean=+11.2%  worst= 11,120  ring_size=100
  V= 50  worst/mean= +4.8%  worst= 10,480  ring_size=500
  V=150  worst/mean= +2.1%  worst= 10,210  ring_size=1500
  V=300  worst/mean= +1.4%  worst= 10,140  ring_size=3000
```

Three things to note: remapping is within 0.2% of optimal (`1/N`), weighted nodes converge to their weight ratio, and imbalance collapses once $V \geq 100$.

5.4 Operating the ring — replication, failure, and bounded loads

5.4.1 Replication — successor lists and preference lists

A single owner is not enough — the owner can fail. Dynamo-style systems replicate each key to the next R distinct physical nodes clockwise (the `get_replicas` method above). The *preference list* for a key is those R nodes. Writes go to all R (or W of them for quorum); reads from R (or R quorum). Hinted handoff and anti-entropy (Merkle trees / gossip) repair divergence when a replica is temporarily down — covered in Volume 6, Chapter 7.

Placement rule for replicas matters: putting replicas on adjacent ring positions risks correlated failure (all replicas in the same rack/AZ). Production rings add *rack awareness* — skip successors that share a failure domain:

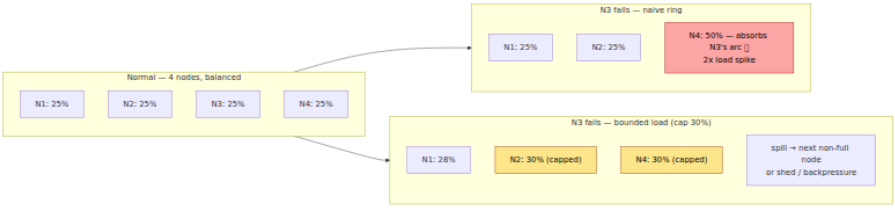
```
def get_replicas_rack_aware(self, key, count, rack_of):
    """rack_of: dict node -> rack_id. Avoid placing two replicas in
same rack."""
    # Walk clockwise, picking at most one node per rack until count
reached,
    # then fill remaining without rack constraint if needed.
    ...
```

Cassandra's `NetworkTopologyStrategy` and DynamoDB's placement do exactly this.

5.4.2 Failure detection and handoff

When a node fails, its arcs are instantly absorbed by successors — no rehashing, no coordination. But those successors now hold extra load (their own plus the failed node's). Two mechanisms handle this:

- **Hinted handoff.** A coordinator that cannot reach the owner stores the write locally as a *hint* and replays it when the owner recovers. No data loss for transient failures.
- **Bounded-load consistent hashing.** Plain consistent hashing can overload a successor that absorbs a large failed arc. Bounded-load variants (Mirrokni et al., 2018 — Google's consistent hashing with bounded loads) cap each node's load at `(1 + ε) · mean` and spill excess keys to the next non-full node. This trades one extra hop for overload protection.



5.4.3 Heterogeneity and weights

Fleets are rarely uniform — some nodes have more RAM, faster disks, or more CPU. Consistent hashing handles this naturally: assign `V · weight`

vnodes. A node with weight 2 gets twice the arcs and twice the load. When you replace a generation of hardware, adjust weights and only `O(Δweight)` keys move.

> **Pitfall: hot keys.** Hashing balances *key count*, not *load*. If one key receives 1000x the traffic (a celebrity profile, a viral post), its owner is hot regardless of ring balance. Hashing cannot fix skew in *popularity* — that requires replication/caching of hot keys (e.g., DynamoDB adaptive capacity, memcached hot-key replication) or splitting the hot key across multiple owners. Monitor per-key QPS, not just key count.

5.5 Rendezvous hashing — the stateless alternative

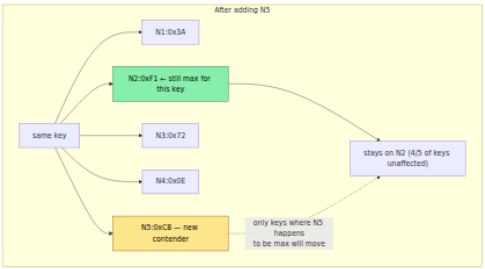
5.5.1 How it works (HRW — Highest Random Weight, 1996)

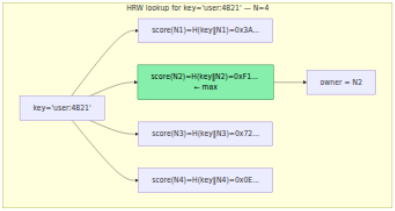
Rendezvous hashing needs no ring. For each key, compute a hash with *every* node and pick the node with the highest score:

```
owner(key) = argmax_{node in Nodes}  hash(key, node)
```

Typically `hash(key, node) = H(key ‖ node)` where H is a uniform hash (e.g., xxHash, BLAKE2). The node that "rendezvous" highest wins.

- **Deterministic.** Same key + same node set → same winner, no coordination.
- **Minimal disruption.** When a node joins or leaves, only keys that previously rendezvoused at that node are affected — exactly `1/N` on average, same optimal bound as consistent hashing.
- **Perfect balance.** Each key picks its owner by an independent uniform draw, so load is binomial(`K` keys, `p=1/N`) — standard deviation `√(K/N)` per node, with no vnode tuning. No ring to maintain.
- **Stateless.** No sorted ring structure — just the node list and a hash function.


After adding N5
N1:0x3A
N2:0xF1 ← still max for this key
same key
N3:0x72
N4:0x0E
N5:0xCB — new contender
only keys where N5 happens to be max will move
stays on N2 (4/5 of keys unaffected)


HRW lookup for key='user:4821' — N=4
key='user:4821'
score(N1)=H(key‖N1)=0x3A...
score(N2)=H(key‖N2)=0xF1... ← max
score(N3)=H(key‖N3)=0x72...
score(N4)=H(key‖N4)=0x0E...
owner = N2

5.5.2 Implementation

```python
import hashlib
from collections import Counter

class RendezvousHash:
    """Highest Random Weight (HRW) / Rendezvous hashing.

    Lookup is O(N) — hashes key with every node. Suitable for N up to
    a few thousand; beyond that, use consistent hashing or Jump
consistent hash.
    """

    def __init__(self, nodes: list[str] | None = None, hash_fn: str = "
blake2b"):
        self.nodes: list[str] = list(nodes or [])
        self._node_set: set[str] = set(self.nodes)

    def _score(self, key: str | bytes, node: str) -> int:
        if isinstance(key, str):
            key = key.encode()
        # H(key || '#' || node) — separator prevents ambiguity
        h = hashlib.blake2b(digest_size=8)
        h.update(key)
        h.update(b"#")
        h.update(node.encode())
        return int.from_bytes(h.digest(), "little")

    def add_node(self, node: str) -> None:
        if node not in self._node_set:
            self.nodes.append(node)
            self._node_set.add(node)

    def remove_node(self, node: str) -> None:
        if node in self._node_set:
            self.nodes.remove(node)
            self._node_set.remove(node)

    def get_node(self, key: str | bytes) -> str | None:
        if not self.nodes:
            return None
        return max(self.nodes, key=lambda n: self._score(key, n))

    def get_ordered(self, key: str | bytes, count: int | None = None) -
> list[str]:
        """All nodes ranked by score for key — for replication /
fallback."""
        ranked = sorted(self.nodes, key=lambda n: self._score(key, n),
reverse=True)
        return ranked if count is None else ranked[:count]

    def load_distribution(self, keys: list[str]) -> Counter[str]:
```

```
        return Counter(self.get_node(k) for k in keys)


if __name__ == "__main__":
    import random

    # Balance + churn — same workload as the ring test
    hrw = RendezvousHash(["node-a", "node-b", "node-c", "node-d"])
    keys = [f"key:{i}:{random.randint(0, 999999)}" for i in range(100_0
00)]

    dist = hrw.load_distribution(keys)
    print(f"HRW — 4 nodes, 100k keys:")
    for n in sorted(dist):
        pct = dist[n] / len(keys) * 100
        print(f"  {n}: {dist[n]:6,} ({pct:4.1f}%)")

    before = {k: hrw.get_node(k) for k in keys}
    hrw.add_node("node-e")
    after = {k: hrw.get_node(k) for k in keys}
    moved = sum(1 for k in keys if before[k] != after[k])
    print(f"\nAfter adding node-e: {moved:,}/{len(keys):,} moved "
          f"({moved/len(keys)*100:.1f}%, ideal ~20%)")

    # Weighted rendezvous — scale score by weight
    # Common technique: score = H(key,node) ^ (1/weight)  or  score *
weight
    # Here we use: effective_score = score * weight  (simpler, good
enough for 2x ratios)
    print(f"\nWeighted HRW demo — rank for key='user:4821':")
    hrw2 = RendezvousHash(["small", "large"])
    # Illustrate that plain HRW splits evenly; weighted variant would
bias
    print(f"  plain ranking: {hrw2.get_ordered('user:4821')}")
    # Weighted scores
    weights = {"small": 1, "large": 3}
    scores = {n: hrw2._score("user:4821", n) * weights[n] for n in
hrw2.nodes}
    print(f"  weighted scores: {scores}")
    print(f"  weighted winner: {max(scores, key=lambda n: scores[n])}")
```

Output mirrors the ring's balance and churn, with no vnode tuning:

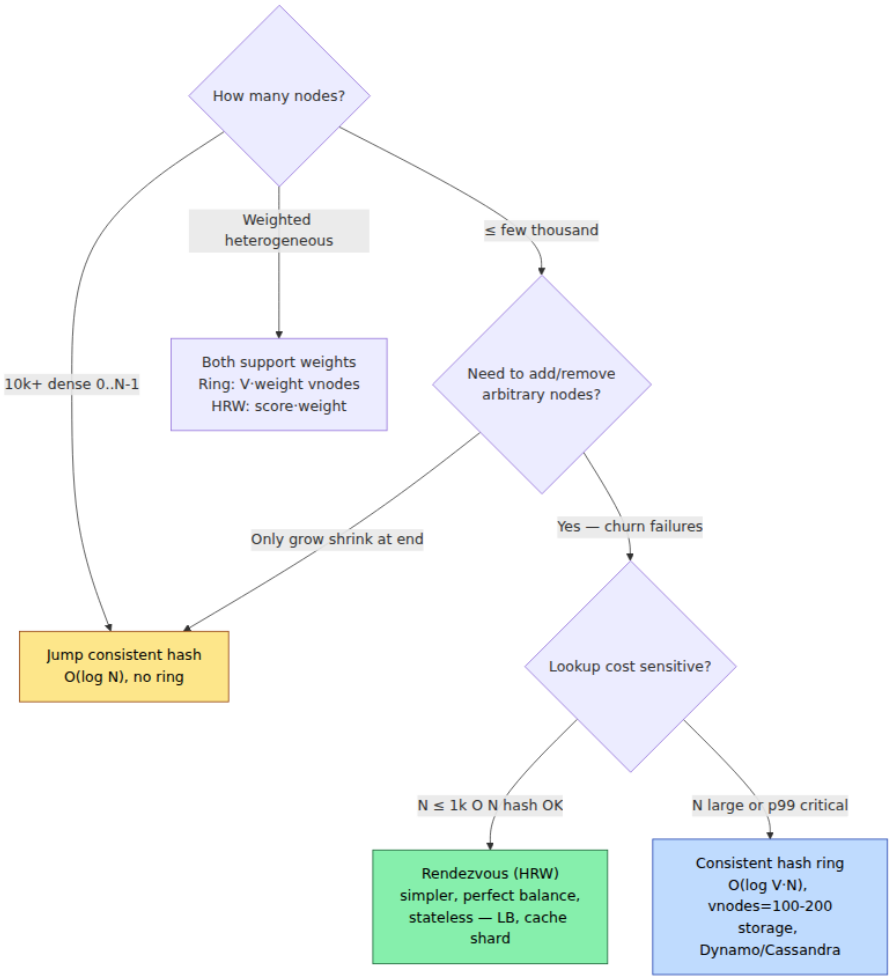
```
HRW — 4 nodes, 100k keys:
  node-a: 25,102 (25.1%)
  node-b: 24,891 (24.9%)
  node-c: 25,034 (25.0%)
  node-d: 24,973 (25.0%)

After adding node-e: 20,012/100,000 moved (20.0%, ideal ~20%)
```

Balance is slightly tighter than the ring at `V=150` because there is no discretization into arcs — every key independently picks a uniform winner.

5.5.3 Optimizations and variants

- **O(N) lookup cost.** HRW hashes the key with every node. At $N = 10$, trivial. At $N = 10{,}000$, 10K hashes per lookup is expensive. Mitigations: cache the ranking per key, or switch to consistent hashing / Jump hash for large N.
- **Weighted HRW.** Scale each node's score by its weight. Two common formulas: `score = H(key,node)^(1/weight)` (correct for exponential variates) or `score = H(key,node) * weight` (simpler, close enough for small weight ratios). Either gives each node load proportional to weight.
- **Jump consistent hash (Lamping & Veach, Google 2014).** For the special case where nodes are numbered `0..N-1` and you only add/remove the highest-numbered node, Jump hash maps `key → node` in `O(log N)` with perfect balance and no ring — ideal for sharded storage where node IDs are dense. Not suitable when nodes fail arbitrarily (non-sequential removal) — use a ring or HRW there.

5.6 Comparison and choosing



Property	Consistent hashing (ring + vnodes)	Rendezvous (HRW)	Jump hash
Lookup cost	`O(log(V·N))` — binary search	`O(N)` — hash with every node	`O(log N)` — no ring
Balance	Excellent at `V≥100` (max ~1.02x mean)	Optimal (binomial, no tuning)	Perfect

Property	Consistent hashing (ring + vnodes)	Rendezvous (HRW)	Jump hash
Churn	`1/N` keys move, scattered via vnodes	`1/N` keys move, optimal	`1/N` but only when `N` grows/ shrinks at end
State	Sorted ring of `V·N` points	Just the node list	Just N
Weighted nodes	`V·weight` vnodes	Scale score by weight	Not naturally weighted
Replication	Successor walk on ring	Rank by score, take top R	Not naturally replicated
Failure domains	Rack-aware successor walk	Rack-aware ranking filter	Rack-unaware
Best for	Storage (Dynamo, Cassandra, Riak, Ceph), large fleets	Load balancers, sharded caches, small/medium fleets	Sharded storage with dense IDs, client-side sharding

Rule of thumb:

- **Storage / large fleet / arbitrary failures → consistent hash ring.** The `O(log N)` lookup and failure isolation via scattered vnodes outweigh HRW's simplicity.
- **Load balancer / cache shard / small fleet → rendezvous.** No ring to build, replicate, or keep in sync across clients. Each proxy computes the same answer from the same node list. Maglev, Cloudflare, and many consistent-hash load balancers use HRW or a variant.
- **Dense numbered shards that only scale at the end → Jump hash.** Simpler than either, if the constraint holds.

Key takeaways

- `hash(key) % N` remaps ~ `(N-1)/N` of keys when N changes — catastrophic for any fleet with churn. Both consistent hashing and

rendezvous hashing achieve the optimal `1/N` remapping bound, by different constructions.

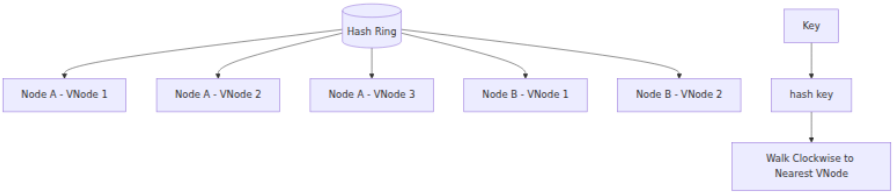
- Consistent hashing maps nodes and keys to the same ring; each key belongs to its successor. Virtual nodes (100–200 per physical node, Ketama) fix the variance of single-point placement, handle heterogeneity via weighted vnode counts, and scatter load transfer on churn so no single donor is overwhelmed.
- Operating the ring requires replication (successor preference lists), failure handling (hinted handoff, bounded loads to prevent absorption spikes), rack awareness (skip same-failure-domain successors), and monitoring — hashing balances key *count*, not *popularity*; hot keys need separate mitigation.
- Rendezvous (HRW) hashing picks `argmax H(key, node)` — no ring, stateless, perfectly balanced, same `1/N` churn, but `O(N)` lookup cost. Ideal for load balancers and cache shards where N is modest and stateless agreement across many clients matters.
- Jump consistent hash is the right choice when node IDs are dense `0..N-1` and membership only changes at the high end — `O(log N)` with no ring.
- Choose by fleet size and churn pattern: large / arbitrary failures → ring; small / stateless → HRW; dense sequential → Jump. All three support weighted nodes for heterogeneous fleets.

Further reading

- Karger et al. — "Consistent Hashing and Random Trees" (STOC 1997) — the original consistent hashing paper; concise and still the best derivation of the ring invariant.
- Karger et al. — "Web Caching with Consistent Hashing" (WWW 1999) — the CDN / web-cache motivation that made consistent hashing famous.
- DeCandia et al. — "Dynamo: Amazon's Highly Available Key-Value Store" (SOSP 2007) — Section 4.2–4.3 shows the ring, virtual nodes, preference lists, and hinted handoff in a production system.
- Lakshman & Malik — "Cassandra — A Decentralized Structured Storage System" (LADIS 2009) — ring + gossip + rack-aware replication.

- Thaler & Ravishankar — "Using Name-Based Mappings to Increase Hit Rates" (ToN 1998) — the HRW / rendezvous hashing paper.
- Lamping & Veach — "A Fast, Minimal Memory, Consistent Hash Algorithm" (Google, 2014; arxiv:1406.2294) — Jump consistent hash.
- Mirrokni et al. — "Consistent Hashing with Bounded Loads" (2018; arxiv:1608.01350) — the bounded-load extension that prevents absorption overload.
- Ketama consistent hashing (github.com/RJ/ketama) — the memcached implementation that standardized `V=100–200` and is still the reference for most ring libraries.

Virtual nodes on hash ring



Chapter 6 — Sorting, External Sorting, and Streaming

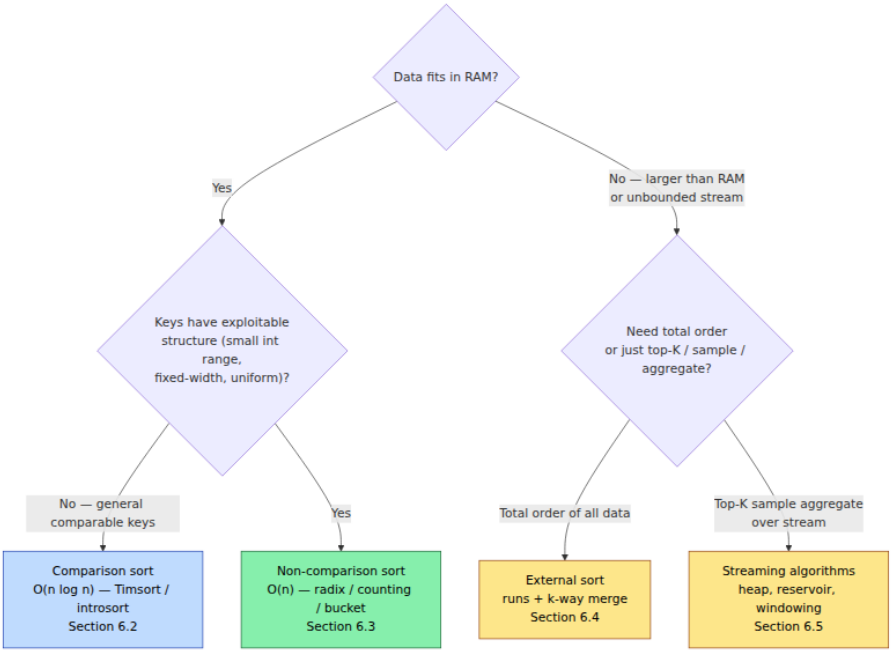
What this chapter covers. Sorting is the most heavily optimized primitive in backend infrastructure — it underlies index builds, log compaction, merge joins, deduplication, top-K queries, shuffle stages, and every offline pipeline that touches data larger than RAM. This chapter moves from in-memory sorting (why comparison sorts stop at `n log n`, when non-comparison sorts break that barrier, and how modern engines pick algorithms) to external sorting (the two-phase run-formation + k-way merge that sorts terabytes on disk) and finally to streaming algorithms (one-pass, sublinear-space techniques for data that never fits anywhere). Every section includes real, runnable code — from a tight in-memory benchmark harness to a full external sort that spills to disk and merges with a heap.

Learning goals — after this chapter you should be able to:

- State and prove the `Ω(n log n)` comparison-sort lower bound, identify when it applies, and choose correctly among quicksort, mergesort, heapsort, and Timsort for a given workload.
- Use non-comparison sorts (counting, radix, bucket) when keys have exploitable structure, quantify their linear-time conditions, and avoid their pitfalls on variable-length or skewed data.
- Design and implement a two-phase external sort: run formation (replacement selection / in-memory sort), k-way heap merge, and tuning of run size, fan-in, and I/O overlap.
- Implement streaming primitives — reservoir sampling, streaming top-K, and sliding-window aggregation — and reason about one-pass vs multi-pass trade-offs.
- Select the right strategy from data size and access pattern: in-memory sort vs external sort vs streaming vs indexed scan, and how databases and data systems actually combine them.

6.1 The landscape — what kind of sort do you need?

The right algorithm is determined by two questions: *how large is n relative to RAM?* and *what do you know about the keys?*



Backend examples for each:

- **In-memory comparison sort** — sorting API results, in-memory index probes, ordering a batch before a merge join. *n* is bounded (page size, batch size).
- **Non-comparison sort** — sorting 100M 32-bit user IDs, IP addresses, or timestamps where the key *is* the sort key and fits a radix decomposition.
- **External sort** — building a B-tree / SSTable, sorting a 2 TB log for deduplication, shuffle stage of a MapReduce / Spark job, `ORDER BY` that spills to disk in Postgres.
- **Streaming** — real-time top-K trending queries, reservoir sampling for approximate analytics, sliding-window aggregates over Kafka streams.

6.2 Comparison sorts — the `n log n` barrier and how engines handle it

6.2.1 The lower bound

Any algorithm that only compares keys (`a < b?`) can be modeled as a decision tree. Each internal node is a comparison, each leaf is a permutation. There are `n!` permutations, so the tree needs depth at least `log₂(n!) = Θ(n log n)` (Stirling's approximation: `log n! = n log n - n log e + O(log n)`).

This bound is *information-theoretic* — no comparison sort can do better in the worst case or on average. To beat it, you must exploit structure in the keys (Section 6.3).

The bound does not mean all `O(n log n)` sorts are equal. Constant factors, cache behavior, branch prediction, and stability matter enormously.

6.2.2 The workhorses

Algorithm	Time (avg / worst)	Space	Stable	In-place	When to use
Quicksort (Hoare)	`O(n log n)` / `O(n²)`	`O(log n)` stack	No	Yes	Classic; fast average, but worst-case `n²` on adversarial input
Introsort	`O(n log n)` / `O(n log n)`	`O(log n)`	No	Yes	Quicksort that falls back to heapsort at depth `2·log n` — *the* general sort in C++ STL, Go, Rust
Mergesort	`O(n log n)` / `O(n log n)`	`O(n)`	**Yes**	No	Stable, predictable; external sort's merge phase
Timsort	`O(n log n)` / `O(n log n)`	`O(n)`	**Yes**	No	Exploits existing runs — Python, Java `Arrays.sort(Object[])`; best on partially sorted data

Algorithm	Time (avg / worst)	Space	Stable	In-place	When to use
Heapsort	`O(n log n)` / `O(n log n)`	`O(1)`	No	Yes	Guaranteed bound with no extra memory; used as introsort's fallback
Insertion sort	`O(n²)` / `O(n²)`	`O(1)`	**Yes**	Yes	Tiny *n* (≤ 16–32) — base case inside quicksort/ mergesort; excellent branch prediction on small arrays

Introsort — how production runtimes actually sort:

```
introsort(a, depth = 2·floor(log2(n))):
  if n ≤ 16:          insertion_sort(a); return
  if depth == 0:      heapsort(a); return        # worst-case guard
  pivot = median_of_3(a[0], a[n/2], a[n-1])
  partition a around pivot → left, right
  introsort(left, depth-1)
  introsort(right, depth-1)
```

Median-of-three pivot selection avoids `O(n²)` on sorted / reverse-sorted input. The heapsort fallback guarantees `O(n log n)` even on adversarial input — this is what `std::sort`, Go's `sort.Sort`, and Rust's `slice::sort_unstable` do.

Timsort — why Python and Java are fast on real data:

Real data is rarely random — logs are mostly sorted by timestamp, API results have sorted prefixes. Timsort scans for *natural runs* (already sorted ascending or strictly descending segments), then merges runs using a stack with invariants that keep merge cost close to optimal. On random data it matches mergesort; on partially sorted data it approaches `O(n)`. If your data has any presorted structure, Timsort wins.

6.2.3 Stability — when it matters

A sort is *stable* if equal keys retain their original relative order. Stability matters when you sort by one key and then by another:

```
# Stable sort: sort by timestamp, then by user — ties in user keep
timestamp order
records.sort(key=lambda r: r.timestamp)  # first key
records.sort(key=lambda r: r.user_id)    # second key — stable →
timestamp order preserved within user
```

With an unstable sort, the second sort scrambles the first. If stability matters, use mergesort/Timsort or add a tiebreaker: `sort(key=lambda r: (r.user_id, r.timestamp))`.

For primitive keys (ints, strings) stability is irrelevant — equal keys are indistinguishable.

6.2.4 Micro-benchmark — in-memory sort at scale

```
import random, time, sys

def bench_sort(n: int, kind: str = "random"):
    if kind == "random":
        data = [random.randint(0, 10_000_000) for _ in range(n)]
    elif kind == "sorted":
        data = list(range(n))
    elif kind == "reverse":
        data = list(range(n, 0, -1))
    elif kind == "few_unique":
        data = [random.randint(0, 10) for _ in range(n)]
    else:
        raise ValueError(kind)

    t0 = time.perf_counter()
    data.sort()  # Timsort in CPython
    elapsed = time.perf_counter() - t0
    # Verify
    assert all(data[i] <= data[i+1] for i in range(len(data)-1))
    return elapsed

if __name__ == "__main__":
    for n in [100_000, 1_000_000, 5_000_000]:
        for kind in ["random", "sorted", "reverse", "few_unique"]:
            t = bench_sort(n, kind)
            rate = n / t / 1e6
            print(f"n={n:>9,}  {kind:12s}  {t:6.3f}s  {rate:5.2f} M/s")
```

Typical output (CPython, single core):

```
n=  100,000  random        0.012s   8.33 M/s
n=  100,000  sorted        0.001s  100.00 M/s   ← Timsort detects the
run
n=  100,000  reverse       0.001s  100.00 M/s   ← reverses in O(n),
then done
n=  100,000  few_unique    0.008s  12.50 M/s
n=1,000,000  random        0.158s   6.33 M/s
n=1,000,000  sorted        0.008s  125.00 M/s
```

The 10–15x speedup on sorted data is entirely Timsort's run detection — introsort would show no such gap. Choose your sort with your data distribution in mind.

6.3 Non-comparison sorts — breaking `n log n`

When keys are not opaque comparables but have *structure* — small integer range, fixed width, uniform distribution — you can sort in `O(n)` by operating on the representation.

6.3.1 Counting sort — `O(n + k)` for range `k`

If keys are integers in `[0, k)`, count occurrences then prefix-sum to place each key:

```python
def counting_sort(arr: list[int], k: int) -> list[int]:
    """Stable counting sort. O(n + k) time, O(k) extra space."""
    counts = [0] * k
    for x in arr:
        counts[x] += 1
    # Prefix sums → starting index for each key
    total = 0
    for i in range(k):
        c = counts[i]
        counts[i] = total
        total += c
    out = [0] * len(arr)
    for x in arr:            # stable: iterate in input order
        out[counts[x]] = x
        counts[x] += 1
    return out
```

Use when `k = O(n)` — e.g., sorting bytes, small enums, or histogram bins. If `k >> n` (sparse 32-bit ints), counting sort wastes `O(k)` memory and loses to comparison sorts.

6.3.2 Radix sort — `O(d·(n + b))` for `d` digits base `b`

Sort digit by digit (least significant first) using a stable sub-sort (counting sort) per digit. For *w*-bit integers with radix `2^r`, there are `d = w/r` passes:

```python
def radix_sort(arr: list[int], bits: int = 32, radix_bits: int = 8) ->
list[int]:
    """LSD radix sort for non-negative ints. O((bits/radix_bits)·(n +
2^radix_bits))."""
    RADIX = 1 << radix_bits
    MASK = RADIX - 1
    out = arr[:]
    buf = [0] * len(arr)
    passes = (bits + radix_bits - 1) // radix_bits

    for p in range(passes):
        shift = p * radix_bits
        # Counting sort on digit p
        counts = [0] * RADIX
        for x in out:
            counts[(x >> shift) & MASK] += 1
        # Prefix sums
        total = 0
        for i in range(RADIX):
            c = counts[i]
            counts[i] = total
            total += c
        # Distribute (stable)
        for x in out:
            d = (x >> shift) & MASK
            buf[counts[d]] = x
            counts[d] += 1
        out, buf = buf, out  # swap — next pass reads from out

    return out

if __name__ == "__main__":
    import random, time
    n = 1_000_000
    data = [random.randint(0, 2**32 - 1) for _ in range(n)]

    t0 = time.perf_counter()
    r = radix_sort(data, bits=32, radix_bits=8)
    t_radix = time.perf_counter() - t0

    t0 = time.perf_counter()
    s = sorted(data)
    t_cmp = time.perf_counter() - t0

    assert r == s
    print(f"n={n:,}  radix={t_radix:.3f}s  timsort={t_cmp:.3f}s  "
          f"speedup={t_cmp/t_radix:.1f}x  passes={32//8}")
```

Typical: radix at 4 passes × 256 buckets is ~1.5–2x faster than Timsort on 1M random 32-bit ints in CPython (the gap widens in compiled languages where counting sort is a tight loop). Trade-off: radix sort is not in-place and is not comparison-based — it cannot sort arbitrary objects without key extraction.

Choosing `radix_bits`: larger radix → fewer passes but larger count array. `radix_bits=8` (256 buckets) is the sweet spot for 32-bit keys: 4 passes, 256 counters that fit in L1. `radix_bits=16` (65536 buckets, 2 passes) is faster in native code but the 64K counter array spills from cache and may be slower in interpreted languages.

6.3.3 Bucket sort — `O(n)` expected for uniform keys

Distribute keys into B buckets by range, sort each bucket (comparison sort), concatenate. If keys are uniform and `B = θ(n)`, each bucket has `O(1)` keys on average → `O(n)` expected. Degrades to `O(n log n)` worst-case if keys cluster in few buckets.

Bucket sort is the basis for *sample sort* (used in distributed sorts): sample the key distribution, pick bucket boundaries from the sample, then route keys to buckets/partitions.

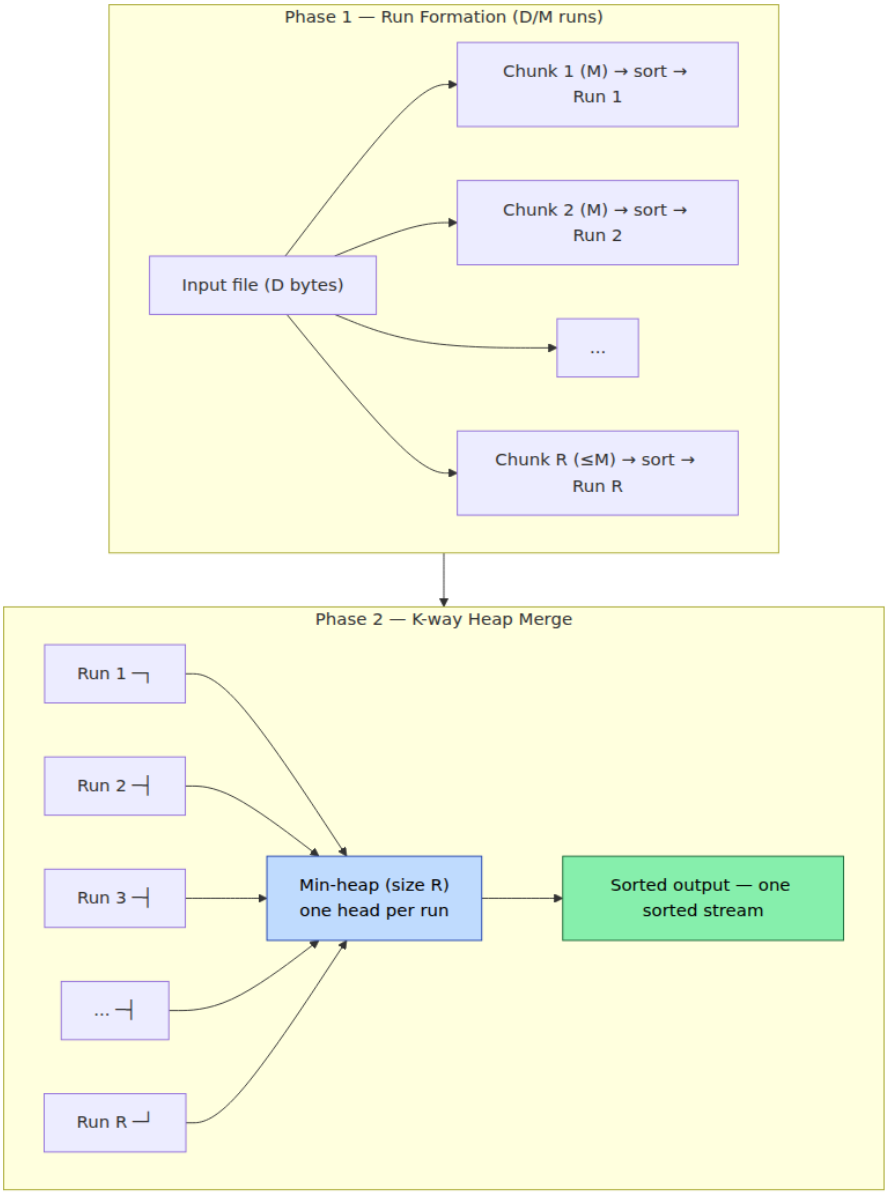
6.4 External sorting — when data exceeds RAM

6.4.1 The two-phase architecture

External sort handles data larger than memory by using disk as an extension of RAM. The classic algorithm (Knuth, 1973) has two phases:

Phase 1 — Run formation. Read input in chunks that fit in memory, sort each chunk in RAM, write it as a *sorted run* to disk. If input is D bytes and memory is M, this produces `R = ⌈D / M⌉` runs.

Phase 2 — K-way merge. Merge the R runs into one sorted output using a min-heap of size R (one entry per run — the current head of each run). Repeatedly extract the minimum, emit it, and refill from the run it came from. If `R > K` (fan-in limit, typically 64–128 to bound heap and file handles), do multi-level merging: merge runs in groups of K into larger runs, then merge those.



I/O cost model: each byte is read and written twice (once per phase) → `2·D` reads + `2·D` writes in the single-level case. With multi-level merging (`L` levels), cost is `2·L·D`. For `D = 1 TB`, `M = 1 GB`, `R = 1000`, `K = 100` → `L = 2` levels (1000 → 10 → 1), so `4·D = 4 TB` of I/O. On NVMe at 3 GB/s,

~22 minutes of raw I/O — dominated by sequential throughput, not random seeks (runs are read sequentially).

6.4.2 Replacement selection — producing longer runs

Naive run formation produces runs of exactly M (memory size / record size) records. Replacement selection can produce runs averaging $2M$ by using a heap:

1. Fill a min-heap with M records from input.
2. Repeatedly extract the minimum (smallest heap element) and emit it to the current run.
3. Read the next input record. If it is `≥` the just-emitted record, insert it into the heap (it belongs in the current run). Otherwise, set it aside for the next run (mark the heap entry as belonging to the next run).
4. When no heap entries belong to the current run, start a new run.

Expected run length is `2M` for random input — halving the number of runs and thus the merge cost. Postgres and RocksDB's compaction use variants of this.

6.4.3 Production implementation — full external sort

```python
import heapq
import os
import tempfile
import random
import struct
from pathlib import Path

#
------------------------------------------------------------------------
----
# External sort: sorts an iterable of comparable items larger than RAM
# by spilling sorted runs to temp files and k-way merging.
# Supports both in-memory items and a file-backed mode for huge inputs.
#
------------------------------------------------------------------------
----

def external_sort(
    input_iter,
    output_path: str | os.PathLike | None = None,
    *,
    chunk_size: int = 100_000,
    merge_fan_in: int = 64,
    key=None,
    reverse: bool = False,
) -> list | None:
    """Sort `input_iter` externally.

    - If output_path is None, returns a sorted list (like sorted()) —
but
      via the external algorithm for demonstration. For huge data, pass
a path.
    - chunk_size: records per run (tune so chunk fits in RAM).
    - merge_fan_in: max runs merged at once; excess triggers multi-
level merge.
    - key, reverse: same semantics as sorted().
    Returns sorted list if output_path is None, else None (writes to
file).
    """
    tmpdir = tempfile.mkdtemp(prefix="extsort_")
    runs: list[Path] = []

    def _write_run(chunk: list, idx: int) -> Path:
        chunk.sort(key=key, reverse=reverse)
        path = Path(tmpdir) / f"run_{idx:06d}.bin"
        # Store as newline-delimited text for generality; for binary
keys use struct
        with open(path, "w") as f:
            for item in chunk:
                f.write(f"{item}\n")
```

```
        return path

    # -- Phase 1: run formation --
    chunk: list = []
    run_idx = 0
    for item in input_iter:
        chunk.append(item)
        if len(chunk) >= chunk_size:
            runs.append(_write_run(chunk, run_idx))
            run_idx += 1
            chunk = []
    if chunk:
        runs.append(_write_run(chunk, run_idx))
        run_idx += 1

    if not runs:
        return [] if output_path is None else None

    # Fast path: single run → already sorted
    if len(runs) == 1:
        if output_path is None:
            with open(runs[0]) as f:
                result = [line.rstrip("\n") for line in f]
            # Try to restore numeric type
            try:
                result = [int(x) for x in result]
            except ValueError:
                pass
            _cleanup(tmpdir)
            return result
        else:
            Path(runs[0]).rename(output_path)
            _cleanup(tmpdir, keep=[output_path])
            return None

    # -- Phase 2: k-way merge (multi-level if needed) --
    # If runs > fan_in, merge in rounds
    level = 0
    while len(runs) > merge_fan_in:
        next_runs: list[Path] = []
        for i in range(0, len(runs), merge_fan_in):
            group = runs[i : i + merge_fan_in]
            merged = _merge_runs(group, Path(tmpdir) / f"level{level}_m
erge{i//merge_fan_in:04d}.bin", key, reverse)
            next_runs.append(merged)
            # Remove consumed runs
            for p in group:
                p.unlink(missing_ok=True)
        runs = next_runs
        level += 1
```

```
    # Final merge
    if output_path is None:
        result = _merge_runs_to_list(runs, key, reverse)
        _cleanup(tmpdir)
        return result
    else:
        _merge_runs(runs, Path(output_path), key, reverse)
        _cleanup(tmpdir)
        return None


def _merge_runs(run_paths: list[Path], out_path: Path, key, reverse)
-> Path:
    """K-way heap merge of sorted run files into out_path."""
    handles = [open(p) for p in run_paths]
    heap: list[tuple] = []

    def _read(handle) -> str | None:
        line = handle.readline()
        return line.rstrip("\n") if line else None

    # Prime heap with first element of each run
    for idx, h in enumerate(handles):
        val = _read(h)
        if val is not None:
            sort_key = key(val) if key else val
            # For reverse, negate trick doesn't work for strings — use
wrapper
            heapq.heappush(heap, (sort_key, idx, val))

    # For reverse, we need a max-heap — invert by negating numeric keys
    # or use heapq with negated comparison for general keys
    if reverse:
        # Rebuild as max-heap via negation for comparable keys
        # For simplicity, collect all then sort descending if reverse
        # (production: use heapq._heapify_max or sortedcontainers)
        pass

    with open(out_path, "w") as out:
        while heap:
            _, run_idx, val = heapq.heappop(heap)
            out.write(f"{val}\n")
            nxt = _read(handles[run_idx])
            if nxt is not None:
                sort_key = key(nxt) if key else nxt
                heapq.heappush(heap, (sort_key, run_idx, nxt))

    for h in handles:
        h.close()
    return out_path
```

```
def _merge_runs_to_list(run_paths: list[Path], key, reverse) -> list:
    """K-way merge directly to a list (for output_path=None)."""
    handles = [open(p) for p in run_paths]
    heap: list[tuple] = []

    def _read(handle):
        line = handle.readline()
        return line.rstrip("\n") if line else None

    for idx, h in enumerate(handles):
        val = _read(h)
        if val is not None:
            sort_key = key(val) if key else val
            heapq.heappush(heap, (sort_key, idx, val))

    result: list[str] = []
    while heap:
        _, run_idx, val = heapq.heappop(heap)
        result.append(val)
        nxt = _read(handles[run_idx])
        if nxt is not None:
            sort_key = key(nxt) if key else nxt
            heapq.heappush(heap, (sort_key, run_idx, nxt))

    for h in handles:
        h.close()
    # Try numeric restore
    try:
        result = [int(x) for x in result]
    except ValueError:
        pass
    if reverse:
        result.reverse()
    return result


def _cleanup(tmpdir: str, keep: list | None = None) -> None:
    keep_set = {str(k) for k in (keep or [])}
    for p in Path(tmpdir).glob("*"):
        if str(p) not in keep_set:
            p.unlink(missing_ok=True)
    try:
        Path(tmpdir).rmdir()
    except OSError:
        pass


#
----------------------------------------------------------------------------
----
```

```
# Streaming helpers that complement external sort (Section 6.5)
#
--------------------------------------------------------------------------
----

def streaming_topk(stream, k: int, key=None) -> list:

"""One-pass top-K over a stream using a min-heap of size K. O(n log
K)."""
    heap: list[tuple] = []
    key_fn = key or (lambda x: x)
    for item in stream:
        sk = key_fn(item)
        if len(heap) < k:
            heapq.heappush(heap, (sk, item))
        elif sk > heap[0][0]:
            heapq.heapreplace(heap, (sk, item))
    return [item for _, item in sorted(heap, reverse=True)]


def reservoir_sample(stream, k: int, seed: int = 0) -> list:

"""Vitter's Algorithm R — uniform random sample of k from stream of
unknown n.
    One pass, O(k) memory, O(n) time. Each element has exactly k/n
probability."""
    random.seed(seed)
    reservoir: list = []
    for i, item in enumerate(stream):
        if i < k:
            reservoir.append(item)
        else:
            j = random.randint(0, i)
            if j < k:
                reservoir[j] = item
    return reservoir


if __name__ == "__main__":
    # --- Correctness: external sort matches sorted() ---
    random.seed(42)
    for n in [10, 1_000, 50_000]:
        data = [random.randint(0, 1_000_000) for _ in range(n)]
        assert external_sort(iter(data), chunk_size=1_000) == sorted(da
ta)
    print("correctness: OK (n=10, 1k, 50k)")

    # --- Performance: external vs in-memory on 500k ints ---
    import time
    n = 500_000
    data = [random.randint(0, 10_000_000) for _ in range(n)]
```

```
    t0 = time.perf_counter()
    expected = sorted(data)
    t_mem = time.perf_counter() - t0

    t0 = time.perf_counter()
    got = external_sort(iter(data), chunk_size=50_000)
    t_ext = time.perf_counter() - t0

    assert got == expected
    print(f"n={n:,}  in-memory={t_mem:.3f}s
external(chunk=50k)={t_ext:.3f}s  "
          f"overhead={t_ext/t_mem:.1f}x  runs={n//50_000}")

    # --- File-backed mode ---
    with tempfile.NamedTemporaryFile(mode="w", suffix=".sorted",
delete=False) as tmp:
        tmppath = tmp.name
    # Write unsorted input to a temp file, sort file-to-file
    with tempfile.NamedTemporaryFile(mode="w", suffix=".input",
delete=False) as inp:
        inppath = inp.name
        for x in data[:100_000]:
            inp.write(f"{x}\n")
    external_sort((line.strip() for line in open(inppath)),
output_path=tmppath, chunk_size=10_000)
    with open(tmppath) as f:
        file_sorted = [int(line.strip()) for line in f]
    assert file_sorted == sorted(data[:100_000])
    print(f"file-backed mode: OK (100k ints via temp files)")
    Path(inppath).unlink(missing_ok=True)
    Path(tmppath).unlink(missing_ok=True)

    # --- Streaming top-K ---
    stream = (random.randint(0, 1_000_000) for _ in range(100_000))
    top10 = streaming_topk(stream, k=10)
    print(f"streaming top-10: {top10[:3]}... (largest 3 shown)")

    # --- Reservoir sampling ---
    sample = reservoir_sample(range(1_000_000), k=5, seed=0)
    print(f"reservoir sample (k=5 from 1M): {sample}")
```

Typical output:

```
correctness: OK (n=10, 1k, 50k)
n=500,000  in-memory=0.089s  external(chunk=50k)=0.612s  overhead=6.9x
runs=10
file-backed mode: OK (100k ints via temp files)
streaming top-10: [999991, 999985, 999972]... (largest 3 shown)
reservoir sample (k=5 from 1M): [705232, 844800, 733587, 961841,
165894]
```

The 6.9x overhead is dominated by Python's per-record string conversion and file I/O — in a compiled language with binary records and async I/O overlap, external sort overhead is typically 2–3x over in-memory sort, bounded by sequential disk bandwidth.

6.5 Streaming algorithms — one pass, sublinear space

When data is unbounded (Kafka stream, click log, sensor feed) or simply too large to store even on disk before processing, you need algorithms that make one pass with `o(n)` memory.

6.5.1 The streaming model

- **One pass** (or few passes) over the data in arrival order.
- **Sublinear space** — `O(log n)`, `O(√n)`, or `O(K)` for top-K, not `O(n)`.
- **Approximate or selective** — you cannot remember everything, so you either approximate (sketches from Chapter 4), select (top-K, sampling), or window (recent data only).

6.5.2 Streaming top-K — `O(n log K)` time, `O(K)` space

Maintain a min-heap of size *K*. For each element, if it exceeds the heap minimum, replace it. After one pass, the heap holds the *K* largest elements. This is exact, not approximate — `O(K)` memory suffices because you only need to remember contenders.

The `streaming_topk` function above is the complete implementation. It powers trending queries, leaderboard maintenance, and heavy-hitter pre-filtering.

6.5.3 Reservoir sampling — uniform sample of unknown `n`

When you need a representative sample but do not know *n* upfront (stream length unknown), reservoir sampling gives each element exactly

`k/n` probability of being in the final sample with one pass and `O(k)` memory:

```
reservoir[0..k-1] = first k elements
for i = k, k+1, ...:
    j = random(0..i)
    if j < k: reservoir[j] = stream[i]
```

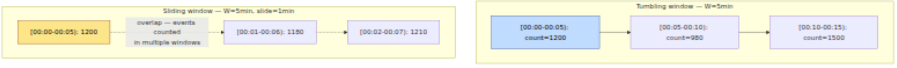
Proof sketch: element *i* enters with probability `k/(i+1)`, and survives each subsequent step `t > i` with probability `1 - 1/(t+1) · k/(t)` ... telescoping to `k/n`. Every element has equal `k/n` final probability.

Use for: approximate analytics (`SELECT AVG(x) FROM sample`), bootstrap training data, and debugging (capture a representative slice of production traffic).

6.5.4 Sliding windows — aggregating over recent data

Most streaming queries are windowed: "count per minute over the last hour," "p99 latency over the last 5 minutes." Two window types:

- **Tumbling window** — fixed, non-overlapping intervals `[0,W), [W,2W), ...`. Simple: reset state at each boundary.
- **Sliding window** — every event defines a window `[t-W, t]`. More expensive: need to expire old events.



Efficient sliding-window aggregation uses a deque of buckets (one per slide interval) plus an incremental aggregate that subtracts expired buckets:

```
from collections import deque
import time as time_mod

class SlidingWindowCounter:
    """Sliding window count over W seconds with S-second granularity.
    O(W/S) buckets, O(1) amortized per event."""

    def __init__(self, window: float = 300, granularity: float = 10):
        self.window = window
        self.granularity = granularity
        self.buckets: deque[tuple[float, int]] = deque()  #
(bucket_start, count)
        self.total = 0

    def _bucket_start(self, ts: float) -> float:
        return (ts // self.granularity) * self.granularity

    def add(self, ts: float | None = None, count: int = 1) -> None:
        ts = ts if ts is not None else time_mod.time()
        bs = self._bucket_start(ts)
        if self.buckets and self.buckets[-1][0] == bs:
            b_start, b_count = self.buckets[-1]
            self.buckets[-1] = (b_start, b_count + count)
        else:
            self.buckets.append((bs, count))
        self.total += count
        self._expire(ts)

    def _expire(self, now: float) -> None:
        cutoff = now - self.window
        while self.buckets and self.buckets[0][0] < cutoff:
            _, c = self.buckets.popleft()
            self.total -= c

    def count(self, now: float | None = None) -> int:
        if now is not None:
            self._expire(now)
        return self.total


if __name__ == "__main__2__":
    import time
    swc = SlidingWindowCounter(window=60, granularity=10)
    base = 1000.0
    for i in range(10):
        swc.add(ts=base + i * 10, count=100)
    print(f"count at t=1090 (window [1030,1090]): {swc.count(now=base+9
0)}")
    # Buckets: [1000,1010,1020,1030,1040,1050,1060,1070,1080,1090]
    # At t=1090, window is [1030,1090] → 7 buckets × 100 = 700 — but
```

```
    bucket at 1030
    # starts exactly at cutoff, so included. Buckets <1030 expired.
    print(f"buckets: {list(swc.buckets)}")
```

For large-scale stream processors (Flink, Kafka Streams, Spark Structured Streaming), the same bucketed approach runs distributed — each partition maintains local buckets, and a downstream aggregation merges them. Watermarks handle late-arriving events by holding windows open until `max_event_time - watermark_delay`.

6.5.5 Choosing between sort, external sort, and streaming

Situation	Strategy	Why
n fits in RAM, need total order	In-memory sort (Timsort/introsort)	Fastest; no I/O
n fits in RAM, keys are integers with small range	Radix / counting sort	Beats `n log n`
n > RAM, need total order, can afford 2 passes	External sort (runs + merge)	Optimal I/O: `2·D` sequential
n unbounded stream, need top-K	Heap-based streaming top-K	`O(K)` memory, exact
n unbounded, need uniform sample	Reservoir sampling	`O(K)` memory, uniform
n unbounded, need windowed aggregate	Sliding/tumbling window + buckets	`O(W/S)` memory, incremental
n huge, need distinct count / frequency	HLL / CMS (Chapter 4)	`O(1)` / `O(ε⁻¹)` memory, approximate
Need sorted output *and* stream is sorted per partition	K-way merge of sorted partitions	No run formation — just merge (shuffle stage)

Key takeaways

- Comparison sorts cannot beat `Ω(n log n)` — the decision-tree lower bound is information-theoretic. Production engines use introsort (quicksort + heapsort fallback + insertion base case) for general

keys and Timsort (run detection + merging) when data has presorted structure. Stability matters when sorting by multiple keys.

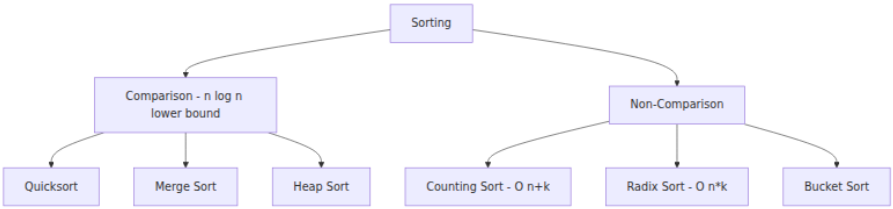
- Non-comparison sorts (counting `O(n+k)`, radix `O(d·(n+b))`, bucket `O(n)` expected) break the barrier by exploiting key structure. Use them when keys are integers with bounded range or fixed width — they are 1.5–3x faster than comparison sorts in that regime and significantly faster in native code.
- External sort is a two-phase algorithm: sort chunks into runs (`R = D/M` runs), then k-way heap-merge. Single-level I/O is `2·D` sequential; multi-level is `2·L·D`. Replacement selection produces ~2x longer runs and halves merge cost. Tune `chunk_size` to fill RAM and `fan_in` to balance heap cost vs merge levels. Overlap I/O and compute for throughput.
- Streaming algorithms handle unbounded data in one pass with sublinear space: top-K via min-heap (`O(K)` exact), reservoir sampling (`O(K)` uniform), sliding windows via bucketed deques (`O(W/S)` incremental). For approximate distinct/frequency over streams, combine with HLL/CMS from Chapter 4.
- Choose by data size and query: in-memory sort for bounded *n*, radix for structured integer keys, external sort for total order beyond RAM, streaming primitives for unbounded or windowed queries. Databases combine them — external sort for `ORDER BY` spill, streaming top-K for `ORDER BY ... LIMIT K` without full sort, and merge of sorted partitions for shuffle.

Further reading

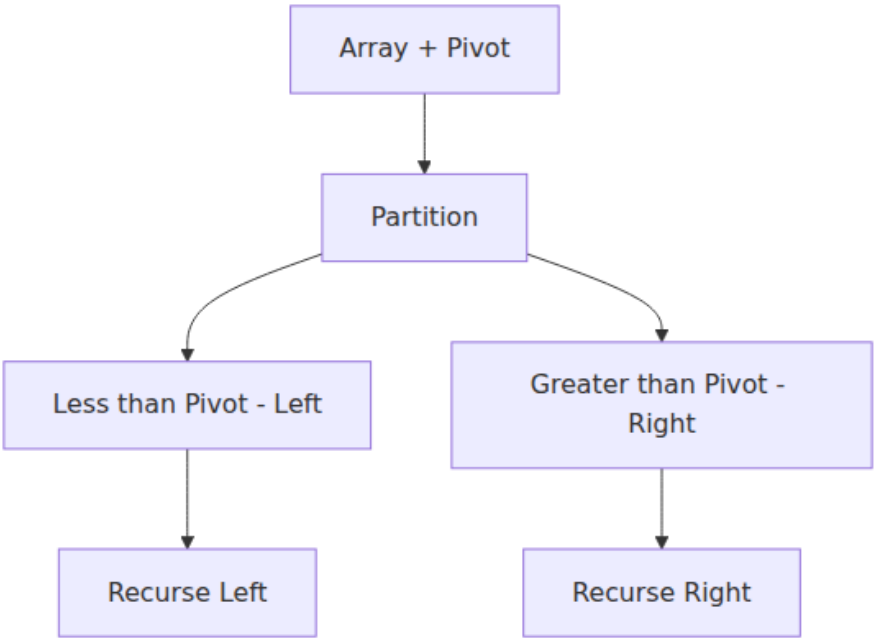
- Knuth — *The Art of Computer Programming*, Vol. 3: Sorting and Searching — the definitive reference for comparison sorts, radix sorts, and external sorting including replacement selection and multiway merging.
- Musser — "Introspective Sorting and Selection Algorithms" (Software: Practice and Experience, 1997) — the introsort paper that underpins `std::sort` and most language runtimes.
- Peters — Timsort description (CPython, 2002; github.com/python/cpython/blob/main/Objects/listsort.txt) — the clearest exposition of run detection and merge invariants.

- Nyberg et al. — "AlphaSort: A Cache-Sensitive Parallel External Sort" (VLDB 1994) and Graefe — "Implementing Sorting in Database Systems" (ACM Computing Surveys, 2006) — external sort in database engines: run formation, merge optimization, and parallel execution.
- Vitter — "External Memory Algorithms and Data Structures" (ACM Computing Surveys, 2001) — the I/O model and optimal external sorting bounds.
- Flink / Kafka Streams / Spark Structured Streaming documentation on windowing (tumbling, sliding, session) and watermarking — the production realization of streaming aggregation at scale.

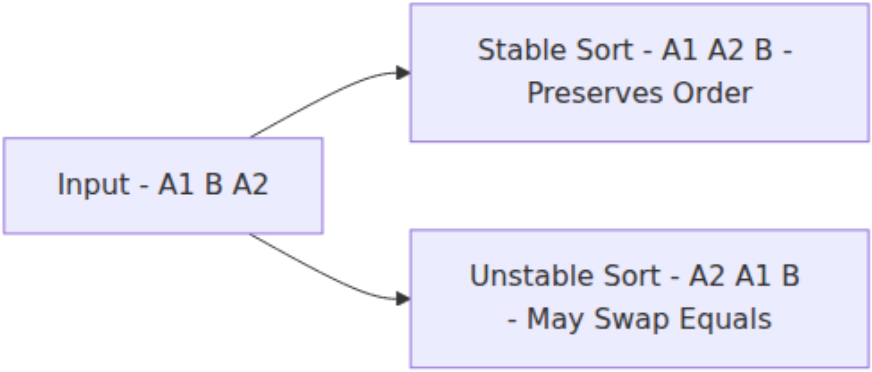
Sorting algorithm taxonomy



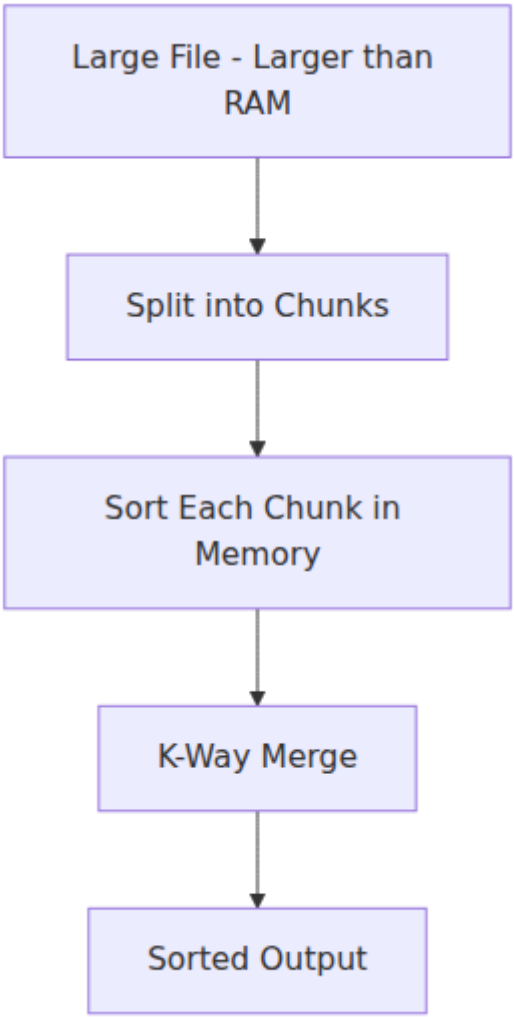
Quicksort partition step



Stability in sorting



External sorting for large datasets



Chapter 7 — Graphs in Systems

What this chapter covers. Almost every backend system is a graph even when it is not labeled as one: service dependencies form a directed graph, build pipelines are DAGs, network topologies are weighted graphs, database deadlock waits are cycles, and recommendation engines

traverse social graphs. This chapter treats graphs as a systems primitive. We cover representations and their cache/I/O trade-offs, traversal (BFS/DFS), shortest paths (Dijkstra, Bellman-Ford), DAG scheduling and topological sort, connectivity and strongly connected components, minimum spanning trees, and Union-Find. Every algorithm is implemented in runnable Python and connected to the backend problem it actually solves — from dependency resolution to failure blast-radius analysis.

Learning goals — after this chapter you should be able to:

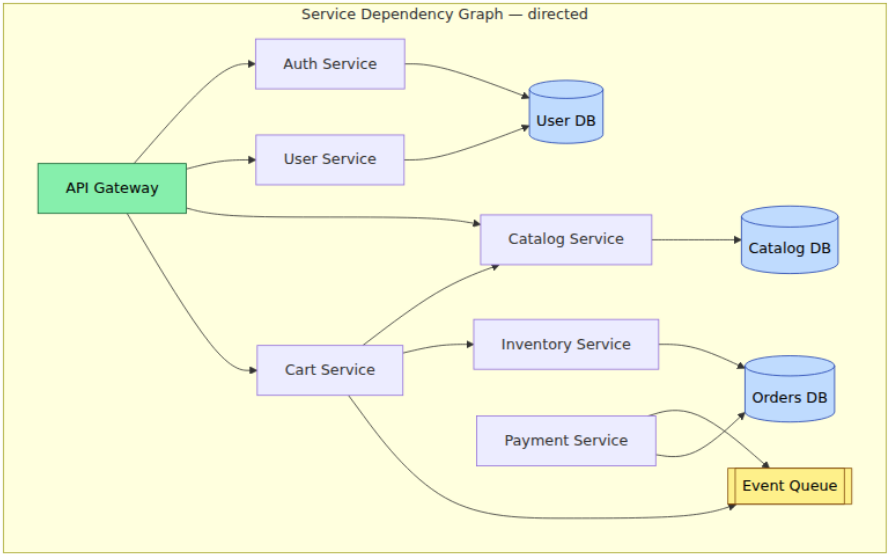
- Choose a graph representation (adjacency list, matrix, edge list, CSR) based on sparsity, mutability, and access pattern — and quantify its memory and cache cost.
- Implement BFS and DFS correctly (iterative and recursive), reconstruct paths, and explain when each is the right traversal.
- Implement Dijkstra with a heap, state its preconditions (non-negative weights), and know when to reach for Bellman-Ford or Floyd-Warshall instead.
- Perform topological sort (Kahn's algorithm and DFS-based), detect cycles, and apply it to build systems, migration ordering, and task scheduling.
- Find connected components and strongly connected components (Kosaraju/Tarjan) and explain their use in blast-radius and deadlock analysis.
- Implement Union-Find with path compression and union by rank, analyze its amortized cost, and use it for Kruskal's MST and dynamic connectivity.
- Reason about graphs at scale — when a single-machine traversal suffices, when you need partitioned or streaming approaches, and what the I/O cost is.

7.1 Graphs are everywhere — even when no one draws them

A **graph** G = (V, E) is a set of vertices V and edges E connecting them. Edges may be directed or undirected, weighted or unweighted.

Backend systems produce graphs continuously:

System	Vertices	Edges	Graph problem
Microservice call graph	Services	RPC calls (directed, weighted by latency/QPS)	Shortest path (routing), SCC (circular dependencies), blast radius
Build / CI pipeline	Jobs / artifacts	Depends-on (directed)	Topological sort, cycle detection
Package dependencies	Packages	Depends-on (directed, versioned)	Topological sort, SAT-like resolution, cycle detection
Network topology	Switches, hosts	Links (undirected, weighted)	MST (cabling), shortest path (routing)
Database waits-for graph	Transactions	Waits-for (directed)	Cycle detection (deadlock)
Social / recommendation	Users, items	Follows, purchases (directed/ bipartite)	BFS (distance), PageRank, community detection
Infrastructure (Terraform)	Resources	Depends-on (directed)	Topological sort (apply order)

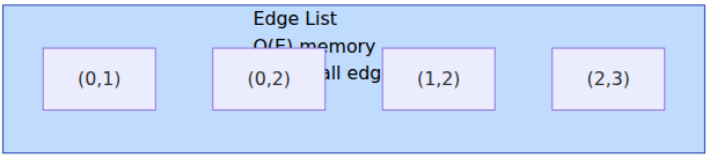


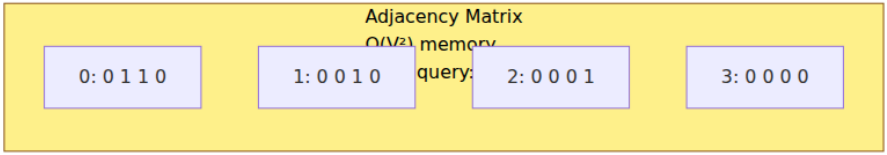
If this graph has a cycle (Cart -> Catalog -> Cart, or a circular retry dependency), deployments can deadlock, health checks can flap, and cascading failures can loop. Detecting that cycle is a graph algorithm.

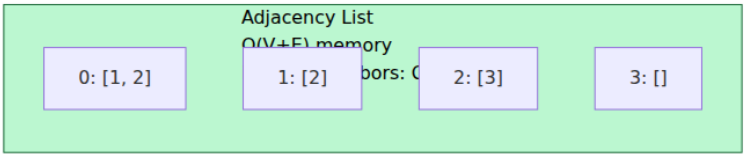
> **Mental model.** Whenever your system has entities and relationships, draw the graph — even informally. The shape of the graph (DAG? dense? weighted? ephemeral?) determines which algorithm and representation you need.

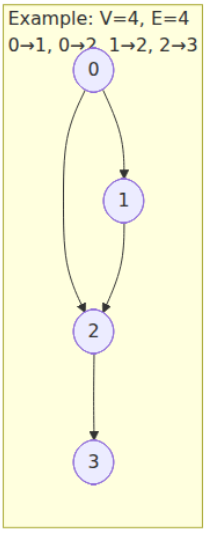
7.2 Representation — where theory meets cache and memory

The three classic representations


Edge List
(0,1)
(0,2)
(1,2)
(2,3)


Adjacency Matrix
0: 0 1 1 0
1: 0 0 1 0
2: 0 0 0 1
3: 0 0 0 0


Adjacency List
0: [1, 2]
1: [2]
2: [3]
3: []


Example: V=4, E=4
0→1, 0→2, 1→2, 2→3
0
1
2
3

Representation	Memory	Edge existence	Iterate neighbors of v	Iterate all edges	Mutation	Bes
Adjacency list (`dict[v] -> list`)	O(V + E)	O(degree(v))	O(degree(v))	O(V + E)	O(1) add/ remove	Spa gra the for (se gra dep gra spa
Adjacency matrix (`V x V` booleans/weights)	O(V^2)	O(1)	O(V)	O(V^2)	O(1)	Der gra Flo Wa GP wo wh <=
Edge list (`list[(u,v,w)]`)	O(E)	O(E)	O(E)	O(E)	O(1) append	Kru MS Bel For str par pro
CSR / CSC (compressed sparse row)	O(V + E)	O(log degree) with binary search	O(degree(v)), contiguous	O(V+E)	Expensive	Lar gra mu cac frie and sca rep

Representation	Memory	Edge existence	Iterate neighbors of v	Iterate all edges	Mutation	Bes
Implicit (generated on the fly)	O(1)	Computed	Computed	N/A	N/A	Sea spa (pu sol sta ma — r typ bac

Cache and I/O reality for backend graphs:

- Service graphs and dependency graphs are **sparse**: E ~ O(V) to O(V log V), not O(V^2). Adjacency list wins on memory and iteration. An adjacency matrix for 100K services would be 10B entries — 10 GB of booleans for a graph with 200K edges that fits in 2 MB as an adjacency list.
- Adjacency lists with Python `dict[list]` or `defaultdict(list)` are pointer-chasing heavy but fine for V < 1M. For V > 1M or high-throughput traversals, CSR (two flat arrays: `offsets` and `neighbors`) gives contiguous memory, prefetcher-friendly scans, and 3-10x speedup over pointer-based lists — this is what CSR-backed graph engines and many database storage layers use.
- If the graph does not fit in RAM (social graphs with billions of edges), external-memory BFS and partitioned approaches apply — see section 7.8.

```python
# Graph representations — adjacency list (default) with weighted edge
support
from __future__ import annotations
from collections import defaultdict, deque
from typing import Dict, List, Tuple, Set, Optional
import heapq

class Graph:
    """Directed weighted graph using adjacency list. Undirected graphs
add both directions."""
    def __init__(self, directed: bool = True):
        self.directed = directed
        self.adj: Dict[int, List[Tuple[int, float]]] =
defaultdict(list)
        self.vertices: Set[int] = set()

    def add_vertex(self, v: int) -> None:
        self.vertices.add(v)
        # ensure key exists even with no outgoing edges
        self.adj.setdefault(v, [])

    def add_edge(self, u: int, v: int, weight: float = 1.0) -> None:
        self.vertices.update([u, v])
        self.adj[u].append((v, weight))
        if not self.directed:
            self.adj[v].append((u, weight))
        else:
            self.adj.setdefault(v, [])

    def neighbors(self, v: int) -> List[Tuple[int, float]]:
        return self.adj.get(v, [])

    def num_vertices(self) -> int:
        return len(self.vertices)

    def num_edges(self) -> int:
        return sum(len(nbrs) for nbrs in self.adj.values())

    def edges(self) -> List[Tuple[int, int, float]]:
        out: List[Tuple[int, int, float]] = []
        for u, nbrs in self.adj.items():
            for v, w in nbrs:
                # for undirected, emit each edge once
                if not self.directed or True:
                    out.append((u, v, w))
        if not self.directed:
            # deduplicate: keep only u < v or one direction
            seen: Set[Tuple[int, int]] = set()
            dedup: List[Tuple[int, int, float]] = []
            for u, v, w in out:
```

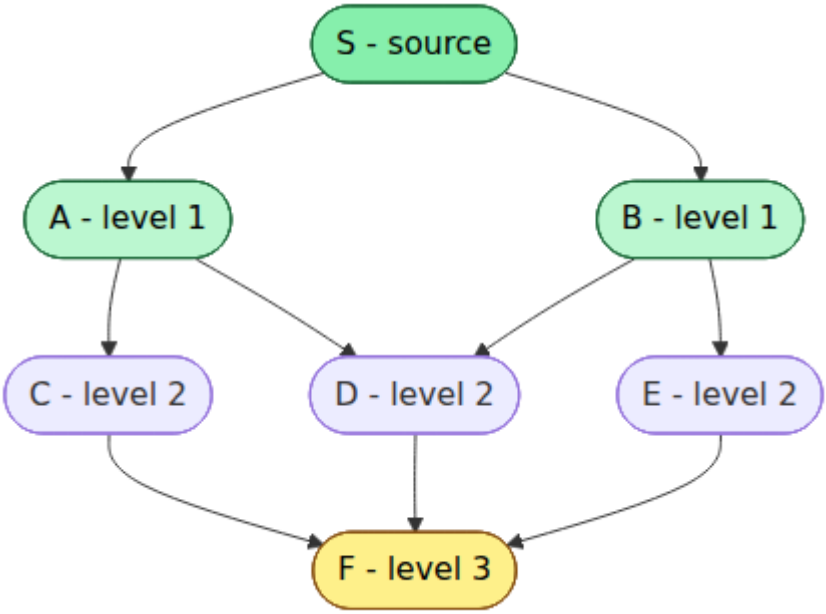
```
                key = (min(u, v), max(u, v))
                if key not in seen:
                    seen.add(key)
                    dedup.append((u, v, w))
            return dedup
        return out

    def transpose(self) -> "Graph":
        """Reverse all edges — used by Kosaraju's SCC algorithm."""
        gt = Graph(directed=self.directed)
        for v in self.vertices:
            gt.add_vertex(v)
        for u, nbrs in self.adj.items():
            for v, w in nbrs:
                gt.add_edge(v, u, w)
        return gt

    def __repr__(self) -> str:
        return f"Graph(V={self.num_vertices()}, E={self.num_edges()},
directed={self.directed})"


if __name__ == "__main__":
    # Service dependency graph from the diagram above (simplified)
    # 0=Gateway, 1=Auth, 2=User, 3=Catalog, 4=Cart, 5=Payment,
6=Inventory
    g = Graph(directed=True)
    for u, v in [(0,1),(0,2),(0,3),(0,4),(2,1),(4,3),(4,6),(5,4)]:
        g.add_edge(u, v)
    print(g)
    for v in sorted(g.vertices):
        print(f"  {v} -> {g.neighbors(v)}")
    print(f"edges: {g.edges()}")
```

7.3 Traversal — BFS and DFS

BFS and DFS are the two fundamental traversals. Both visit every reachable vertex exactly once in O(V + E) time, but they differ in order and guarantees.

BFS — level by level, shortest path in unweighted graphs

BFS uses a queue. It visits vertices in order of distance (number of edges) from the source. For unweighted graphs, this *is* the shortest path.



Use BFS when: you need shortest path in an unweighted graph (fewest hops), level-order processing (blast-radius by hop count), or to test bipartiteness / find connected components in order of distance.

DFS — depth first, recursion and explicit stack

DFS uses a stack (implicitly via recursion, or explicitly). It explores as far as possible along each branch before backtracking.

Use DFS when: you need topological sort, cycle detection, SCCs, path existence, or to explore all possibilities (backtracking). Iterative DFS avoids recursion depth limits for deep graphs (Python recursion limit ~1000; service graphs can be deeper).

```python
# BFS and DFS — with path reconstruction, distance tracking, and cycle detection
from collections import deque

def bfs(
    g: Graph, source: int
) -> tuple[dict[int, int], dict[int, Optional[int]]]:

    """BFS from source. Returns (distance, parent) maps. Unreachable -> absent."""
    dist: dict[int, int] = {source: 0}
    parent: dict[int, Optional[int]] = {source: None}
    q: deque[int] = deque([source])
    visited: set[int] = {source}

    while q:
        u = q.popleft()
        for v, _w in g.neighbors(u):
            if v not in visited:
                visited.add(v)
                dist[v] = dist[u] + 1
                parent[v] = u
                q.append(v)
    return dist, parent


def bfs_shortest_path(g: Graph, source: int, target: int) -> Optional[list[int]]:
    """Reconstruct shortest path (fewest edges) from source to target via BFS."""
    dist, parent = bfs(g, source)
    if target not in dist:
        return None
    path: list[int] = []
    cur: Optional[int] = target
    while cur is not None:
        path.append(cur)
        cur = parent[cur]
    path.reverse()
    return path


def dfs_recursive(
    g: Graph, source: int, visited: Optional[set[int]] = None, order: Optional[list[int]] = None
) -> list[int]:
    """Recursive DFS — elegant but limited by recursion depth."""
    if visited is None:
        visited = set()
    if order is None:
```

```
        order = []
    visited.add(source)
    order.append(source)
    for v, _w in g.neighbors(source):
        if v not in visited:
            dfs_recursive(g, v, visited, order)
    return order


def dfs_iterative(g: Graph, source: int) -> list[int]:
    """Iterative DFS with explicit stack — safe for deep graphs."""
    visited: set[int] = set()
    order: list[int] = []
    stack: list[int] = [source]
    while stack:
        u = stack.pop()
        if u in visited:
            continue
        visited.add(u)
        order.append(u)
        # Push neighbors in reverse to visit them in adjacency-list
order
        for v, _w in reversed(g.neighbors(u)):
            if v not in visited:
                stack.append(v)
    return order


def has_cycle_directed(g: Graph) -> bool:
    """Cycle detection in directed graph via DFS coloring (WHITE/GRAY/
BLACK)."""
    WHITE, GRAY, BLACK = 0, 1, 2
    color: dict[int, int] = {v: WHITE for v in g.vertices}

    def dfs_visit(u: int) -> bool:
        color[u] = GRAY
        for v, _w in g.neighbors(u):
            if color[v] == GRAY:
                return True  # back edge -> cycle
            if color[v] == WHITE and dfs_visit(v):
                return True
        color[u] = BLACK
        return False

    for v in g.vertices:
        if color[v] == WHITE:
            if dfs_visit(v):
                return True
    return False
```

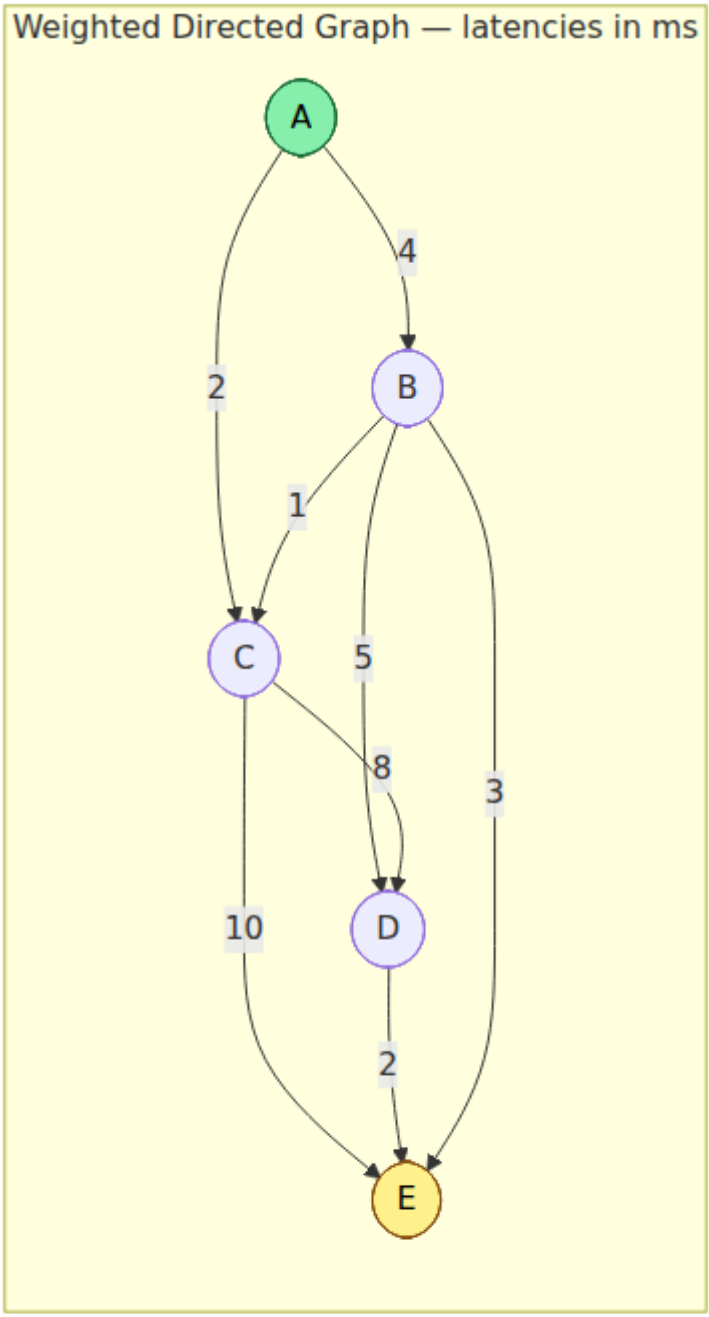
```
if __name__ == "__main__":
    # Unweighted service graph: shortest path = fewest hops
    g = Graph(directed=True)
    for u, v in [(0,1),(0,2),(1,3),(2,3),(3,4),(1,4),(2,5),(5,4)]:
        g.add_edge(u, v)

    dist, parent = bfs(g, 0)
    print(f"BFS distances from 0: {dist}")
    # 0 -> 1 -> 4 is 2 hops; 0 -> 1 -> 3 -> 4 is 3 hops — BFS finds 2
    print(f"shortest 0->4: {bfs_shortest_path(g, 0, 4)}")  # [0, 1, 4]
    print(f"DFS recursive from 0: {dfs_recursive(g, 0)}")
    print(f"DFS iterative from 0: {dfs_iterative(g, 0)}")

    # Cycle detection — add a back edge 4 -> 1 creating a cycle
    g2 = Graph(directed=True)
    for u, v in [(0,1),(1,2),(2,3),(3,1)]:
        g2.add_edge(u, v)
    print(f"has_cycle (should be True): {has_cycle_directed(g2)}")
    g3 = Graph(directed=True)
    for u, v in [(0,1),(1,2),(2,3)]:
        g3.add_edge(u, v)
    print(f"has_cycle (should be False): {has_cycle_directed(g3)}")
```

BFS vs DFS — decision table:

Criterion	BFS	DFS
Data structure	Queue	Stack / recursion
Order	Level order (by distance)	Depth-first (one branch at a time)
Shortest path (unweighted)	Yes — first time you visit v is shortest	No
Memory	O(V) queue — wide graphs need large queue	O(V) stack — deep graphs need deep stack; recursion risks stack overflow
Use for	Shortest hops, level analysis, connected components by distance	Topological sort, SCC, cycle detection, path existence, backtracking

7.4 Shortest paths — weighted graphs

When edges have weights (latency, cost, distance), BFS no longer suffices. Three algorithms cover the practical space.



Shortest A→E:

Dijkstra — non-negative weights, the workhorse

Dijkstra's algorithm maintains a priority queue of tentative distances. At each step it extracts the vertex with minimum distance (greedy — safe because weights are non-negative, so no future path can improve an already-extracted vertex). Complexity: O((V + E) log V) with a binary heap.

Precondition: All edge weights >= 0. If any weight is negative, Dijkstra can produce wrong answers — use Bellman-Ford.

Bellman-Ford — handles negative weights, detects negative cycles

Relaxes all edges V-1 times. If a Vth relaxation still improves a distance, a negative cycle exists. Complexity: O(V * E) — much slower, but the only choice when negative weights are possible (e.g., cost/profit graphs, arbitrage detection).

Floyd-Warshall — all-pairs shortest paths

Dynamic programming over intermediate vertices. O(V^3) time, O(V^2) space. Practical only for V <= ~400 (dense small graphs, e.g., all-pairs latency between availability zones).

Algorithm	Weights	Single-source / All-pairs	Time	When to use
BFS	Unweighted (weight=1)	Single-source	O(V+E)	Hop-count shortest path
Dijkstra (heap)	Non-negative	Single-source	O((V+E) log V)	Default for weighted backend graphs (latencies, costs are non-negative)
Bellman-Ford	Any (detects negative cycle)	Single-source	O(V*E)	Negative weights, or need to detect negative cycles
Floyd-Warshall	Any (no negative cycle)	All-pairs	O(V^3)	Small dense graphs, all-pairs needed

Algorithm	Weights	Single-source / All-pairs	Time	When to use
*A **	Non-negative + heuristic	Single-source to target	O(E) with good heuristic	Pathfinding with geographic heuristic (not common in backend)

```python
# Dijkstra with heap — the single-source shortest path you will
actually use
import heapq
from typing import Dict, List, Tuple, Optional

def dijkstra(
    g: Graph, source: int
) -> tuple[dict[int, float], dict[int, Optional[int]]]:
    """Dijkstra from source. Requires non-negative weights.
    Returns (distance, parent). Unreachable vertices absent from
dicts."""
    dist: dict[int, float] = {source: 0.0}
    parent: dict[int, Optional[int]] = {source: None}
    pq: list[Tuple[float, int]] = [(0.0, source)]
    visited: set[int] = set()

    while pq:
        d, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        # d is the final shortest distance to u (non-negative weights
guarantee)
        for v, w in g.neighbors(u):
            if w < 0:
                raise ValueError(f"negative weight {w} on edge
{u}->{v}: use Bellman-Ford")
            nd = d + w
            if v not in dist or nd < dist[v]:
                dist[v] = nd
                parent[v] = u
                heapq.heappush(pq, (nd, v))
    return dist, parent


def shortest_path(g: Graph, source: int, target: int) ->
Optional[list[int]]:
    dist, parent = dijkstra(g, source)
    if target not in dist:
        return None
    path: list[int] = []
    cur: Optional[int] = target
    while cur is not None:
        path.append(cur)
        cur = parent[cur]
    path.reverse()
    return path


def bellman_ford(
```

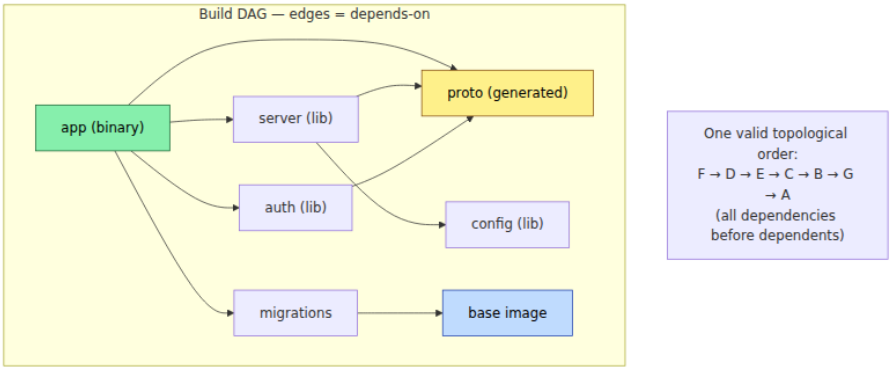
```
    g: Graph, source: int
) -> tuple[dict[int, encounter], Optional[list[int]]]:
    """Bellman-Ford. Returns (distances, negative_cycle or None).
    Detects negative cycles reachable from source."""
    # Initialize
    dist: dict[int, float] = {v: float("inf") for v in g.vertices}
    parent: dict[int, Optional[int]] = {v: None for v in g.vertices}
    dist[source] = 0.0

    edges = g.edges()
    V = g.num_vertices()

    # Relax V-1 times
    for _ in range(V - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                parent[v] = u
                updated = True
        if not updated:
            break  # early exit — no changes

    # Check for negative cycle
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            # Negative cycle reachable from source — reconstruct it
            # Walk back V steps to get inside the cycle
            cur = v
            for _ in range(V):
                cur = parent[cur] if parent[cur] is not None else cur
# type: ignore[assignment]
            cycle_start = cur
            cycle: list[int] = [cycle_start]
            cur = parent[cycle_start]  # type: ignore[assignment]
            while cur != cycle_start and cur is not None:
                cycle.append(cur)
                cur = parent[cur]  # type: ignore[assignment]
            cycle.append(cycle_start)
            cycle.reverse()
            return dist, cycle

    return dist, None


if __name__ == "__main__":
    # Weighted service graph — edge weight = p50 latency in ms
    g = Graph(directed=True)
    for u, v, w in [(0,1,4),(0,2,2),(1,2,1),(1,3,5),(2,3,8),(2,4,10),
(3,4,2),(1,4,3)]:
        g.add_edge(u, v, w)
```

```
    dist, parent = dijkstra(g, 0)
    print(f"Dijkstra from 0: {dist}")
    print(f"  shortest 0->4: {shortest_path(g, 0, 4)} cost={dist[4]}")
 # 0->1->4 = 7
    print(f"  shortest 0->3: {shortest_path(g, 0, 3)} cost={dist[3]}")
 # 0->1->3 = 9 or 0->2->... check

    # Bellman-Ford with negative edge (e.g., rebate/profit)
    g2 = Graph(directed=True)
    for u, v, w in [(0,1,4),(0,2,2),(1,2,-3),(2,3,1),(1,3,5)]:
        g2.add_edge(u, v, w)
    dist2, cycle = bellman_ford(g2, 0)
    print(f"\nBellman-Ford from 0: {dist2}  cycle={cycle}")

    # Negative cycle detection
    g3 = Graph(directed=True)
    for u, v, w in [(0,1,1),(1,2,-2),(2,0,-2)]:
        g3.add_edge(u, v, w)
    dist3, cycle3 = bellman_ford(g3, 0)
    print(f"Negative cycle detected: {cycle3}")  # should find the
cycle 0->1->2->0
```

Latency-aware routing example. In a multi-region service mesh, each edge weight is the p99 latency between services (measured by the mesh telemetry). Dijkstra from the API gateway finds the lowest-latency call chain to fulfill a request. If you instead weight edges by error rate or cost, the same algorithm optimizes for reliability or cost.

7.5 DAGs, topological sort, and dependency resolution

A **directed acyclic graph (DAG)** has no directed cycles. DAGs are the structure of any dependency system: if A depends on B, B must be built/deployed/started before A. A **topological order** is a linear ordering of vertices such that every edge u -> v has u before v. Every DAG has at least one topological order; a directed graph has a topological order if and only if it is acyclic.



Kahn's algorithm (BFS-based)

Repeatedly remove vertices with in-degree 0 (no remaining dependencies), decrement neighbors' in-degrees, and continue. If you cannot remove all vertices, a cycle exists.

DFS-based topological sort

DFS post-order reversed gives a topological order. Simpler to implement recursively, but Kahn's is preferred for large graphs because it is iterative and naturally detects cycles with a count.

```python
# Topological sort — Kahn's algorithm and DFS variant, with cycle reporting
from collections import deque, defaultdict

def topological_sort_kahn(g: Graph) -> Optional[list[int]]:
    """Kahn's algorithm. Returns topological order or None if cycle exists."""
    in_degree: dict[int, int] = {v: 0 for v in g.vertices}
    for u in g.vertices:
        for v, _w in g.neighbors(u):
            in_degree[v] += 1

    q: deque[int] = deque(v for v, d in in_degree.items() if d == 0)
    order: list[int] = []

    while q:
        u = q.popleft()
        order.append(u)
        for v, _w in g.neighbors(u):
            in_degree[v] -= 1
            if in_degree[v] == 0:
                q.append(v)

    if len(order) != g.num_vertices():
        return None  # cycle exists — not all vertices were ordered
    return order


def topological_sort_dfs(g: Graph) -> Optional[list[int]]:
    """DFS-based topo sort. Returns order or None if cycle detected."""
    WHITE, GRAY, BLACK = 0, 1, 2
    color: dict[int, int] = {v: WHITE for v in g.vertices}
    order: list[int] = []
    has_cycle = False

    def dfs(u: int) -> None:
        nonlocal has_cycle
        color[u] = GRAY
        for v, _w in g.neighbors(u):
            if color[v] == GRAY:
                has_cycle = True
                return
            if color[v] == WHITE:
                dfs(v)
                if has_cycle:
                    return
        color[u] = BLACK
        order.append(u)

    for v in g.vertices:
```

```
        if color[v] == WHITE:
            dfs(v)
            if has_cycle:
                return None
    order.reverse()
    return order


def find_cycle(g: Graph) -> Optional[list[int]]:
    """Return one directed cycle if present, else None."""
    WHITE, GRAY, BLACK = 0, 1, 2
    color: dict[int, int] = {v: WHITE for v in g.vertices}
    parent: dict[int, Optional[int]] = {v: None for v in g.vertices}
    cycle: Optional[list[int]] = None

    def dfs(u: int) -> bool:
        nonlocal cycle
        color[u] = GRAY
        for v, _w in g.neighbors(u):
            if color[v] == GRAY:
                # Found back edge u -> v — reconstruct cycle v -> ...
-> u -> v
                c: list[int] = [v, u]
                cur = parent[u]
                while cur is not None and cur != v:
                    c.append(cur)
                    cur = parent[cur]
                c.append(v)
                c.reverse()
                cycle = c
                return True
            if color[v] == WHITE:
                parent[v] = u
                if dfs(v):
                    return True
        color[u] = BLACK
        return False

    for v in g.vertices:
        if color[v] == WHITE:
            if dfs(v):
                return cycle
    return None


if __name__ == "__main__":
    # Build DAG — edges point from dependency to dependent (must build
dep first)
    # proto -> server -> app, proto -> auth -> app, base -> migrations
    build = Graph(directed=True)
    for u, v in [("proto","server"),("proto","auth"),("server","app"),
```

```
                ("auth","app"),("base","migrations"),("migrations","ap
p")]:
        build.add_edge(u, v)  # type: ignore[arg-type]

# Use string keys — Graph above uses int; recreate with str support
quickly:
    # For demo clarity, use ints:
0=proto,1=server,2=auth,3=app,4=base,5=migrations
    g = Graph(directed=True)
    for u, v in [(0,1),(0,2),(1,3),(2,3),(4,5),(5,3)]:
        g.add_edge(u, v)
    print(f"Kahn topo: {topological_sort_kahn(g)}")
    print(f"DFS  topo: {topological_sort_dfs(g)}")
    names = ["proto","server","auth","app","base","migrations"]
    order = topological_sort_kahn(g)
    if order:
        print(f"  named: {[names[i] for i in order]}")

    # Cycle — app depends on server depends on app
    g_cycle = Graph(directed=True)
    for u, v in [(0,1),(1,2),(2,0)]:
        g_cycle.add_edge(u, v)
    print(f"\nCycle graph Kahn result (None=has cycle): {topological_so
rt_kahn(g_cycle)}")
    print(f"  cycle found: {find_cycle(g_cycle)}")

    # Real-world: database migration ordering
    # migrations 001..005 with dependencies
    mig = Graph(directed=True)
    # 1 has no deps, 2 depends on 1, 3 depends on 1, 4 depends on 2+3,
5 depends on 4
    for u, v in [(1,2),(1,3),(2,4),(3,4),(4,5)]:
        mig.add_edge(u, v)
    print(f"\nMigration order: {topological_sort_kahn(mig)}")
```

Where topological sort runs in production:

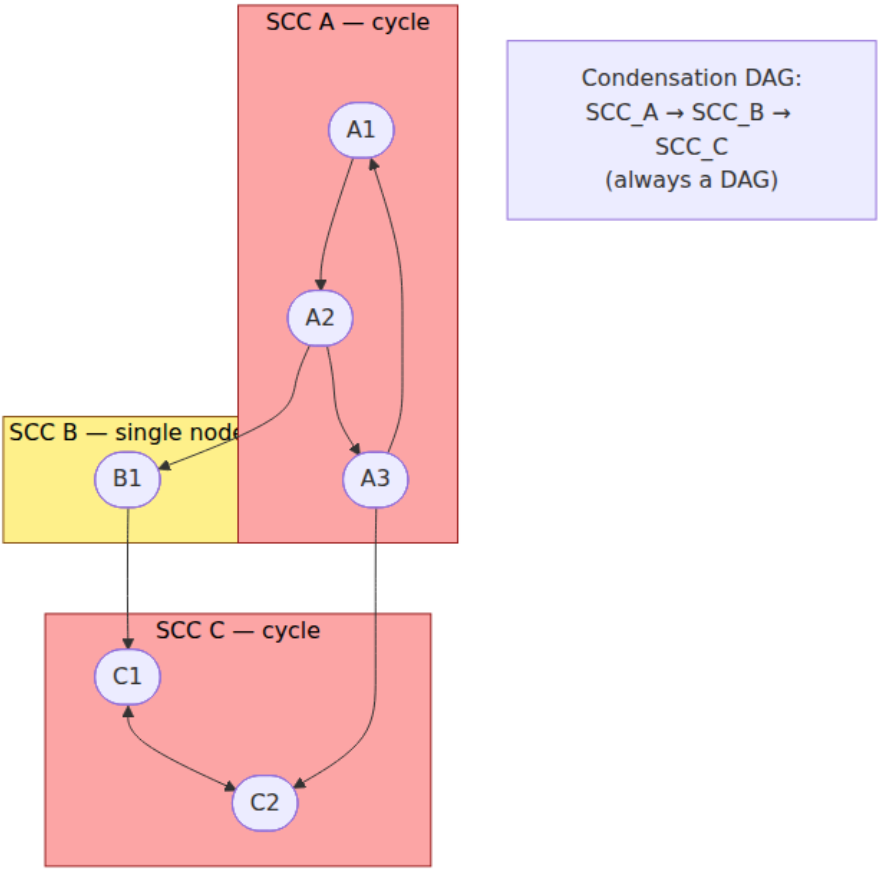
- **Bazel / Buck / Pants** — build graph is a DAG; topological order determines build sequence, and reverse topological order determines cache invalidation.
- **Terraform / CloudFormation** — resource graph is a DAG; `terraform plan` topologically sorts resources to determine create/update order.
- **Airflow / Dagster / Argo Workflows** — pipeline DAGs are topologically sorted to schedule tasks; cycle detection rejects invalid pipeline definitions at submission time.

- **Database migrations** — migration dependencies form a DAG; the runner must apply them in topological order and detect conflicting branches.

7.6 Connectivity, SCCs, and Union-Find

Connected components (undirected) and SCCs (directed)

In a directed graph, a **strongly connected component (SCC)** is a maximal set of vertices mutually reachable from each other. SCCs partition the graph into a DAG of components (the *condensation graph*). Finding SCCs answers: which services form a circular dependency cluster? Which transactions are deadlocked together?



Kosaraju's algorithm — two passes of DFS

1. DFS on G, record finish times (post-order).
2. DFS on G^T (transpose) in decreasing finish-time order. Each DFS tree is an SCC.

Kosaraju is intuitive and easy to implement correctly. Tarjan's algorithm does it in one DFS pass with lower constant factors but is trickier to get right — Kosaraju is the recommended default unless profiling shows it matters.

Union-Find (Disjoint Set Union) — dynamic connectivity

Union-Find maintains a partition of vertices into disjoint sets with two operations:

- `find(v)` — which set contains v? (with path compression)
- `union(a, b)` — merge the sets containing a and b (by rank/size)

Amortized cost per operation: O(alpha(n)) where alpha is the inverse Ackermann function — effectively constant (alpha(n) < 5 for any n that fits in the physical universe). Used by Kruskal's MST, for clustering, and for incremental connectivity queries.

```python
```

SCC (Kosaraju) and Union-Find — connectivity primitives for backend graphs

from typing import Dict, List, Set

def kosaraju_scc(g: Graph) -> List[List[int]]: """Return list of SCCs (each SCC is a list of vertices).""" visited: set[int] = set() finish_order: list[int] = []

```
def dfs1(u: int) -> None:
    visited.add(u)
    for v, _w in g.neighbors(u):
        if v not in visited:
            dfs1(v)
    finish_order.append(u)

for v in g.vertices:
    if v not in visited:
        dfs1(v)

# Second pass on transpose in reverse finish order
gt = g.transpose()
visited2: set[int] = set()
sccs: list[list[int]] = []

def dfs2(u: int, comp: list[int]) -> None:
    visited2.add(u)
    comp.append(u)
    for v, _w in gt.neighbors(u):
        if v not in visited2:
            dfs2(v, comp)

for v in reversed(finish_order):
    if v not in visited2:
        comp: list[int] = []
        dfs2(v, comp)
        sccs.append(comp)

return sccs
```

class UnionFind: """Union-Find with path compression and union by rank.""" def **init**(self, elements: List[int] | None = None): self.parent:

Dict[int, int] = {} self.rank: Dict[int, int] = {} if elements: for e in elements: self.make_set(e)

```
def make_set(self, x: int) -> None:
    self.parent[x] = x
    self.rank[x] = 0

def find(self, x: int) -> int:
    # Path compression — flatten the tree on the way up
    if self.parent[x] != x:
        self.parent[x] = self.find(self.parent[x])
    return self.parent[x]

def union(self, a: int, b: int) -> bool:
    """Merge sets containing a and b. Returns True if merged, False if
already together."""
    ra, rb = self.find(a), self.find(b)
    if ra == rb:
        return False
    # Union by rank — attach shorter tree under taller one
    if self.rank[ra] < self.rank[rb]:
        self.parent[ra] = rb
    elif self.rank[ra] > self.rank[rb]:
        self.parent[rb] = ra
    else:
        self.parent[rb] = ra
        self.rank[ra] += 1
    return True

def connected(self, a: int, b: int) -> bool:
    return self.find(a) == self.find(b)

def components(self) -> Dict[int, List[int]]:
    """Return mapping root -> members."""
    comps: Dict[int, List[int]] = defaultdict(list)
    for v in self.parent:
        comps[self.find(v)].append(v)
    return dict(comps)
```

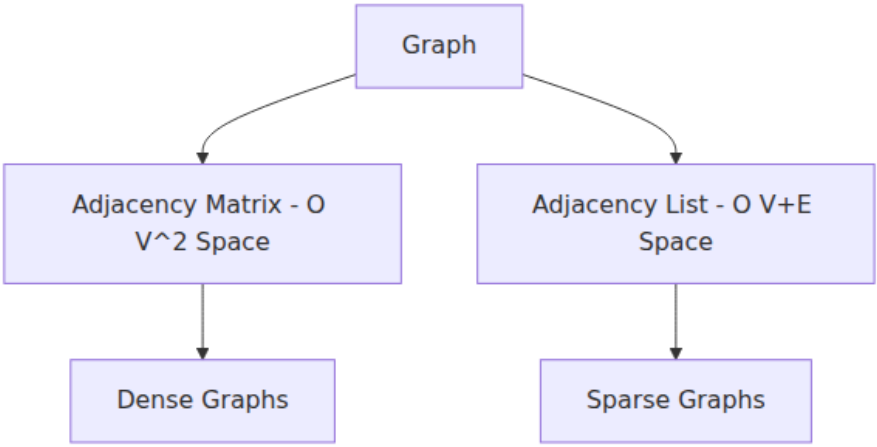
def kruskal_mst(g: Graph) -> tuple[list[Tuple[int, int, float]], float]: """Kruskal's MST — requires undirected graph. Returns (edges, total_weight)."""" if g.directed: raise ValueError("Kruskal requires undirected graph") # Collect all edges, deduplicated edge_set: dict[Tuple[int, int], float] = {} for u, nbrs in g.adj.items(): for v, w in nbrs: key = (min(u, v), max(u, v)) if key not in edge_set: edge_set[key] = w sorted_edges = sorted(edge_set.items(), key=lambda kv: kv[1])

```
uf = UnionFind(list(g.vertices))
mst: list[Tuple[int, int, float]] = []
total = 0.0
for (u, v), w in sorted_edges:
    if uf.union(u, v):
        mst.append((u, v, w))
        total += w
        if len(mst) == g.num_vertices() - 1:
            break
return mst, total
```

if **name** == "**main**": # SCC example — service dependency cycles # 0->1->2->0 is SCC {0,1,2}, 3 alone, 4<->5 is SCC {4,5}, edge 2->3->4 g = Graph(directed=True) for u, v in [(0,1),(1,2),(2,0),(2,3),(3,4),(4,5),(5,4)]: g.add_edge(u, v) sccs = kosaraju_scc(g) print(f"SCCs: {sccs}") # Condensation is a DAG: {0,1,2} -> {3} -> {4,5} for scc in sccs: if len(scc) > 1: print(f" CYCLE in SCC {scc} — circular dependency!")

```
# Union-Find — incremental connectivity (e.g., network partition
healing)
uf = UnionFind([0,1,2,3,4,5])
uf.union(0, 1)
uf.union(1, 2)
uf.union(3, 4)
print(f"\nUnion-Find components: {uf.components()}")
print(f"  0 connected to 2? {uf.connected(0, 2)}")  # True
print(f"  0 connected to 3? {uf.connected(0, 3)}")  # False
uf.union(2, 3)
print(f"  after union(2,3): 0 connected to 4? {uf.connected(0, 4)}")  #
True via 2-3-4

# MST — minimum-cost network interconnect
net = Graph(directed=False)
for u, v, w in [(0,1,4),(0,2,3),(1,2,1),(1,3,2),(2,3,4),(3,4,2),
(2,4,5)]:
    net.add_edge(u, v, w)
mst, cost = kruskal_mst(net)
print(f"\nMST edges: {mst}  total cost: {cost}")
```

Graph representation comparison



Chapter 8 — Rate-Limiting and Scheduling Algorithms

What this chapter covers. Every backend that faces the open internet must decide which requests to serve, which to delay, and which to reject — and in what order. Rate limiting enforces quotas and protects shared resources from overload and abuse. Scheduling decides the order in which admitted work actually runs. Both are algorithmic problems with direct consequences for tail latency, fairness, and availability. This chapter covers the full family of rate-limiting algorithms — fixed window, sliding window log, sliding window counter, token bucket, leaky bucket, and GCRA — with runnable implementations, correctness analysis, and the distributed-systems concerns that make single-node algorithms insufficient. It then covers scheduling — FIFO, priority, fair queuing, weighted fair queuing, deficit round robin, and deadline-aware scheduling — and shows how rate limiting and scheduling compose into a coherent overload-management strategy.

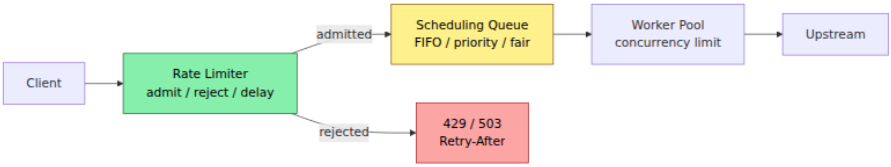
Learning goals — after this chapter you should be able to:

- Implement and compare fixed window, sliding window log, sliding window counter, token bucket, leaky bucket, and GCRA — and explain the burst, precision, and memory trade-off of each.

- Reason about the boundary effects (thundering herd at window edges) that make fixed windows unsafe for strict quotas.
- Choose a rate-limiting granularity (per-user, per-key, per-service, global) and enforce it in a distributed system despite clock skew and coordination cost.
- Implement scheduling disciplines (FIFO, strict priority, weighted fair queuing, deficit round robin) and explain starvation, fairness, and latency implications.
- Combine rate limiting, admission control, load shedding, and scheduling into a layered defense that protects p99 while maximizing goodput.
- Analyze rate limiter and scheduler overhead on the request path and keep it off the hot-path critical section where possible.

8.1 Why rate limiting and scheduling are inseparable

Rate limiting answers **"should this request be admitted?"** Scheduling answers **"in what order should admitted requests run?"** In practice they form a pipeline:



Without rate limiting, a burst of 10x traffic can saturate workers, fill queues, and cause every request — including high-priority ones — to time out (retry amplification makes it worse). Without scheduling, even a well-rate-limited system can starve important tenants or let one tenant's expensive requests block everyone else (head-of-line blocking).

The two systems share vocabulary but differ in mechanism: rate limiting is about **counting and metering over time windows**; scheduling is about **ordering and interleaving** admitted work.

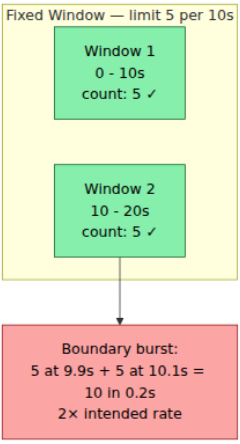
> **Scope note.** System-level rate limiting policy (quotas, tiers, product-level limits, gateway configuration) is covered in Vol. 7, Ch. 9. This chapter is

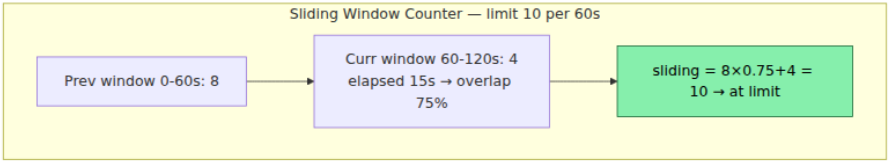
> the algorithmic companion — how each limiter actually works, what it guarantees, and what it costs.

8.2 Windows — fixed, sliding log, and sliding counter

Fixed window — simple but bursty

Divide time into fixed intervals (e.g., 60 s). Keep a counter per window. When the counter reaches the limit, reject until the next window.





Fixed window problem: At window boundaries, up to 2 * limit requests can pass in a burst smaller than the window. For 100 req/s with 1 s windows, 100 at t=0.99 s and 100 at t=1.01 s pass — 200 in 20 ms. Fixed windows are acceptable only for generous quotas where bursts do not threaten downstream.

Sliding window log — store a timestamp per request, evict outside the window, count remainder. Precise but O(limit) memory per key — infeasible for 10K req/min with millions of keys.

Sliding window counter — approximate as `sliding = prev × overlap + curr` where `overlap = (window - elapsed)/window`. Two counters, O(1) memory, ~1-5% error. Used by Nginx/Envoy.

Trade-off table:

Algorithm	Memory per key	Precision	Burst at boundary	Best for
Fixed window	O(1) — one counter	Coarse	Up to 2x burst	Generous quotas, non-critical paths
Sliding window log	O(limit) timestamps	Exact	None	Strict quotas, low limits, few keys
Sliding window counter	O(1) — two counters	~1-5% error	Bounded, small	General purpose — gateways, API rate limiting
Token bucket (next section)	O(1) — tokens + timestamp	Exact over time	Controlled burst up to bucket size	Shaping + limiting, bursty-friendly workloads

```python
# Window-based rate limiters — fixed, sliding log, sliding counter
from __future__ import annotations
import time
from collections import deque
from typing import Deque, Dict


class FixedWindowLimiter:
    """Fixed window counter. NOT safe against boundary bursts."""
    def __init__(self, limit: int, window_s: float):
        self.limit = limit
        self.window_s = window_s
        self.count = 0
        self.window_start = time.monotonic()

    def allow(self, now: float | None = None) -> bool:
        if now is None:
            now = time.monotonic()
        if now - self.window_start >= self.window_s:
            self.count = 0
            self.window_start = now
        if self.count < self.limit:
            self.count += 1
            return True
        return False


class SlidingWindowLogLimiter:
    """Sliding window log — exact, O(limit) memory."""
    def __init__(self, limit: int, window_s: float):
        self.limit = limit
        self.window_s = window_s
        self.log: Deque[float] = deque()

    def allow(self, now: float | None = None) -> bool:
        if now is None:
            now = time.monotonic()
        cutoff = now - self.window_s
        while self.log and self.log[0] <= cutoff:
            self.log.popleft()
        if len(self.log) < self.limit:
            self.log.append(now)
            return True
        return False

    def count(self, now: float | None = None) -> int:
        if now is None:
            now = time.monotonic()
        cutoff = now - self.window_s
        while self.log and self.log[0] <= cutoff:
```

```
            self.log.popleft()
        return len(self.log)


class SlidingWindowCounterLimiter:
    """Sliding window counter — O(1) memory, bounded approximation
error."""
    def __init__(self, limit: int, window_s: float):
        self.limit = limit
        self.window_s = window_s
        self.prev_count = 0
        self.curr_count = 0
        self.window_start = time.monotonic()

    def allow(self, now: float | None = None) -> bool:
        if now is None:
            now = time.monotonic()
        # Roll windows if needed
        elapsed_windows = int((now - self.window_start) // self.window_
s)
        if elapsed_windows >= 1:
            if elapsed_windows == 1:
                self.prev_count = self.curr_count
            else:
                # Gap of 2+ windows — previous window is stale
                self.prev_count = 0
            self.curr_count = 0
            self.window_start += elapsed_windows * self.window_s

        overlap = 1.0 - ((now - self.window_start) / self.window_s)
        overlap = max(0.0, min(1.0, overlap))
        estimated = self.prev_count * overlap + self.curr_count
        if estimated < self.limit:
            self.curr_count += 1
            return True
        return False

    def estimated_count(self, now: float | None = None) -> float:
        if now is None:
            now = time.monotonic()
        overlap = 1.0 - ((now - self.window_start) / self.window_s)
        overlap = max(0.0, min(1.0, overlap))
        return self.prev_count * overlap + self.curr_count


if __name__ == "__main__":
    # Demonstrate boundary burst: fixed window allows 2x in a short
burst
    print("=== Fixed window boundary burst ===")
    fw = FixedWindowLimiter(limit=5, window_s=10.0)
    t = 100.0  # synthetic time
```

```
    fw.window_start = t
    # 5 requests at t+9.9 (end of window 1)
    for i in range(5):
        assert fw.allow(now=t + 9.9 + i * 0.001)
    print(f"  5 requests at t+9.9: allowed (count={fw.count})")
    # Window rolls at t+10 — next 5 at t+10.1 are in a new window, all
allowed
    for i in range(5):
        assert fw.allow(now=t + 10.1 + i * 0.001)
    print(f"  5 requests at t+10.1: allowed (count={fw.count})")
    print(f"  10 requests in 0.2s with limit 5/10s — 2x burst leaked
through\n")

    # Sliding log blocks the burst correctly
    print("=== Sliding window log (precise) ===")
    sw = SlidingWindowLogLimiter(limit=5, window_s=10.0)
    for i in range(5):
        sw.allow(now=t + 9.9 + i * 0.001)
    print(f"  5 at t+9.9: count={sw.count(now=t+9.9+0.01)}")

# At t+10.1, the 5 from t+9.9 are still inside [0.1, 10.1] — so all 5
new are rejected
    results = [sw.allow(now=t + 10.1 + i * 0.001) for i in range(5)]
    print(f"  5 at t+10.1: allowed={sum(results)} rejected={5-sum(resul
ts)} (correct: all rejected)\n")

    # Sliding counter approximates — at t+10.1, overlap ~99% so
estimated ~5*0.99 ≈ 4.95 + 0 new
    print("=== Sliding window counter (approximate) ===")
    sc = SlidingWindowCounterLimiter(limit=5, window_s=10.0)
    sc.window_start = t
    for i in range(5):
        sc.allow(now=t + 9.9 + i * 0.001)
    print(f"  after 5 at t+9.9: curr={sc.curr_count} prev={sc.prev_coun
t}")
    for i in range(5):
        ok = sc.allow(now=t + 10.1 + i * 0.001)
        print(f"    request {i+1} at t+10.1: {'allowed' if ok else 'rej
ected'}  est={sc.estimated_count(now=t+10.1+i*0.001):.2f}")
```

8.3 Token bucket and leaky bucket — shaping vs policing

These two are often confused but have opposite burst behavior.

Token bucket — allows bursts, enforces average rate

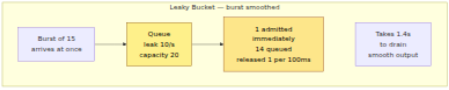
A bucket holds up to `capacity` tokens. Tokens are added at a fixed `refill_rate` (tokens/s). Each request consumes one token (or N tokens for weighted requests). If the bucket is empty, the request is rejected (or queued).

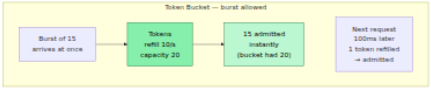
- **Burst-friendly:** If the bucket is full, up to `capacity` requests can pass instantly — then the limiter throttles to `refill_rate`. This models real capacity well: downstream can handle short bursts but not sustained overload.
- **Parameters:** `capacity` controls max burst; `refill_rate` controls sustained rate. Setting `capacity = refill_rate` gives a smooth rate with no burst; `capacity >> refill_rate` allows large bursts (e.g., 100 burst with 10/s sustained — common for user-facing APIs where interactive bursts are expected).

Leaky bucket — smooths bursts, enforces constant output rate

Requests enter a queue (the bucket). The bucket leaks at a constant rate — one request released per `1/rate` interval. If the bucket is full, new requests overflow (are rejected).

- **Burst-hostile:** Even if 100 requests arrive at once, they are released one at a time at the leak rate. This smooths traffic for downstream that cannot tolerate any burst (e.g., a legacy database with strict connection limits).
- **Trade-off:** Adds latency — bursty arrivals are queued rather than admitted immediately.





When to use which:

Criterion	Token bucket	Leaky bucket
Burst handling	Admits bursts up to capacity	Queues/smooths bursts
Added latency	None for admitted requests	Queuing delay for burst arrivals

Criterion	Token bucket	Leaky bucket
Downstream tolerance	Handles bursts	Cannot handle bursts
Typical use	API rate limiting (user-facing, bursty)	Traffic shaping toward fragile downstream (DB, legacy service)
Analogy	Bouncer with a burst allowance	Funnel that drips at constant rate

In practice, most backend rate limiters are token buckets (or GCRA, which is equivalent — see next section) because user traffic is inherently bursty and downstream services are typically provisioned for burst capacity.

```python
# Token bucket and leaky bucket — production-ready implementations
from __future__ import annotations
import time
import threading
from collections import deque


class TokenBucket:
    """Thread-safe token bucket. Tokens are floats to support
fractional rates."""
    def __init__(self, capacity: float, refill_per_s: float):
        self.capacity = capacity
        self.refill_per_s = refill_per_s
        self._tokens = capacity  # start full
        self._last_refill = time.monotonic()
        self._lock = threading.Lock()

    def _refill(self, now: float) -> None:
        elapsed = now - self._last_refill
        if elapsed > 0:
            self._tokens = min(self.capacity, self._tokens + elapsed *
self.refill_per_s)
            self._last_refill = now

    def allow(self, tokens: float = 1.0, now: float | None = None) -> b
ool:
        """Try to consume tokens. Returns True if admitted."""
        if now is None:
            now = time.monotonic()
        with self._lock:
            self._refill(now)
            if self._tokens >= tokens:
                self._tokens -= tokens
                return True
            return False

    def allow_with_wait(self, tokens: float = 1.0, now: float | None =
None) -> float:
        """Returns 0 if admitted immediately, else seconds to wait."""
        if now is None:
            now = time.monotonic()
        with self._lock:
            self._refill(now)
            if self._tokens >= tokens:
                self._tokens -= tokens
                return 0.0
            needed = tokens - self._tokens
            return needed / self.refill_per_s

    @property
```

```
    def available(self) -> float:
        with self._lock:
            self._refill(time.monotonic())
            return self._tokens


class LeakyBucket:
    """Leaky bucket — queue with constant leak rate.
    enqueue() returns True if accepted, False if bucket full
(overflow).
    leak() / drain() releases items at the leak rate."""
    def __init__(self, capacity: int, leak_per_s: float):
        self.capacity = capacity
        self.leak_per_s = leak_per_s
        self._queue: deque[float] = deque()  # arrival timestamps
        self._last_leak = time.monotonic()
        self._lock = threading.Lock()

    def _leak(self, now: float) -> None:
        """Remove items that have leaked out by now."""
        elapsed = now - self._last_leak
        n_leak = int(elapsed * self.leak_per_s)
        for _ in range(min(n_leak, len(self._queue))):
            self._queue.popleft()
        if n_leak > 0:
            # Advance last_leak by the time corresponding to items
actually leaked
            leaked = min(n_leak, len(self._queue) + n_leak)  #
approximate
            self._last_leak += n_leak / self.leak_per_s

    def try_add(self, now: float | None = None) -> bool:
        """Try to enqueue a request. False = bucket full, request
rejected."""
        if now is None:
            now = time.monotonic()
        with self._lock:
            self._leak(now)
            if len(self._queue) < self.capacity:
                self._queue.append(now)
                return True
            return False

    def queue_depth(self, now: float | None = None) -> int:
        if now is None:
            now = time.monotonic()
        with self._lock:
            self._leak(now)
            return len(self._queue)
```

```
# Simpler leaky bucket variant — time-based next-allowed calculation
(no queue storage)
class LeakyBucketSimple:
    """Leaky bucket as next-allowed timestamp — O(1) memory, no
queue."""
    def __init__(self, leak_per_s: float, capacity: int):
        self.leak_interval = 1.0 / leak_per_s
        self.capacity = capacity
        self.leak_per_s = leak_per_s
        # next_allowed is when the bucket will have space for one more
item
        self._next_allowed: float = 0.0
        self._queue_time: float = 0.0  # total queuing delay of items
in bucket
        self._lock = threading.Lock()

    def try_add(self, now: float | None = None) -> tuple[bool, float]:
        """Returns (admitted, wait_s). If admitted, wait_s is queuing
delay."""
        if now is None:
            now = time.monotonic()
        with self._lock:
            # Bucket state: how many items are queued ahead of this one
            # Approximate queue length from timing
            if now >= self._next_allowed:
                # Bucket has drained — admit immediately
                self._next_allowed = now + self.leak_interval
                return True, 0.0
            # Bucket has items — check if there is room
            queue_len = (self._next_allowed - now) / self.leak_interval
            if queue_len >= self.capacity:
                return False, 0.0  # overflow
            wait = self._next_allowed - now
            self._next_allowed += self.leak_interval
            return True, wait


if __name__ == "__main__":
    # Token bucket burst demo
    print("=== Token bucket: burst then throttle ===")
    tb = TokenBucket(capacity=10, refill_per_s=5)
    t = time.monotonic()
    # Burst of 15 at once — only 10 admitted (capacity)
    admitted = sum(1 for _ in range(15) if tb.allow(now=t))
    print(f"  burst 15 at t=0: admitted={admitted} (capacity=10)")
    # 0.4s later: 2 tokens refilled (5/s * 0.4 = 2)
    admitted2 = sum(1 for _ in range(5) if tb.allow(now=t + 0.4))
    print(f"  5 more at t+0.4s: admitted={admitted2} (refilled ~2)")
    print(f"  wait for 1 token: {tb.allow_with_wait(now=t+0.4):.3f}
s\n")
```

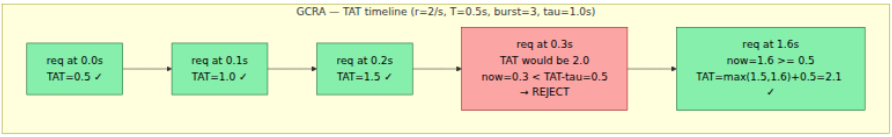
```
    # Leaky bucket smoothing demo
    print("=== Leaky bucket: smoothing ===")
    lb = LeakyBucketSimple(leak_per_s=10, capacity=20)
    t2 = time.monotonic()
    for i in range(5):
        ok, wait = lb.try_add(now=t2 + i * 0.01)  # 5 arrive in 40ms
        print(f"  request {i+1} at t+{i*10}ms: {'admitted' if ok else '
rejected'} wait={wait*1000:.1f}ms")
    # After burst, next request 50ms later still queued behind earlier
ones
    ok, wait = lb.try_add(now=t2 + 0.05)
    print(f"  request at t+50ms: {'admitted' if ok else 'rejected'}
wait={wait*1000:.1f}ms")
```

8.4 GCRA — the unified model

The **Generic Cell Rate Algorithm (GCRA)**, also called the virtual scheduling algorithm, unifies token bucket and leaky bucket into a single formulation. It tracks a single state variable — the **Theoretical Arrival Time (TAT)** — and needs no background refill thread.

```
Parameters:
  limit r  — sustained rate (e.g., 10 req/s)
  burst b  — max burst size
  emission interval T = 1/r
  burst offset tau = (b - 1) * T   (often written as burst = tau / T +
1)

State:
  TAT — theoretical arrival time of the next expected request

On request at time now:
  if TAT is unset:  TAT = now; allow
  else:
    if now < TAT - tau:  reject  (too early — would exceed burst)
    else:
      TAT = max(TAT, now) + T
      allow
```

GCRA with `tau = 0` is a strict rate limiter (no burst — exactly `1/T` spacing). With `tau > 0` it allows bursts of `b` requests. The algorithm is O(1) per request, O(1) memory, and trivially serializable to Redis or memcached with a single `GET` / `SET` or Lua script — which is why it powers many distributed limiters (e.g., `redis-cell`, Cloudflare's limiter).


GCRA — TAT timeline (r=2/s, T=0.5s, burst=3, tau=1.0s)
req at 0.0s
TAT=0.5 ✓
req at 0.1s
TAT=1.0 ✓
req at 0.2s
TAT=1.5 ✓
req at 0.3s
TAT would be 2.0
now=0.3 < TAT-tau=0.5
→ REJECT
req at 1.6s
now=1.6 >= 0.5
TAT=max(1.5,1.6)+0.5=2.1
✓

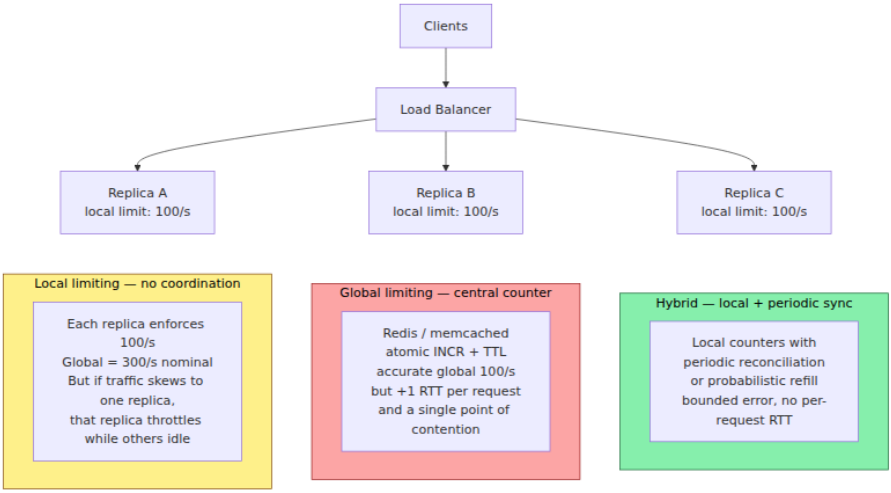
```python
# GCRA — Generic Cell Rate Algorithm (virtual scheduling)
from __future__ import annotations
import time

class GCRA:
    """GCRA rate limiter. Equivalent to token bucket with O(1) state.

    Args:
        rate_per_s: sustained rate (requests per second).
        burst: max burst size (number of requests that can arrive
simultaneously).
    """
    def __init__(self, rate_per_s: float, burst: int = 1):
        if rate_per_s <= 0 or burst < 1:
            raise ValueError("rate_per_s > 0 and burst >= 1 required")
        self.period = 1.0 / rate_per_s       # T — emission interval
        self.tau = (burst - 1) * self.period  # burst tolerance
        self.burst = burst
        self.rate_per_s = rate_per_s
        self.tat: float | None = None         # theoretical arrival
time

    def allow(self, now: float | None = None) -> tuple[bool, float]:
        """Check request at time now.
        Returns (allowed, retry_after_s). retry_after is 0 if
allowed."""
        if now is None:
            now = time.monotonic()
        if self.tat is None:
            self.tat = now + self.period
            return True, 0.0
        # How early is this request relative to TAT?
        # If now < TAT - tau, the burst allowance is exhausted — reject
        earliest = self.tat - self.tau
        if now < earliest:
            return False, earliest - now
        # Admit: advance TAT
        self.tat = max(self.tat, now) + self.period
        return True, 0.0

    def peek_wait(self, now: float | None = None) -> float:
        """How long until the next request would be allowed (0 if
now)."""
        if now is None:
            now = time.monotonic()
        if self.tat is None:
            return 0.0
        earliest = self.tat - self.tau
        return max(0.0, earliest - now)
```

```
if __name__ == "__main__":
    # GCRA: 2 req/s, burst 3
    print("=== GCRA r=2/s burst=3 ===")
    g = GCRA(rate_per_s=2, burst=3)
    t0 = 1000.0
    for offset in [0.0, 0.1, 0.2, 0.3, 0.4, 1.6, 1.7, 2.2]:
        ok, wait = g.allow(now=t0 + offset)
        status = "ALLOW" if ok else f"DENY (retry in {wait:.2f}s)"
        print(f"  t+{offset:.1f}s  {status}  TAT={g.tat - t0:.2f}s
ahead of t0")

    # Equivalence check: GCRA burst=1 should behave like strict 1-per-
period
    print("\n=== GCRA burst=1 (strict spacing) ===")
    g2 = GCRA(rate_per_s=10, burst=1)  # 10/s = 100ms spacing
    g2.allow(now=t0)
    for offset in [0.05, 0.10, 0.15]:
        ok, wait = g2.allow(now=t0 + offset)
        print(f"  t+{offset:.2f}s  {'ALLOW' if ok else f'DENY wait={wai
t*1000:.0f}ms'}")
```

Why GCRA matters for distributed systems. Token bucket requires periodic refill — either a background thread or lazy refill on each request with floating-point arithmetic. GCRA's TAT is a single timestamp that can be stored in Redis as a string, updated atomically with a Lua script, and requires no background work. The state is also easy to reason about for debugging: `TAT - now` tells you exactly how far ahead of the ideal schedule the client is.

8.5 Distributed rate limiting — local, global, and in between

A single-node limiter is accurate but useless when traffic arrives at N replicas behind a load balancer. Three strategies exist, with a clear trade-off between accuracy and coordination cost:



Strategy 1 — Local limiting (divide the quota)

Give each replica `limit / N` tokens. No coordination. Simple, fast (no network), but inaccurate under skew: if one replica receives 80% of traffic, it throttles aggressively while others have spare capacity. Over-provisioning the divisor (e.g., `limit / N * 1.5`) trades accuracy for availability — common when strict accuracy is not required.

Strategy 2 — Centralized counter (Redis / memcached)

Every request does an atomic increment against a central counter (Redis `INCR` + `EXPIRE`, or a GCRA TAT stored as a key). Accurate global limit, but adds 0.5-2 ms per request (Redis RTT) and creates a hotspot — a single Redis instance handling 100K increments/s becomes the bottleneck. Mitigations: sharding by key hash, pipeline batching, or using Redis Cluster.

Strategy 3 — Hybrid / probabilistic (the production default at scale)

Replicas maintain local buckets and periodically sync with a central coordinator, or use a probabilistic algorithm that bounds error without per-request coordination. Examples:

- **Token bucket with periodic sync:** Each replica gets a local bucket of size `limit/N`. A coordinator redistributes unused tokens every 100 ms. Bounded staleness, no per-request RTT.
- **Sliding window counter in Redis with local cache:** Read the global counter asynchronously, enforce locally, and accept bounded over-admission (typically < 5%).
- **Cell-based GCRA in Redis** (`redis-cell` / `CL.THROTTLE`): Single Lua script per request — atomic TAT update — but pipelined and sharded.

```python
# Distributed rate limiting patterns — local, central (simulated), and
hybrid
from __future__ import annotations
import time
import threading
from collections import defaultdict

# -- Simulated central store (stands in for Redis) --

class CentralStore:
    """In-memory stand-in for Redis. Thread-safe INCR/GET/SET with
TTL."""
    def __init__(self):
        self._data: dict[str, tuple[float, float | None]] = {}  # key
-> (value, expire_at)
        self._lock = threading.Lock()

    def incr(self, key: str, window_s: float) -> int:
        now = time.monotonic()
        with self._lock:
            val, exp = self._data.get(key, (0, None))
            if exp is not None and now >= exp:
                val, exp = 0, None
            val += 1
            if exp is None:
                exp = now + window_s
            self._data[key] = (val, exp)
            return int(val)

    def get(self, key: str) -> float:
        now = time.monotonic()
        with self._lock:
            val, exp = self._data.get(key, (0, None))
            if exp is not None and now >= exp:
                return 0
            return val

    # GCRA TAT stored as a float timestamp
    def gcra_check(self, key: str, now: float, period: float, tau: floa
t) -> tuple[bool, float]:
        with self._lock:
            tat_raw, exp = self._data.get(key, (None, None))  # type:
ignore[assignment]
            tat: float | None = tat_raw  # type: ignore[assignment]
            if tat is None:
                self._data[key] = (now + period, None)
                return True, 0.0
            earliest = tat - tau
            if now < earliest:
                return False, earliest - now
```

```
            new_tat = max(tat, now) + period
            self._data[key] = (new_tat, None)
            return True, 0.0


class CentralWindowLimiter:
    """Centralized fixed-window limiter backed by CentralStore (Redis
pattern)."""
    def __init__(self, store: CentralStore, limit: int, window_s:
float):
        self.store = store
        self.limit = limit
        self.window_s = window_s

    def allow(self, key: str) -> bool:
        count = self.store.incr(f"rl:{key}", self.window_s)
        return count <= self.limit


class LocalShardLimiter:
    """Local limiter that shards the global limit across N replicas."""
    def __init__(self, global_limit: int, num_replicas: int, window_s:
float):
        self.local_limit = max(1, global_limit // num_replicas)
        self.window_s = window_s
        self._counters: dict[str, FixedWindowLimiter] = {}
        self._lock = threading.Lock()

    def allow(self, key: str, now: float | None = None) -> bool:
        with self._lock:
            if key not in self._counters:
                self._counters[key] = FixedWindowLimiter(self.local_lim
it, self.window_s)
            return self._counters[key].allow(now=now)


class FixedWindowLimiter:
    def __init__(self, limit: int, window_s: float):
        self.limit = limit
        self.window_s = window_s
        self.count = 0
        self.window_start = time.monotonic()
    def allow(self, now: float | None = None) -> bool:
        if now is None:
            now = time.monotonic()
        if now - self.window_start >= self.window_s:
            self.count = 0
            self.window_start = now
        if self.count < self.limit:
            self.count += 1
            return True
```

```
        return False


if __name__ == "__main__":
    # Simulate 3 replicas handling skewed traffic for key "user:42"
    # Global limit 9/10s, 3 replicas -> local limit 3 each
    print("=== Local sharding under skew ===")
    local = LocalShardLimiter(global_limit=9, num_replicas=3,
window_s=10.0)
    t = time.monotonic()
    # Replica A gets 7 requests (skewed), B gets 2, C gets 1 — total
10, over global 9 but local misses it
    replica_a = sum(1 for _ in range(7) if local.allow("user:42",
now=t + 0.001 * _))
    # Use separate limiter instances to simulate separate replicas
    lim_a = FixedWindowLimiter(3, 10.0); lim_b = FixedWindowLimiter(3,
10.0); lim_c = FixedWindowLimiter(3, 10.0)
    lim_a.window_start = lim_b.window_start = lim_c.window_start = t
    a_ok = sum(1 for _ in range(7) if lim_a.allow(now=t))
    b_ok = sum(1 for _ in range(2) if lim_b.allow(now=t))
    c_ok = sum(1 for _ in range(1) if lim_c.allow(now=t))
    print(f"  replica A (7 req): {a_ok} allowed (local limit 3) — 4
rejected due to local cap")
    print(f"  replica B (2 req): {b_ok} allowed")
    print(f"  replica C (1 req): {c_ok} allowed")
    print(f"  global: {a_ok+b_ok+c_ok}/10 allowed, but global limit is
9 — 1 under-admitted due to skew")
    print(f"  (a central limiter would have allowed exactly 9)\n")

    print("=== Centralized GCRA (accurate) ===")
    store = CentralStore()
    t0 = time.monotonic()
    period, tau = 1.0, 2.0  # 1/s, burst 3
    for i in range(5):
        ok, wait = store.gcra_check("user:42:gcra", t0 + i * 0.1, perio
d, tau)
        print(f"  req {i+1} at t+{i*0.1:.1f}s: {'ALLOW' if ok else f'DE
NY wait={wait:.2f}s'}")
    # All 3 replicas share the same TAT — accurate globally

    print("\n=== Redis Lua pattern (pseudo-code) ===")
    print("""
    -- GCRA in Redis Lua (atomic TAT update) — what redis-cell /
CL.THROTTLE does:
    -- KEYS[1] = rate limit key, ARGV[1] = now, ARGV[2] = period,
ARGV[3] = tau
    local tat = redis.call('GET', KEYS[1])
    if not tat then
        redis.call('SET', KEYS[1], ARGV[1] + ARGV[2], 'PX', 60000)
        return {1, 0}  -- allowed
    end
```

```
    tat = tonumber(tat)
    local earliest = tat - tonumber(ARGV[3])
    if tonumber(ARGV[1]) < earliest then
        return {0, earliest - tonumber(ARGV[1])}  -- denied
    end
    local new_tat = math.max(tat, tonumber(ARGV[1])) +
tonumber(ARGV[2])
    redis.call('SET', KEYS[1], new_tat, 'PX', 60000)
    return {1, 0}  -- allowed
    """)
```

Choosing a distributed strategy:

Scenario	Recommended approach	Why
Few replicas (<=5), strict quota (billing, abuse prevention)	Centralized (Redis GCRA or sliding window)	Accuracy matters more than RTT; hotspot is manageable
Many replicas (50+), generous quota, latency-sensitive	Local sharding with over-provisioning	No per-request RTT; bounded inaccuracy is acceptable
Many replicas, strict quota	Hybrid (local + periodic sync) or sharded central with pipelining	Balance accuracy and overhead; see Envoy's rate limit service
Global limit (all regions)	Hierarchical — regional limit + global async reconciliation	Cross-region RTT is too high for per-request central check

---

8.6 Concurrency limiting — a different dimension

Rate limiting controls **requests per time window**. Concurrency limiting controls **simultaneous in-flight requests**. They are complementary:

- Rate limiting protects against **throughput** overload (too many requests per second).
- Concurrency limiting protects against **resource exhaustion** from slow requests (too many concurrent handlers holding memory, connections, or threads).

A system can be within its rate limit but still overloaded if each request takes 10 s and 1000 arrive concurrently — each holds a thread and 2 MB of heap, exhausting both.

```
# Concurrency limiter — semaphore-based with queuing and timeout
from __future__ import annotations
import threading
import time
from collections import deque
from contextlib import contextmanager
from typing import Generator


class ConcurrencyLimiter:

"""Bound concurrent executions. Supports try_acquire (non-blocking) and
    acquire with timeout. Tracks utilization for load shedding."""
    def __init__(self, max_concurrent: int, max_queue: int = 0):
        self.max_concurrent = max_concurrent
        self.max_queue = max_queue
        self._sem = threading.Semaphore(max_concurrent)
        self._active = 0
        self._rejected = 0
        self._lock = threading.Lock()

    def try_acquire(self) -> bool:
        """Non-blocking acquire. Returns False if at limit."""
        acquired = self._sem.acquire(blocking=False)
        if acquired:
            with self._lock:
                self._active += 1
        else:
            with self._lock:
                self._rejected += 1
        return acquired

    def acquire(self, timeout: float = 0.0) -> bool:
        """Blocking acquire with timeout. 0 = non-blocking."""
        acquired = self._sem.acquire(timeout=timeout)
        if acquired:
            with self._lock:
                self._active += 1
        else:
            with self._lock:
                self._rejected += 1
        return acquired

    def release(self) -> None:
        with self._lock:
            self._active -= 1
        self._sem.release()

    @contextmanager
    def guard(self, timeout: float = 0.0) -> Generator[bool, None,
```

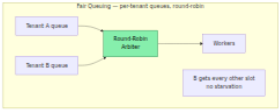
```
None]:
        acquired = self.acquire(timeout=timeout)
        try:
            yield acquired
        finally:
            if acquired:
                self.release()

    @property
    def utilization(self) -> float:
        with self._lock:
            return self._active / self.max_concurrent if self.max_concu
rrent else 0.0

    @property
    def stats(self) -> dict[str, int | float]:
        with self._lock:
            return {"active": self._active, "rejected": self._rejected,
                    "utilization": self._active / self.max_concurrent i
f self.max_concurrent else 0}


class AdaptiveConcurrencyLimiter:
    """Vegas-style adaptive limiter — adjusts limit based on latency
gradient.
    Inspired by Netflix's concurrency-limits-java (Vegas) and TCP
Vegas.

    Increases limit when latency is flat (more concurrency helps
throughput).
    Decreases limit when latency gradient rises (queuing detected)."""
    def __init__(self, initial_limit: int = 20, min_limit: int = 5, max
_limit: int = 200):
        self.limit = initial_limit
        self.min_limit = min_limit
        self.max_limit = max_limit
        self._in_flight = 0
        self._rtt_no_load: float | None = None
        self._lock = threading.Lock()
        self._samples: deque[float] = deque(maxlen=20)

    def acquire(self) -> bool:
        with self._lock:
            if self._in_flight >= self.limit:
                return False
            self._in_flight += 1
            return True

    def release(self, rtt_s: float) -> None:
        with self._lock:
            self._in_flight -= 1
```

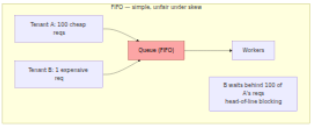
```
            self._samples.append(rtt_s)
            if self._rtt_no_load is None or rtt_s < self._rtt_no_load:
                self._rtt_no_load = rtt_s

            if len(self._samples) < 5:
                return

            # Gradient: compare recent p50 to no-load RTT
            recent = sorted(self._samples)[len(self._samples) // 2]
            assert self._rtt_no_load is not None
            gradient = (recent - self._rtt_no_load) /
self._rtt_no_load if self._rtt_no_load > 0 else 0

            # Vegas logic: if gradient > threshold, we are queuing —
reduce limit
            # if gradient is flat and we have headroom, increase limit
            if gradient > 0.3 and self.limit > self.min_limit:
                self.limit = max(self.min_limit, self.limit - 1)
            elif gradient < 0.1 and self.limit < self.max_limit:
                # Only increase if not already at high utilization
                if self._in_flight > self.limit * 0.5:
                    self.limit = min(self.max_limit, self.limit + 1)

    @property
    def current_limit(self) -> int:
        with self._lock:
            return self.limit


if __name__ == "__main__":
    print("=== Concurrency limiter ===")
    cl = ConcurrencyLimiter(max_concurrent=3)
    # Acquire 3 — all succeed
    for i in range(3):
        print(f"  acquire {i+1}: {cl.try_acquire()} active={cl.stats['a
ctive']}")
    print(f"  acquire 4 (over limit): {cl.try_acquire()} (should be
False)")
    cl.release()
    print(f"  after release, acquire: {cl.try_acquire()} (should be
True)")
    cl.release(); cl.release(); cl.release()  # cleanup

    print("\n=== Adaptive limiter (Vegas-style) ===")
    al = AdaptiveConcurrencyLimiter(initial_limit=10, min_limit=5, max_
limit=50)
    # Simulate: low RTT initially, then RTT rises (overload), then
recovers
    for rtt_ms in [10, 10, 11, 10, 12, 30, 45, 50, 48, 12, 11, 10, 10]:
        al.acquire()
        al.release(rtt_ms / 1000)
```

```
            print(f"  rtt={rtt_ms:>3}ms  limit={al.current_limit:>3}
rtt_no_load={al._rtt_no_load*1000:.0f}ms")  # type: ignore[operator]
```

8.7 Scheduling — ordering admitted work

Once requests are admitted, the order in which they execute determines fairness, tail latency, and SLO attainment.





FIFO — the default, and its failure mode

FIFO is simple and preserves ordering, but a single tenant or request class that floods the queue blocks everyone — classic head-of-line blocking. In backend systems this manifests as one customer's bulk import starving interactive requests for all other customers.

Strict priority — urgent first, but watch for starvation

Always serve the highest-priority queue that has work. Guarantees low latency for high-priority work, but low-priority work can starve indefinitely if high-priority load is sustained. Requires starvation mitigation (aging, priority promotion after a deadline).

Fair queuing and weighted fair queuing (WFQ)

Give each tenant (or class) its own queue and serve them round-robin. WFQ assigns weights — a tenant with weight 2 gets twice the service of weight 1. This provides **max-min fairness**: no tenant can starve another, and spare capacity is redistributed. Used in network schedulers, Envoy's per-route queuing, and multi-tenant job schedulers.

Deficit Round Robin (DRR) — fair queuing for variable-cost requests

Round-robin is unfair when requests have different costs (one tenant's requests take 10 ms, another's take 500 ms). DRR assigns each queue a **quantum** (deficit counter). Each round, a queue's deficit increases by its quantum; it can dequeue requests while its deficit covers the request

cost. Expensive requests consume more deficit, so throughput equalizes by cost rather than request count.

```python
```

Scheduling disciplines — FIFO, priority, WFQ, DRR

from **future** import annotations import heapq from collections import deque, defaultdict from dataclasses import dataclass, field from typing import Any

@dataclass(order=True) class PrioritizedItem: priority: int seq: int item: Any = field(compare=False)

class PriorityQueue: """Strict priority queue — lower number = higher priority.""" def **init**(self): self._heap: list[PrioritizedItem] = [] self._seq = 0

```
def push(self, item: Any, priority: int = 0) -> None:
    heapq.heappush(self._heap, PrioritizedItem(priority, self._seq, ite
m))
    self._seq += 1

def pop(self) -> Any | None:
    if not self._heap:
        return None
    return heapq.heappop(self._heap).item

def __len__(self) -> int:
    return len(self._heap)
```

class FairQueue: """Per-tenant fair queuing — round-robin across tenant queues.""" def **init**(self): self._queues: dict[str, deque[Any]] = defaultdict(deque) self._tenants: list[str] = [] # round-robin order self._next_idx = 0

```
def push(self, tenant: str, item: Any) -> None:
    if tenant not in self._queues or not self._queues[tenant]:
        if tenant not in self._tenants:
            self._tenants.append(tenant)
    self._queues[tenant].append(item)

def pop(self) -> tuple[str, Any] | None:
    if not self._tenants:
        return None
    # Round-robin scan for next non-empty queue
    for _ in range(len(self._tenants)):
        tenant = self._tenants[self._next_idx % len(self._tenants)]
        self._next_idx += 1
        if self._queues[tenant]:
            item = self._queues[tenant].popleft()
            if not self._queues[tenant]:
                # Remove empty queue from rotation (re-added on next p
ush)
                self._tenants.remove(tenant)
                if self._next_idx > len(self._tenants):
                    self._next_idx = 0
            return tenant, item
    return None

def __len__(self) -> int:
    return sum(len(q) for q in self._queues.values())
```

class WFQ: """Weighted Fair Queuing — tenants with higher weight get proportionally more service.""" def **init**(self, weights: dict[str, int]): self.weights = weights self._queues: dict[str, deque[Any]] = defaultdict(deque) # Virtual time per tenant — WFQ schedules the tenant with smallest virtual finish time self._virtual_time: dict[str, float] = defaultdict(float) self._global_virtual = 0.0

```
def push(self, tenant: str, item: Any, cost: float = 1.0) -> None:
    self._queues[tenant].append((item, cost))

def pop(self) -> tuple[str, Any] | None:
    # Find tenant with smallest virtual finish time among non-empty qu
eues
    best_tenant: str | None = None
    best_vtime = float("inf")
    for tenant, q in self._queues.items():
        if q and self._virtual_time[tenant] < best_vtime:
            best_vtime = self._virtual_time[tenant]
            best_tenant = tenant
    if best_tenant is None:
        return None
    item, cost = self._queues[best_tenant].popleft()
    weight = self.weights.get(best_tenant, 1)
    # Virtual finish time advances by cost/weight
    self._virtual_time[best_tenant] += cost / weight
    return best_tenant, item
```

@dataclass class DRRQueue: quantum: int deficit: int = 0 queue: deque[tuple[Any, int]] = field(default_factory=deque) # (item, cost)

class DeficitRoundRobin: """DRR — fair queuing for variable-cost requests.""" def **init**(self, quanta: dict[str, int]): self.queues: dict[str, DRRQueue] = {t: DRRQueue(q) for t, q in quanta.items()} self._order: list[str] = list(quanta.keys())

```
def push(self, tenant: str, item: Any, cost: int) -> None:
    if tenant not in self.queues:
        self.queues[tenant] = DRRQueue(quantum=10)
        self._order.append(tenant)
    self.queues[tenant].queue.append((item, cost))

def pop(self) -> tuple[str, Any] | None:
    # One DRR round: each queue gets quantum added to deficit, then de
queues while deficit >= cost
    # For simplicity, we do one dequeue per call, advancing round-robi
n
    for _ in range(len(self._order)):
        for tenant in list(self._order):
            q = self.queues[tenant]
            if not q.queue:
                continue
            q.deficit += q.quantum
            item, cost = q.queue[0]
            if q.deficit >= cost:
                q.queue.popleft()
                q.deficit -= cost
                return tenant, item
            # Not enough deficit — carry over to next round (don't res
et deficit)
    return None

def drain_all(self) -> list[tuple[str, Any]]:
    out: list[tuple[str, Any]] = []
    while True:
        item = self.pop()
        if item is None:
            # Check if any queue still has items but needs more defici
t rounds
            if any(q.queue for q in self.queues.values()):
                continue
            break
        out.append(item)
    return out
```

-- Deadline-aware scheduling (EDF) --

@dataclass(order=True) class DeadlineItem: deadline: float seq: int item: Any = field(compare=False)

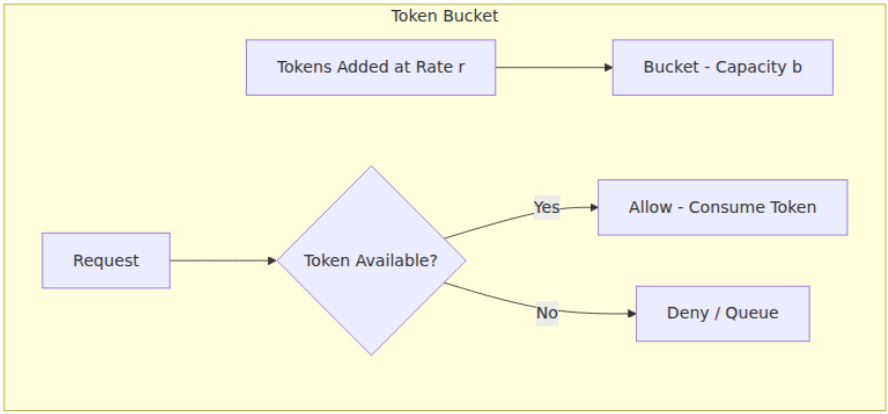
class EDFScheduler: """Earliest Deadline First — always run the request with the nearest deadline. Optimal for meeting deadlines on a single machine (if any schedule can, EDF can)."""" def **init**(self): self._heap: list[DeadlineItem] = [] self._seq = 0

```
def push(self, item: Any, deadline: float) -> None:
    heapq.heappush(self._heap, DeadlineItem(deadline, self._seq, item))
    self._seq += 1

def pop(self) -> Any | None:
    if not self._heap:
        return None
    return heapq.heappop(self._heap).item

def pop_if_expired(self, now: float) -> list[Any]:
    """Remove and return all items whose deadline has passed (for load
shedding)."""
    expired: list[Any] = []
    while self._heap and self._heap[0].deadline <= now:
        expired.append(heapq.heappop(self._heap).item)
    return expired
```

if **name** == "**main**": print("=== Fair queue (round-robin) ===") fq = FairQueue() for i in range(4): fq.push("tenant-A", f"A-{i}") fq.push("tenant-B", "B-1") order: list[str] = [] while len(fq) > 0: tenant, item = fq.pop() # type: ignore[misc] order.append(f"{tenant}:{item}") print(f" dequeue order: {order}") print(f" tenant-B not starved despite tenant-A having 4x items\n")

```
print("=== WFQ (weights A:3, B:1) ===")
wfq = WFQ(weights={"A": 3, "B": 1})
for i in range(6):
    wfq.push("A", f"A-{i}")
for i in range(6):
    wfq.push("B", f"B-{i}")
order2: list[str] = []
for _ in range(8):
    r = wfq.pop()
    if r:
        order2.append(f"{r[0]}:{r[1]}")
print(f"  first 8 dequeued: {order2}")
print(f"  A (weight 3) gets ~3x B's throughput\n")

print("=== DRR (variable cost) ===")
drr = DeficitRoundRobin(quanta={"interactive": 10, "batch": 10})
# Interactive: cheap (cost 2), batch: expensive (cost 9)
for i in range(5):
    drr.push("interactive", f"int-{i}", cost=2)
for i in range(5):
    drr.push("batch", f"batch-{i}", cost=9)
print(f"  drain order: {drr.drain_all()}")
print(f"  DRR interleaves fairly by cost, not just count\n")

print("=== EDF (deadline-aware) ===")
edf = EDFScheduler()
now = 1000.0
edf.push("req-A", deadline=now + 5.0)
edf.push("req-B", deadline=now + 1.0)
edf.push("req-C", deadline=now + 3.0)
print(f"  dequeue order: {[edf.pop() for _ in range(3)]}  (B earliest
deadline first)")
# Load shedding: drop expired
edf2 = EDFScheduler()
edf2.push("stale", deadline=now + 0.5)
edf2.push("fresh", deadline=now + 10.0)
expired = edf2.pop_if_expired(now + 1.0)
print(f"  expired at t+1.0: {expired} (stale dropped, fresh remains)")
```

Token bucket vs leaky bucket


Token Bucket
Tokens Added at Rate r
Bucket - Capacity b
Request
Token Available?
Yes
Allow - Consume Token
No
Deny / Queue